Appunti di Sistemi Operativi Laboratorio

Alessio Novel

Usare un ambiente linux: terminal and bash....2
Comandi....3
Ambiente e variabili....5
Subshell....7
Script / Batch....9
Eseguire e compilare su Linux: Docker, GCC and Makefile....12
Docker....12
GCC....20
Make....21
Le basi di C....25
Vettori e stringhe....32
Interazione con i file: streams and file descriptors....34
Streams....35
File descriptors....38
Canali standard....44
Piping con bash....44
Comunicazione con il kernel e processi: system calls e forking....45
Architettura....45
System calls....46
Forking....55
Inter-Process-Communication: signals....61
Errori in C....72
Process groups....72
IPC: pipes and fifos....74
Pipe anonime....75
FIFO....79
IPC: message queues....82
Gestire lavori multipli e la concorrenza: threads and mutexes....91
Threads....91
Mutex....99

Usare un ambiente linux: terminal and bash

Useremo Ubuntu 22.04.1 LTS

Il **terminale** è l'ambiente testuale di interazione con il sistema operativo.
Tipicamente è utilizzato come applicazione all'interno dell'ambiente grafico ed è possibile avviarne più istanze, pur essendo anche disponibile direttamente all'avvio (in questo caso normalmente in più istanze accessibili con la combinazione CTRL+ALT+Fx).

All'interno del terminale, l'interazione avviene utilizzando un'applicazione specifica in esecuzione al suo interno, comunemente detta **SHELL**. Quindi si può dire che il terminale è il programma che esegue la shell.
Essa propone un prompt per l'immissione diretta di comandi da tastiera e fornisce un feedback testuale. È anche possibile eseguire sequenze di comandi preorganizzate contenute in file testuali (chiamati script o batch). A seconda della modalità (diretta/script) alcuni comandi possono avere senso o meno o comportarsi in modo particolare.
L'insieme dei comandi e delle regole di composizione di una shell costituisce un linguaggio di programmazione orientato allo scripting.

Il terminale è dunque il programma che esegue la shell. In informatica, questo tipo di programma è chiamato: "wrapper", involucro, ovvero quel programma che ne ricopre un altro. Il terminale, quindi, ti permette di accedere alla shell e a lui, di fatto, non interessa quale shell sia installata.

Esistono numerose shell. **Bash** è una delle più utilizzate sui sistemi operativi linux e le altre shell sono comunque simili tra loro, ma hanno sempre qualche differenza (e anche comandi analoghi possono avere opzioni o comportamenti non identici). Tipicamente (almeno in sessioni non grafiche) al login un utente ha associata una shell particolare. Un paio di alternative sono: zsh (sul mio mac), ksh, sh, …

Portable **O**perating **S**ystem **I**nterface for Uni**x** (**POSIX**): è una famiglia di standard IEEE. Nel caso delle shell, in particolare, definisce una serie di regole e di comportamenti che possono favorire la portabilità (che però dipende anche da altri fattori del sistema!).
La shell *bash* soddisfa molti requisiti POSIX, ma presenta anche alcune differenze ed "estensioni" per agevolare, almeno in parte, la programmazione (ad esempio i costrutti per i confronti logici).

La shell attende un input dall'utente e al completamento (conferma con *INVIO*) lo elabora.
Per indicare l'attesa mostra all'utente un PROMPT, ovvero il simbolo che compare sul video per segnalare che il computer è pronto a ricevere il successivo comando (può essere modificato).
Fondamentalmente in una shell si individuano 3 canali:

- **Standard input** (tipicamente la tastiera), canale 0, detto *stdin*
- **Standard output** (tipicamente il video), canale 1, detto *stdout*
- **Standard error** (tipicamente il video), canale 2, detto *stderr*

Comandi

Solitamente un comando è identificato da una parola chiave cui possono seguire uno o più "argomenti" opzionali o obbligatori, accompagnati da un valore di riferimento (in questo caso gli argomenti hanno dei parametri) o meno (in questo caso gli argomenti hanno valore di "flag", ovvero attivo o non attivo) e di tipo posizionale (quindi la posizione dei parametri è influente) o nominale. A volte sono ripetibili.
Ad esempio: *ls -alh /tmp*
Gli argomenti nominali sono indicati con un trattino cui segue una voce (stringa alfanumerica) e talvolta presentano una doppia modalità di riferimento: breve con un trattino (tipicamente voce di un singolo carattere) e lunga con due trattini (tipicamente un termine mnemonico)

`app -h`	`app --help`

Vediamo un po' di caratteristiche:

- l'esecuzione dei comandi avviene "per riga" ovvero quello che si immette fino a *INVIO* (per i file batch/script l'argomento sarà trattato in modo approfondito successivamente)
- un **termine** (istruzione, argomento, opzione, etc.) è solitamente una stringa alfanumerica senza spazi
- spaziature multiple sono solitamente valide come singole e non sono significative se non per separare i termini
- è possibile solitamente usare gli apici singoli o doppi per forzare una sequenza (con spazi) come termine singolo
- gli spazi iniziali e finali di una riga collassano
- le righe vuote sono ignorate

È anche possibile utilizzare dei **commenti** da passare alla shell. L'unico modo formale è l'utilizzo del carattere # per cui esso e tutto ciò che segue fino al termine della riga è considerato un commento ed è sostanzialmente ignorato.

```
ls -la #-r this part is ignored
```

I commenti sono fondamentali quando si creano file batch (gli script).

I comandi possono essere "builtins" (ovvero funzionalità intrinseche dell'applicazione shell utilizzata) oppure "esterni" (ovvero applicazioni eseguibili che risiedono su disco).
Ecco alcuni comandi builtins fondamentali:

- *clear*: pulisci terminale
- *pwd*: print working directory
- *ls*: list, -*l* (più dettagli), -*a* (all, ad esempio anche i file nascosti)
- *cd*: change directory
- *wc*: word count di un file (facendo *ls | wc* listiamo i file con il loro word count)
- *date*: ottieni data corrente
- *cat*: mostra contenuto del file
- *echo*: stampa a video una stringa

- *alias* / *unalias*: per impostare/togliere alias sui comandi (e.g. *alias ciao="echo ciao come stai"*), gli alias sono temporanei (per renderli persistenti dobbiamo cambiare i file di impostazione della bash)
- *test*: esegue un confronto logico (il valore del risultato è invertito rispetto alla normale logica booleana)
  Con *echo $?* visualizziamo il codice di risposta dell'ultima operazione/programma eseguito
- *read*: leggere a video una variabile, ovvero assegnare un valore ad una variabile
- *file*: per determinare il tipo del file e i suoi dati
- *chown* / *chmod*: cambiare i permessi e il proprietario di un file
- *cp* / *mv*: copy e move
- *help*: visualizzare la lista dei comandi della shell
- *type*: info su un comando
- *grep*: evidenziare alcune parole in un output a video
- *function*: per creare una funzione

Con | possiamo concatenare i comandi (lo vedremo meglio successivamente)

Ogni comando lavora su un insieme di **canali**, come standard output o un file descriptor (l'argomento sarà trattato in modo approfondito in un blocco successivo).

```
ls # mostra output della cartella corrente

ls not-existent-item #mostra un messaggio d'errore
```

I canali possono essere **redirezionati** (anche in cascata), usando la sintassi *source*>*target*.

```
• ls 1>/tmp/out.txt 2>/tmp/err.txt
• ls not-existent-item 1>/tmp/all.txt 2>&1
```

Scrivendo 2>&1 redirezioniamo il canale 2 sullo stesso "target" del canale 1.

I vari tipi di redirezionamento sono:

- <
  Usato in questa maniera: *command*<*file.txt* invia il contenuto del file come input al comando (read only)
  ```
  mail -s "Subject" rcpt < content.txt (anziché interattivo)
  ```
- <>
  Uguale al funzionamento di < ma il file è aperto in read-write (usato raramente)
- >
  Come abbiamo già detto viene usato in questa maniera: *command source*>*target*.
  source può essere sottinteso (e valere 1) e *target* può essere un altro canale, con la sintassi &*n*
- >|
  Si comporta come > ma forza la sovrascrittura anche nel caso in cui sia bloccata nelle configurazioni

- >>
  Si comporta come > ma opera un append se la destinazione esiste
- <<
  here-document, consente all'utente di specificare un terminatore testuale, dopodiché accetta l'input interattivo fino alla ricezione di tale terminatore (rende interattivo ciò che non è interattivo). La modalità di inserimento terminerà non appena verrà inserita una riga contenente solo il termine di fine.
- <<<
  here-string, uguale a here-document ma consente di inserire una sola stringa (rende non interattivo ciò che è interattivo)

Esistono molte varianti e possibilità di combinazione dei vari operatori di redirezionamento.
Un caso molto utilizzato è la "soppressione" dell'output (utile per gestire solo side-effects, come ad esempio il codice di ritorno), e si fa redirezionando i canali su $/dev/null$, ad esempio:

`type command 1>/dev/null 2>&1` (per sapere se *command* esiste)

Ambiente e variabili

La shell può utilizzare delle **variabili** per memorizzare e recuperare certi valori.
I valori sono generalmente trattati come stringhe o interi: sono presenti anche semplici array (vettori di elementi).
Il formato del nome delle variabili è del tipo $\wedge[_[:alpha:]][_[:alnum:]]*\$$:

1. $\wedge[_[:alpha:]]$: Questa parte corrisponde all'inizio del nome della variabile e indica che il primo carattere può essere un underscore () o una lettera alfabetica.
2. $[_[:alnum:]]$*: Questa parte permette di avere zero o più caratteri successivi nel nome della variabile. I caratteri ammissibili sono underscore () o alfanumerici.
3. $\$$: Questo simbolo indica la fine del nome della variabile.

In sintesi, il formato richiede che il nome di una variabile inizi con un underscore o una lettera alfabetica e che i caratteri successivi siano underscore o alfanumerici.

Per impostare il valore di una variabile (set) si usa $variabile = valore$. Lo scope è generalmente quello del processo attuale, anteponendo "*export*" al nome della variabile si rende disponibile anche agli eventuali processi figli (quando abbiamo il concetto di processi multipli)
$export \quad variabile = valore$.

Per ottenere il valore di una variabile (get) si usa $\$ variabile$ o $\$\{variabile\}$ e viene eseguita una sostituzione letterale. Ad esempio se $abc = ciao$, il comando $echo \quad \$\{abc\} sd$ mostra $ciaosd$

La shell opera in un ambiente in cui ci sono alcuni riferimenti impostabili e utilizzabili attraverso l'uso delle cosiddette "variabili d'ambiente" con cui si intendono generalmente le variabili con un significato particolare per la shell stessa. Tra le variabili d'ambiente più comuni troviamo ad esempio:

- $SHELL$, contiene il riferimento alla shell corrente (path completo)

- *PATH*, contiene i percorsi in ordine di priorità in cui sono cercati i comandi esterni, separati da ":" (questo serve per poter eseguire gli eseguibili senza dover specificare il loro percorso completo ma semplicemente con il loro nome)
- *TERM*, contiene il tipo di terminale corrente
- *PWD*, contiene il percorso della cartella corrente
- *PS*1, contiene il prompt (si possono usare marcatori speciali)
- *HOME*, contiene la cartella principale dell'utente corrente

La variabile *PATH* contiene le cartelle che contengono eseguibili che possono essere lanciati senza dover specificare il percorso completo degli eseguibili.
Essendo variabili a tutti gli effetti, le variabili d'ambiente si impostano ed usano come le altre

Quando si immette un comando (o una sequenza di comandi) la shell analizza quanto inserito (ovvero esegue il **parsing**) e capisce quali termini sono nella posizione di comandi da eseguire. Se i comandi sono builtins ne esegue le funzionalità direttamente altrimenti cerca di individuare un corrispondente file eseguibile, questo è normalmente cercato nel file-system **solo e soltanto nei percorsi definiti dalla variabile PATH** a meno che non sia specificato un percorso (relativo o assoluto) nel qual caso viene utilizzato esso direttamente.
Dall'ultima osservazione discende che per un'azione abbastanza comune come lanciare un file eseguibile (non "installato", ovvero non inserito nelle cartelle puntate da *PATH*) nella cartella corrente occorre qualcosa come: ./*nomefile*

Gli **array** nelle shell sono delle sequenze di elementi eterogenei separati da spazi, è importante notare che gli array permettono di avere posizioni vuote, infatti esiste un modo per ottenere la lista degli indici "popolati".

- Definizione: `lista=("a" 1 "b" 2 "c" 3)` **separati da spazi!**
- Output completo: `${lista[@]}`
- Accesso singolo: `${lista[x]}` (0-based)
- Lista indici: `${!lista[@]}`
- Dimensione: `${#lista[@]}`
- Set elemento: `lista[x]=value`
- Append: `lista+=(value)`
- Sub array: `${lista[@]:s:n}` (from index s, length n)

La dimensione ritorna il numero degli indici popolati.

Ci sono due variabili non modificabili (la stessa sintassi lo impedisce dato che i nomi sarebbero *$* e *?*, non utilizzabili normalmente):

- *$$* : contiene il PID del processo attuale (il concetto di PID sarà approfondito successivamente)
- *$?* : contiene il codice di ritorno dell'ultimo comando eseguito

La riga dei comandi è elaborata con una serie di azioni "in sequenza" e poi rielaborata eventualmente più volte. Le azioni consistono in:

- Sostituzioni speciali della shell (es. “!” per accedere alla “history” dei comandi)
- Sostituzione variabili
- Elaborazione subshell

Le azioni sono svolte con questo ordine di priorità e poi l’intera riga è rielaborata (vedi Subshell).

È possibile concatenare più comandi in un'unica riga in varie maniere:

- **Concatenazione semplice**
  La sintassi è *comando*1 ; *comando*2 e consiste in una semplice esecuzione in sequenza
- **Concatenazione logica AND**
  La sintassi è *comando*1 && *comando*2 e l’esecuzione di *comando*2 procede solo se *comando*1 non fallisce, ovvero solo se *comando*1 ritorna codice zero (true)
- **Concatenazione logica OR**
  La sintassi è *comando*1 || *comando*2 e l’esecuzione di *comando*2 procede solo se *comando*1 fallisce, ovvero solo se *comando*1 ritorna codice non zero (false)
- **Concatenazione con piping**
  La sintassi è *comando*1 | *comando*2
  La concatenazione con gli operatori di piping “cattura” l’output del primo comando e lo passa in input al secondo
  Utilizzando | viene catturato solo *stdout*
  Utilizzando |& viene catturato sia *stdout* che *stderr*

  - `ls | wc -l` : cattura solo stdout
  - `ls |& wc -l` : cattura stdout e stderr

Il comando *ls* ha un comportamento atipico: il suo output di base è differente a seconda che il comando sia diretto al terminale o a un piping, questo perchè nel caso in cui l’output sia diretto al terminale formatta l’output in una certa maniera specifica con le colonne, altrimenti produce un output semplificato che facilita l'elaborazione successiva nel piping (internamente sfrutta *isatty*(*STDOUT_FILENO*) per capire se è diretto al terminale o meno)

Subshell

È possibile avviare una **subshell**, ossia un sottoambiente, in particolare raggruppando i comandi tra parentesi tonde (attenzione al fatto che le parentesi tonde sono utilizzate anche per definire array):

(… *comandi* …)

Solitamente si usa **catturare l’output** standard (*stdout*) di una subshell, e questo si può fare in due modi (attenzione agli spazi):

- $(… *comandi* …)
- ' … *comandi* … '

Le subshell sono dei sottoambienti usati per eseguire una serie di comandi e ritornare solo il risultato finale alla shell principale.

Ecco un esempio di comandi con subshell:

```
echo "/tmp" > /tmp/tmp.txt ; ls $(cat /tmp/tmp.txt)
```

i comandi sono eseguiti rispettando la sequenza:

- `echo "/tmp" > /tmp/tmp.txt` crea un file temporaneo con `"/tmp"`
- `ls $(cat /tmp/tmp.txt)` è prima eseguita la subshell:
  - `cat /tmp/tmp.txt` genera in stdout `"/tmp"` e poi con sostituzione:
  - `ls /tmp` mostra il contenuto della cartella `/tmp`

La sintassi base per una subshell è da non confondere con **l'espansione aritmetica** che utilizza le doppie parentesi tonde. All'interno delle doppie parentesi tonde si possono rappresentare varie espressioni matematiche inclusi assegnamenti e confronti.

Alcuni esempi:

```
(( a = 7 )) (( a++ )) (( a < 10 )) (( a = 3<10?1:0 ))

b=$((c+a))
```

I costrutti fondamentali per i **confronti logici** sono il comando *test* e i raggruppamenti tra parentesi quadre singole e doppie:

- *test* … e […]
  Sono built-in e sono equivalenti
- [[…]]
  È una coppia di shell-keywords

In tutti i casi il blocco di confronto genera il codice di uscita 0 in caso di successo, un valore differente (tipicamente 1) altrimenti.

I **builtins** sono sostanzialmente dei comandi il cui corpo d'esecuzione è incluso nell'applicazione shell direttamente (non sono eseguibili esterni) e quindi seguono sostanzialmente le "regole generali" dei comandi.
Le **shell-keywords** sono gestite come marcatori speciali così che possono "attivare" regole particolari di parsing.
Un caso esemplificativo sono gli operatori < e > che normalmente valgono come redirezionamento, ma all'interno di [[…]] valgono come operatori relazionali.

Le parentesi quadre singole sono POSIX-compliant, mentre le doppie sono un'estensione bash. Nel primo caso gli operatori relazionali "tradizionali" (minore-di, maggiore-di, etc.) non possono usare i termini comuni (<, >, etc.) perché hanno un altro significato (salvo eventualmente utilizzare il raggruppamento con doppie parentesi tonde per le espansioni aritmetiche) e quindi se ne usano di specifici che però hanno un equivalente più "tradizionale" nel secondo caso.

Gli operatori e la sintassi variano a seconda del tipo di informazioni utilizzate: una distinzione sottile c'è per **confronti tra stringhe e confronti tra interi**.

Confronti logici - interi e stringhe

interi		
	`[ … ]`	`[[ … ]]`
uguale-a	`-eq`	`==`
diverso-da	`-ne`	`!=`
minore-di minore-o-uguale-a	`-lt` `-le`	`<` `<=`
maggiore-di maggiore-o-uguale-a	`-gt` `-ge`	`>` `>=`

stringhe		
	`[ … ]`	`[[ … ]]`
uguale-a	= o ==	
diverso-da	!=	
minore-di (ordine alfabetico)	`\<`	`<`
maggiore-di (ordine alfabetico)	`\>`	`>`
nota: occorre lasciare uno spazio prima e dopo i "simboli" (es. non "=" ma " = ")		

Esistono alcuni **operatori unari** ad esempio per verificare se una stringa è vuota o meno oppure per controllare l'esistenza di un file o di una cartella.

Alcuni esempi:

```
[[ -f /tmp/prova ]]      : è un file?

[[ -e /tmp/prova ]]      : file esiste?

[[ -d /tmp/prova ]]      : è una cartella?
```

Sono essenziali gli spazi dopo [[e prima di]]. È possibile usare sia [che [[.

Il carattere "*!*" (punto esclamativo) può essere usato per **negare** il confronto seguente.

Alcuni esempi:

```
[[ ! -f /tmp/prova ]]

[[ ! -e /tmp/prova ]]

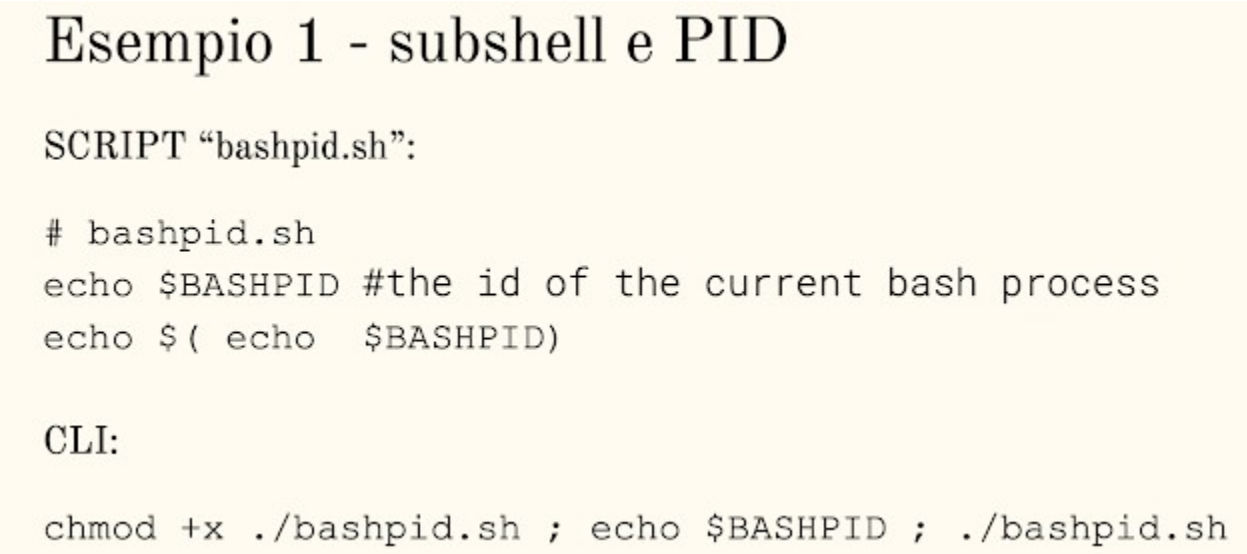
[[ ! -d /tmp/prova ]]
```

Script / Batch

È possibile raccogliere sequenze di comandi in un file di testo che può poi essere eseguito in due maniere:

- Richiamando il tool *bash* e passando il file come argomento
  Esempio: *bash ../file.sh*

- Impostando il bit x e specificando il percorso completo, o solo il nome se la cartella è in $PATH
  Esempio: $chmod\ +x\ ../file.sh\ \&\&\ ../file.sh$



Con il comando $chmod\ +x\ ./bashpid.sh$ stiamo rendendo il nostro file eseguibile.

Negli script le righe vuote e i commenti (#) sono ignorati.
La riga iniziale del file script può essere un metacommento, noto anche come "hash-bang" o "she-bang", che inizia con i caratteri "$\#!$" seguiti da un percorso all'applicazione o all'interprete che dovrebbe essere utilizzato per eseguire lo script. Questa è la convenzione utilizzata nei file script in Bash per specificare l'interprete da utilizzare per eseguire lo script.
Ad esempio, se si desidera che lo script venga eseguito utilizzando l'interprete Bash, la prima riga potrebbe essere:

$$\#!/bin/bash$$

L'interprete specificato nella riga hash-bang può anche avere opzioni aggiuntive associate.

Sono disponibili variabili speciali, in particolare:

- $\$@$: lista completa degli argomenti passati allo script
- $\$\#$: numero di argomenti passati allo script
- $\$0, \$1, \$2, \ldots$: n-th argomento

Altri costrutti che possono essere usati in uno script sono:

- Condizione If
- Cicli For
- Cicli While

For loop:

```
for i in ${!lista[@]}; do
    echo ${lista[$i]}
done
```

While loop:

```
while [[ $i < 10 ]]; do
    echo $i ; (( i++ ))
done
```

If condition:

```
if [ $1 -lt 10 ]; then
    echo less than 10
elif [ $1 -gt 20 ]; then
    echo greater than 20
else
    echo between 10 and 20
fi
```

Questi costrutti possono essere anche scritti su una sola riga e usati in un terminale.

Ecco un esempio in cui “scorriamo” gli argomenti passati allo script:

SCRIPT “args.sh”:

```
#!/usr/bin/env bash
nargs=$#
while [[ $1 != "" ]]; do
    echo "ARG=$1"
    shift
done
```

CLI:

```
chmod +x ./args.sh
./args.sh uno
./args.sh uno due tre
```

Possono venir dichiarate delle **funzioni**, semplicemente con la sintassi
function nomeFunzione(…)[…]

```
function_name () {
    local var1='C' #use a local scoped variable
    echo $1 $2 #normal arguments variables
    return 44 #only return codes
}

function function_name () { commands; }

function_name arg1 arg2  #call the function with 2 args
```

In conclusione l’utilizzo di BASH (tramite CLI o con SCRIPT) è basilare per poter interagire attraverso comandi con il file-system, con le risorse del sistema e per poter invocare tools e applicazioni.

Eseguire e compilare su Linux: Docker, GCC and Makefile

Docker

Docker è una tecnologia di virtualizzazione a livello del sistema operativo che consente la creazione, la gestione e la creazione di applicazioni attraverso containers.
I containers sono ambienti leggeri, dinamici ed isolati che vengono eseguiti sopra il kernel di Linux.

I Docker sono una piattaforma di containerizzazione che permette di creare, distribuire e gestire applicazioni in modo isolato. I **container Docker** contengono tutto il necessario per eseguire un'applicazione, inclusi il codice, le librerie, le dipendenze e le variabili di ambiente, offrendo un ambiente coerente e riproducibile su diversi ambienti di esecuzione.

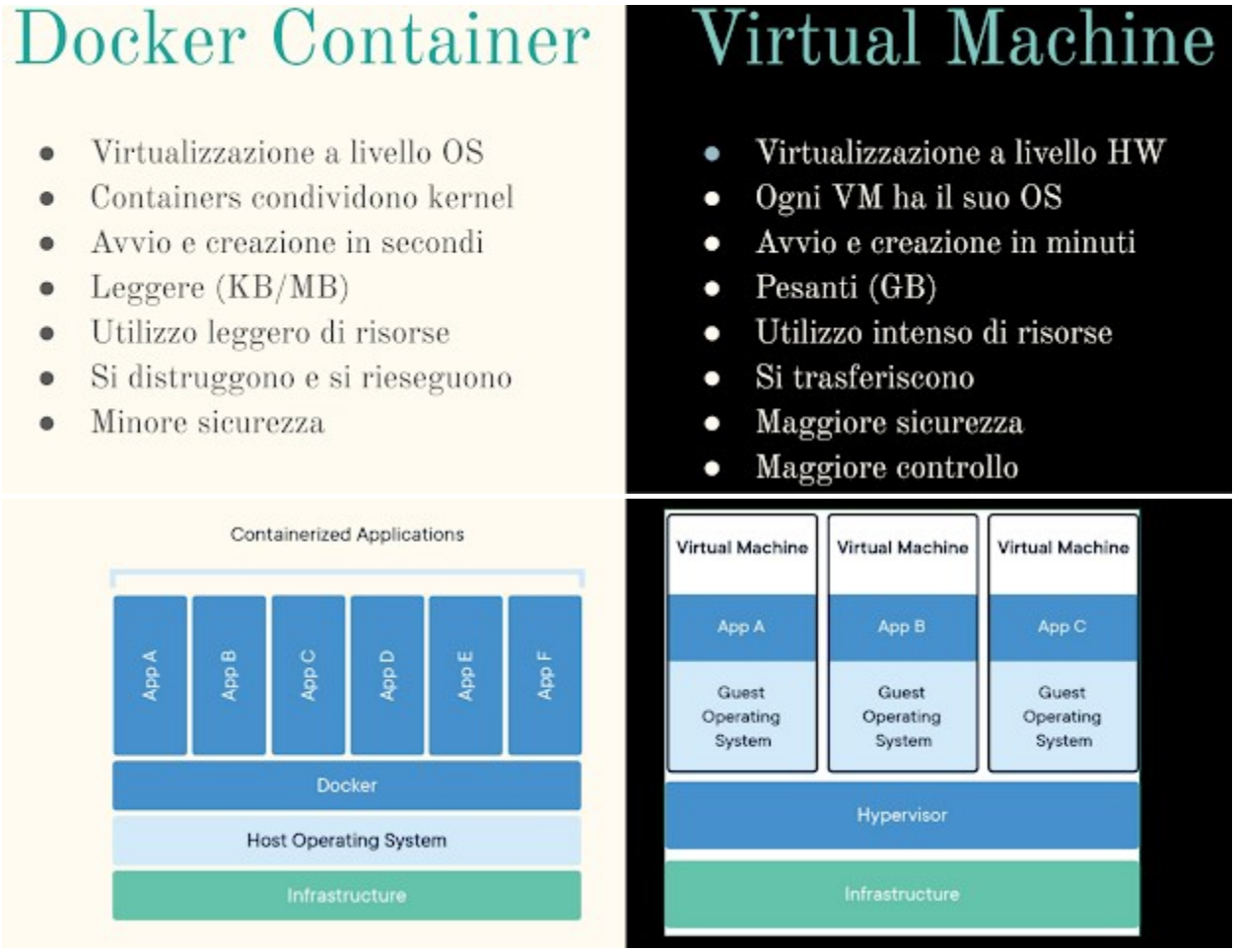
A differenza delle macchine virtuali (VM), i container Docker non emulano un intero sistema operativo, ma condividono il kernel del sistema ospite. Ciò significa che i container sono più leggeri e richiedono meno risorse rispetto alle VM. Mentre le VM richiedono l'installazione di un'intera copia di un sistema operativo, i container Docker utilizzano un'immagine di base che viene condivisa tra più container, rendendo più efficiente l'utilizzo delle risorse.

Oltre alla maggiore efficienza, i Docker offrono altri vantaggi rispetto alle VM. I container Docker possono essere avviati in pochi secondi, mentre le VM richiedono tempi di avvio più lunghi. Inoltre, i Docker forniscono un isolamento leggero tra i container, consentendo loro di funzionare indipendentemente l'uno dall'altro senza interferenze. Ciò rende i Docker ideali per la creazione di ambienti di sviluppo, test e produzione.

Un altro vantaggio dei Docker è la portabilità. Poiché i container Docker includono tutte le dipendenze necessarie per eseguire un'applicazione, possono essere facilmente spostati tra diversi ambienti di esecuzione, come computer locali, server remoti o ambienti cloud. Ciò semplifica la distribuzione delle applicazioni, garantendo che l'applicazione funzioni in modo coerente ovunque venga eseguita.

In sintesi, i Docker sono una tecnologia di containerizzazione che offre un modo efficiente e portatile per creare, distribuire e gestire applicazioni. A differenza delle macchine virtuali, i Docker sono più leggeri, avviano più rapidamente e consentono un utilizzo efficiente delle risorse.

Useremo docker per replicare l’ambiente del laboratorio sui nostri computer casalinghi.
Le principali differenze tra un Docker Container e una Virtual Machine sono le seguenti:



In sintesi Docker non si basa sul concetto di macchina (come fa la Virtual Machine) ma si basa sul concetto di Container.
Con Docker si possono installare e runnare software come apache, python, sql e soprattutto Ubuntu senza la pesantezza e la lentezza delle Virtual Machine.
Con docker non virtualizziamo l'hardware (come le Virtual machine) ma virtualizziamo il Sistema Operativo.
Con docker i Containers condividono il kernel, quindi non siamo vincolati dalla scelta di quale sistema operativo usare, invece nelle Virtual Machine ognuna deve avere il proprio Sistema Operativo.
Ovviamente il fatto che il kernel è condiviso può essere un problema di sicurezza.
I Containers non possono essere trasferiti, però possono venire condivise le istruzioni per ricrearlo.
Solitamente i Container si creano prima di utilizzarli e si distruggono quando si ha finito di usarli, questo perché si può distruggere e ricreare l'ambiente in maniera molto rapida.

Compatibilità sui vari OS

Linux: docker gestisce i containers usando il kernel linux nativo.



Windows: docker gestisce i containers usando il kernel linux tramite WSL2 (originariamente virtualizzato tramite Hyper-V). Gestito da un'applicazione.



Mac: docker gestisce i containers usando il kernel linux virtualizzato tramite xhyve hypervisor. Gestito da un'applicazione.



WSL: Windows Subsystem for Linux
Sui sistemi senza linux bisogna virtualizzare un sistema linux

Un'**immagine docker** è un insieme di "istruzioni" per la creazione di un container.
Essa consente di raggruppare varie applicazioni ed eseguirle, con una certa configurazione, in maniera rapida attraverso un container.
I containers sono invece gli ambienti virtualizzati gestiti da docker, che possono essere creati, avviati, fermati ed eliminati. I container devono essere basati su un'immagine! (esiste un'opzione di creazione da zero, FROM scratch, che però è raramente usata in pratica).
Quando condividiamo un container, condividiamo semplicemente le immagini usate per crearlo, così che si sia in grado di ricrearlo.

I comandi di Docker per gestire i containers sono i seguenti:

`docker run [options] <image>`: crea un nuovo container da un'immagine

`docker container ls [options]`: mostra i containers attivi (`[-a]` tutti)

`docker start/stop <container>`: avvia/ferma l'esecuzione del container

`docker exec [options] <container> <command>`: esegue il comando all'interno del container

`docker stats`: mostra le statistiche di utilizzo dei containers

E la panacea di tutti i dubbi... `docker <command> --help`

Generalmente si usa solo il comando *run*.

Nei Docker solitamente si utilizza il comando `run` anziché `stop` e `start` per avviare e fermare i container. Ciò è dovuto al fatto che i container Docker sono progettati per essere processi

isolati e autogestiti, piuttosto che istanze di un sistema operativo completo come avviene nelle macchine virtuali.

Quando si esegue il comando `run`, si crea un nuovo container partendo da un'immagine di base specificata. Questo container viene avviato in modo autonomo come processo separato. A differenza delle macchine virtuali, un container non viene avviato come un'entità persistente che richiede un processo di accensione o spegnimento. Invece, i container Docker sono progettati per essere "usa e getta" e possono essere facilmente creati, avviati e fermati secondo necessità.

Quando si desidera fermare un container Docker, è possibile utilizzare il comando `stop`. Questo comando ferma il processo del container in modo pulito, consentendo al container di salvare lo stato corrente e chiudere tutte le risorse utilizzate. Successivamente, il container può essere riavviato in qualsiasi momento con il comando `start`, riprendendo dallo stato in cui si trovava al momento della chiusura.

Tuttavia, nella pratica, è più comune utilizzare il comando `run` per creare e avviare nuovi container e il comando `stop` per fermare e rimuovere i container inutilizzati. Questo approccio è più flessibile e consente una gestione più dinamica dei container, adattandosi meglio a scenari di sviluppo, test e produzione in cui possono essere necessari molteplici container simultaneamente o in rapida successione.

Il comando *run* ha vari parametri opzionali:

- `--name <nome>`: assegna un nome specifico al container
- `-d`: detach mode → scollega il container (ed il suo input/output) dalla console*
- `-ti`: esegue container in modalità interattiva*
- `--rm`: elimina container all'uscita
- `--hostname <nome>`: imposta l'hostname nel container
- `--workdir <path>`: imposta la cartella di lavoro nel container
- `--network host`: collega il container alla rete locale **
- `--privileged`: esegue il container con i privilegi dell'host

La detach mode (modalità di distacco) di un container Docker è una modalità di esecuzione in cui il container viene avviato in background e viene liberata la console o il terminale da cui è stato lanciato. In altre parole, il container viene eseguito come un processo separato in background, consentendo all'utente di continuare a lavorare sulla console senza essere bloccato nell'attesa che il container termini l'esecuzione.

Quando si avvia un container Docker, di default viene utilizzata la modalità di attacco (attach mode), in cui la console o il terminale resta associato al processo del container, mostrando l'output del processo e consentendo l'interazione con esso. Questo è utile per visualizzare i log e l'output dell'applicazione eseguita nel container o per interagire con essa in tempo reale.

Tuttavia, in situazioni in cui non è necessaria un'interazione immediata con il container o quando si desidera eseguire il container come processo in background, è possibile utilizzare la detach mode. Ciò consente di avviare il container in background e liberare la console o il terminale per eseguire altre attività.

Una volta avviato in detach mode, è possibile controllare lo stato del container, visualizzare i log o interagire con esso utilizzando comandi specifici come `docker logs`, `docker exec` o `docker attach`.

La detach mode è particolarmente utile quando si eseguono container in ambienti di produzione o quando si desidera mantenere i container in esecuzione in background per un lungo periodo di tempo senza dover mantenere una console o un terminale aperti costantemente.

Se si avvia il container in detach mode (non si aggancia la console all'output di quel container), ci si può ricollegare con il comando

$$docker \;\; attach \;\; <container>$$

Per scollegarsi nuovamente bisogna utilizzare la combinazione di tasti $Ctrl + P$ e $Ctrl + Q$.

La modalità host non funziona su W10 e MacOS a causa della VM sottostante, la modalità host consente di collegare il container alla rete locale e questo è molto utile per far comunicare i container (ad esempio tra frontend e backend)

Con il parametro $-ti$ il container non termina subito dopo l'esecuzione ma c'è la possibilità di usarlo interattivamente.

Il parametro `-ti` nel comando `docker run` viene utilizzato per allocare una pseudo-TTY (teletype) e mantenere aperto uno standard input interattivo, consentendo l'interazione con il container. Questo parametro combina due opzioni: `-t` che assegna una TTY al container e `-i` che mantiene aperto lo standard input.
Quando si utilizza `-ti`, il container viene avviato in modalità di attacco (attach mode) e il prompt del container viene collegato al terminale corrente. Ciò consente di visualizzare l'output del processo del container e di interagire con esso in tempo reale. È utile quando si desidera monitorare l'applicazione o interagire con essa tramite comandi o input da tastiera.

Entrambi i parametri `-d` e `-ti` possono essere utilizzati insieme o separatamente, a seconda delle necessità specifiche dell'utilizzo del container.

1. Utilizzo del parametro `-d` (detach mode) senza `-ti`:

arduino Copy code

```
docker run -d nome_immagine
```

Questo avvia il container in background, liberando la console o il terminale da cui viene eseguito. Il container continuerà a eseguirsi in background e potrà essere controllato successivamente tramite comandi come `docker logs` o `docker exec`.

2. Utilizzo del parametro `-ti` senza `-d`:

arduino Copy code

```
docker run -ti nome_immagine
```

Questo avvia il container in modalità di attacco (attach mode) mantenendo aperto lo standard input e assegnando una TTY (teletype). Viene collegato il prompt del container al terminale corrente, consentendo l'interazione con l'applicazione nel container in tempo reale. Ad esempio, è possibile avviare un'interfaccia a riga di comando all'interno del container.

3. Utilizzo di entrambi i parametri `-d` e `-ti` insieme:

arduino Copy code

```
docker run -d -ti nome_immagine
```

Questo avvia il container in modalità detach e allo stesso tempo mantiene aperto uno standard input interattivo e assegna una TTY. Il container viene eseguito in background, ma è ancora possibile interagire con esso tramite il collegamento al suo prompt. Questo può essere utile per avviare un'applicazione in background e, se necessario, interagire con essa successivamente.

4. Nessun parametro `-d` né `-ti`:

arduino Copy code

```
docker run nome_immagine
```

Questo avvia il container in modo predefinito, senza la modalità di distacco o il collegamento a una TTY. Il comportamento specifico dipenderà dall'immagine e dalla configurazione predefinita dell'immagine stessa. In genere, il container verrà eseguito in modalità attach (interattiva) e l'output del processo del container verrà visualizzato nella console o nel terminale corrente.

Non è importante salvarsi i container, questo perché generalmente vengono creati e distrutti ogni volta. I dati rilevanti che vorremmo salvare saranno salvati da un’altra parte.

La community di docker offre migliaia di immagini pronte all'uso:

`docker images`: mostra le immagini salvate

`docker rmi <imageID>`: elimina un'immagine (se non in uso!)

`docker search <keyword>`: cerca un'immagine nella repository di docker

`docker commit <container> <repository/imageName>`: crea una nuova immagine dai cambiamenti nel container

Le immagini che non vengono trovate localmente vengono cercate e scaricate online.
Possiamo specificare la versione dell'immagine che vogliamo utilizzare, questo viene fatto mettendo i due punti e dopo il numero della versione dopo il nome dell'immagine.

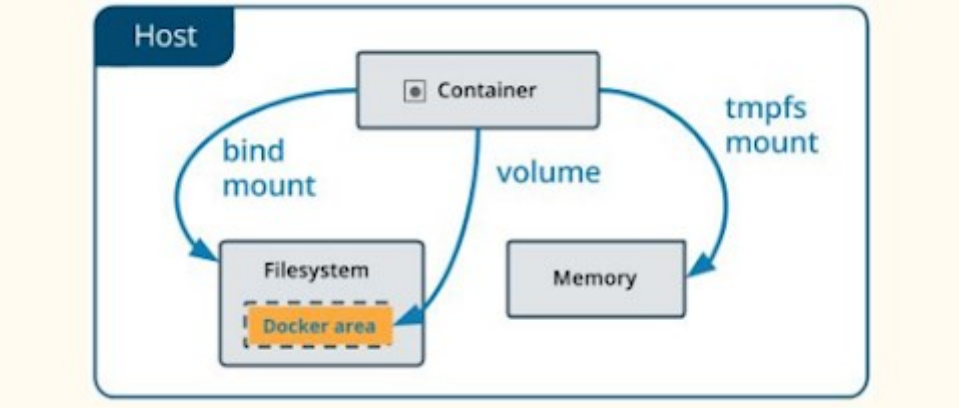
Il comando *commit* serve per creare una nuova immagine sulla base del container che sto utilizzando.

Altrimenti si possono creare nuove immagini con dei dockerfile.
I **dockerfile** (che sono file senza estensione) sono dei documenti testuali che raccolgono una serie di comandi necessari alla creazione di una nuova immagine. Ogni nuova immagine sarà generata a partire da un'immagine di base, come Ubuntu o l'immagine minimale "scratch". La creazione a partire da un docker file viene gestita attraverso del caching che ne permette la ricompilazione rapida in caso di piccoli cambiamenti.

```
FROM ubuntu:22.04
RUN apt-get update && apt-get install build-essential nano -y
RUN mkdir /home/labOS
CMD cd /home/labOS && bash
```

```
docker build -t labos/ubuntu - < dockerfile
```

Per poter salvare i file e i dati che trattiamo nei container dobbiamo trovare un modo per farli sopravvivere alla distruzione del container, quindi si usano i **volumi**.
Docker salva i file persistenti su bind mount o su dei volumi. Sebbene i **bind mount** siano strettamente collegati con il filesystem dell'host OS, consentendo dunque una facile comunicazione con il containers, i **volumi** sono ormai lo standard in quanto indipendenti, facili da gestire e più in linea con la filosofia di docker.



Con i bind mount viene dato al container l'accesso a una cartella del sistema principale (opzione pericolosa per quanto riguarda la sicurezza).
Con i volumi invece creiamo uno spazio fisico che viene usato esclusivamente attraverso docker (logicamente è come se fosse una chiavetta usb attaccata al docker).

Ecco i comandi di docker per gestire i volumi:

`docker volume create <volumeName>`: crea un nuovo volume

`docker volume ls`: mostra i volumi esistenti

`docker volume inspect <volumeName>`: esamina volume

`docker volume rm <volumeName>`: rimuovi volume

`docker run -v <volume>:</path/in/container> <image>` : crea un nuovo container con il **volume** specificato montato nel percorso specificato

`docker run -v <pathHost>:<path/in/container> <image>` : crea un nuovo container con un **bind mount** specificato montato nel percorso specificato

Per ricreare il nostro ambiente del laboratorio dobbiamo runnare il docker così:

```
docker run -ti --rm --name="labOS" --privileged \
-v /:/host -v "$(pwd):/home/labOS" \
--hostname "labOS" --workdir /home/labOS \
ubuntu:22.04 /bin/bash
```

E successivamente eseguire

```
apt-get update && apt-get install -y nano build-essential
```

Nota: se non aggiungiamo il flag *--rm*, ogni volta che usciamo dal container esso verrà fermato e potrà essere riavviato con *docker start labOS*

Altrimenti possiamo scrivere un dockerfile nel seguente modo:

FROM ubuntu:22.04
RUN apt-get update && apt-get install build-essential nano -y
RUN mkdir /home/labOS
CMD cd /home/labOS && bash

ed eseguire

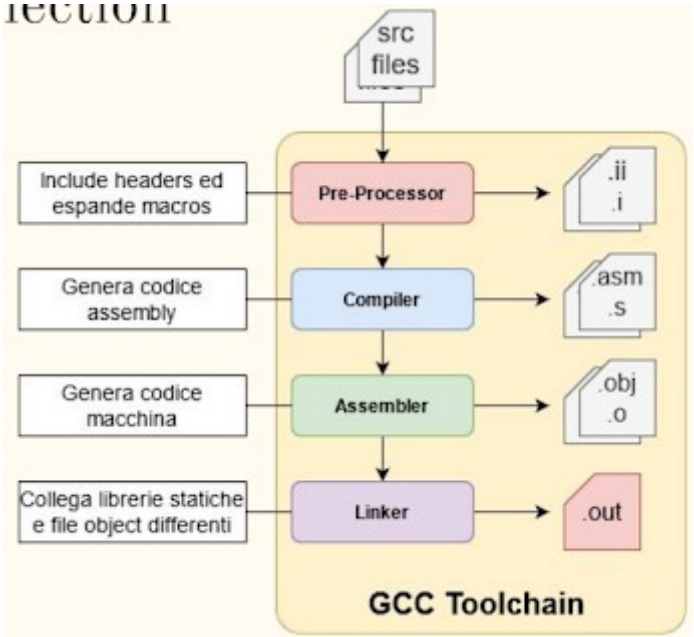
docker build -t labOS/ubuntu - < dockerfile

E poi possibile eseguire un container basato sulla nuova immagine ‘labOS/ubuntu’ con il comando

docker run -ti --rm --name="labOS" --privileged \
-v /:/host -v "$(pwd):/home/labOS" \
--hostname "labOS" labos/ubuntu

GCC

GCC (Gnu Compiler Collection) è un insieme di strumenti open-source che costituisce lo standard per la creazione di eseguibili su Linux.
GCC supporta diversi linguaggi, tra cui C, e consente la modifica dei vari passaggi intermedi per una completa personalizzazione dell’eseguibile. Ecco un’immagine che rappresenta tutti gli strumenti di GCC:



Gli strumenti GCC possono essere chiamati singolarmente:

```
gcc -E <sorgente.c> -o <preProcessed.ii|.i>

gcc -S <preProcessed.i|.ii> -o <assembly.asm|.s>

gcc -c <assembly.asm|.s> -o <objectFile.obj|.o>

gcc <objectFile.obj|.o> -o <executable.out>
```

L’input di ogni comando può essere qualsiasi file al di sopra di quello corrente (quindi possiamo dare il file sorgente in input ad ogni comando). L’ultimo comando è in grado di creare direttamente l’eseguibile, però l’assembly e il codice macchina generato dipendono dall’architettura di destinazione.

Make

Il **Make tool** è uno strumento della collezione GNU che può essere usato per gestire la compilazione automatica e selettiva di grandi e piccoli progetti. Make consente di specificare delle dipendenze tra i vari file, per esempio consentendo solo la compilazione di librerie i cui sorgenti sono stati modificati.
La compilazione si dice selettiva perché permette di compilare i file solo se necessario (ovvero quando il file non è ancora stato compilato oppure se ci sono state delle modifiche).
Make può anche essere usato per gestire il deployment di un’applicazione, assumendo alcune delle capacità di uno script bash.

Il Make tool può eseguire dei **makefiles**, i quali contengono tutte le direttive utili alla compilazione di un’applicazione (o svolgimento di un altro task):

$$make\ -f\ makefile$$

In alternativa, il comando make senza argomenti processerà il file ‘makefile’ presente nella cartella di lavoro (nell’ordine cerca: $GNUmakefile$, $makefile$ e $Makefile$).
Makefiles secondari possono essere inclusi nel makefile principale con delle direttive specifiche, consentendo una gestione più articolata di grandi progetti.

I makefile sono composti da:

- Target
- Prerequisiti
- Ricette

Una **ricetta** (o regola) è una lista di comandi bash che vengono eseguiti indipendentemente dal resto del makefile. Le regole devono sempre essere racchiuse in target ed è importante inserire il tab prima delle regole.
I **target** sono generalmente dei files generati da uno specifico insieme di regole.
Ogni target può specificare dei **prerequisiti**, ovvero degli altri file che devono esistere affinchè le regole di un target vengano eseguite. Un prerequisito può essere esso stesso un target!

```
target: prerequisite
→   recipe/rule
→   recipe/rule
    ...
```

```
target1: target2 target3
    rule (3)
    rule (4)
    ...

target2: target3
    rule (1)

target3:
    rule (2)
```

L'esecuzione di un file make inizia specificando uno o più target $make\ -f\ makefile\ target1\ \ldots$ e prosegue a seconda dei vari prerequisiti.

Un makefile è un file di testo "plain" in cui righe vuote e parti di testo dal carattere "#" fino alla fine della riga non in una ricetta (considerato un commento: sempre che non sia usato l'escaping con "\#" o che compaia dentro una stringa con ' o ") sono ignorati.
Le ricette DEVONO iniziare con un carattere di TAB (non spazi).

Una ricetta che inizia con il carattere @ (dopo il TAB) non viene visualizzata in output, quindi non verrà stampato in *stdout* l'output di quel comando.
Invece con - diciamo che non venga stampato proprio il comando (altrimenti viene stampato il comando prima di essere eseguito).
Una riga che ha un solo TAB è una ricetta vuota.
Esistono costrutti più complessi per necessità particolari (ad esempio costrutti condizionali).

Se alla chiamata del make non viene passato alcun target, viene eseguito quello di default: il primo disponibile. Esistono poi dei target che hanno un significato e comportamento speciale:

- *.INTERMEDIATE*
  Questo target viene utilizzato per specificare i file intermedi generati durante la compilazione che non sono desiderati come output finale. Questi file intermedi vengono automaticamente eliminati dal sistema una volta che il loro utilizzo è terminato. Di solito, questi sono file temporanei generati durante il processo di compilazione, come file oggetto (.o) o file di dipendenza (.d).
- *.SECONDARY*
  Questo target viene utilizzato per specificare i file che sono considerati intermediari, ma che devono essere mantenuti anche dopo la compilazione. A differenza di .INTERMEDIATE, i file elencati come .SECONDARY non vengono automaticamente

eliminati dopo l'uso. Questo può essere utile quando si desidera conservare file intermedi per scopi di debug o per altre ragioni.

- $.PHONY$
  Questo target viene utilizzato per dichiarare le regole che non corrispondono a file effettivi. Ad esempio, se si ha una regola denominata "clean" che elimina i file generati durante la compilazione, è possibile dichiararla come .PHONY per indicare che non corrisponde a un file effettivo.
  Ha come prerequisiti i target che non corrispondono a dei files, o comunque da eseguire "sempre" senza verificare l'eventuale file omonimo

In un target, il carattere % sostituisce qualunque stringa. In un prerequisito corrisponde alla stringa sostituita nel target.

```
all: ...
  rule

.SECONDARY: target1 ..

.PHONY: target2 …

%.s: %.c
  #prova.s: prova.c
  #src/h.s: src/h.c
```

Le variabili utente si definiscono con la sintassi $nome := valore$ oppure $nome = valore$ e per accedervi si usa $\$(nome)$.
C'è una differenza tra le due definizioni però

- $:=$ in questo caso il valore della variabile viene definita una volta soltanto, quindi la variabile assume il valore che ha la variabile al momento della dichiarazione
- $=$ in questo caso il valore della variabile viene "ricalcolato" ogni volta che viene chiamata

Inoltre le variabili possono essere sovrascritte da riga di comando con $make \;\; nome = value$, le variabili definite da linea di comando sovrascriveranno le variabili definite dentro al makefile.
La variabile SHELL può essere usata per specificare la shell di riferimento per l'esecuzione del makefile. Per esempio, $SHELL = /bin/bash$ abilita bash anziché SH.

Le variabili automatiche possono essere usate all'interno delle regole per riferirsi ad elementi specifici relativi al target corrente:

- $\$@$ il nome del target
- $\$\wedge$ la lista di tutti i prerequisiti
- $\$<$ per ottenere il prossimo prerequisito

```
ONCE:=hello $(LATER)
EVERY=hello $(LATER)
LATER=world

target1:
  echo $(ONCE) # 'hello'
  echo $(EVERY) # 'hello world'
```

```
target: pre1 pre2 pre3
  echo $@ is 'target'
  echo $^ is 'pre1 pre2 pre3'
  echo $< is 'pre1'
```

Funzioni speciali

`$(eval ...)`: consente di creare nuove regole make dinamiche.

`$(shell ...)`: cattura l'output di un commando shell.

`$(wildcard *)`: restituisce un elenco di file che corrispondono alla stringa specificata.

```
LATER=hello
PWD=$(shell pwd)
OBJ_FILES:=$(wildcard *.o)

target1:
  echo $(LATER) #hello
  $(eval LATER+= world)
  echo $(LATER) #hello world
```

Ecco un esempio di makefile:

```
all: main.out
     @echo "Application compiled"

%.s: %.c
     gcc -S $< -o $@

%.out: %.s
     mkdir -p build
     gcc $< -o build/$@

clean:
     rm -rf build *.out *.s

.PHONY: clean

.SECONDARY: make.s
```

In conclusione Docker, GCC e make possono essere utilizzati per la gestione delle varie applicazioni in C. Ognuno di questi strumenti non è indispensabile ma permette di creare un flusso di lavoro coerente e strutturato.

Le basi di C

Scegliamo di utilizzare C, rispetto ad altri linguaggi di programmazione ad alto livello, per svariati motivi, tra cui:

- Struttura minimale, non c'è troppa sintassi
- Poche parole chiave, questo ha i suoi pro e i suoi contro, ad esempio che la flessibilità è limitata, però è più semplice da memorizzare
- È Unix compliant, questo è fondamentale per quanto riguarda le performance
- È organizzato a passi (file sorgente, file assembly, file oggetto e file eseguibile)
- C'è una vasta disponibilità di librerie conosciute che costituiscono lo standard (sono molto famose)
- È efficiente perché è di basso livello
- Abbiamo il pieno controllo del programma e delle sue risorse (in altri linguaggi come Python e Java non è così)
- È ottimo per interagire con il sistema operativo

Direttive e istruzioni fondamentali

- `#include … / #define … /`
- `char / int / … / enum (v. esempio seguente)`
- `for ( initialization ; test; increment ) { … ; }`
- `break / continue`
- `switch (expression){ case val: … [break;] [default: …] }`
- `while (expression) { … } / do { … } while (expression)`
- `if (expression) { … } [else { … }]`
- `struct / union`

C è un linguaggio debolmente tipizzato (si possono effettuare delle conversioni/cast tra tipi) che utilizza 8 tipi fondamentali.
Un linguaggio debolmente tipizzato, come il linguaggio C, è un linguaggio di programmazione in cui le variabili non sono strettamente vincolate a un tipo di dato specifico. Ciò significa che le variabili possono essere utilizzate in modi diversi senza la necessità di convertirle esplicitamente tra i diversi tipi di dati.
A differenza dei linguaggi fortemente tipizzati, come Java o Python, che richiedono conversioni esplicite tra i tipi di dati e forniscono un controllo più rigoroso sui tipi, i linguaggi debolmente tipizzati come il C offrono maggiore flessibilità ma richiedono anche una maggiore responsabilità da parte del programmatore nell'assicurarsi che i tipi di dati vengano gestiti correttamente.

Ad esempio si può fare il seguente casting:

$$float\ \ a\ =\ 3.5;$$
$$\int\ \ b\ =\ (\int)a;$$

Il casting esplicito è consigliato ma non necessario, infatti i compilatori di oggi capiscono le intenzioni del programmatore e eseguono un casting implicito.
La grandezza delle variabili è dipendente dall' architettura di riferimento e i valori massimi per ogni tipo cambiano a seconda se la variabile è signed o unsigned.
Le grandezze standard sono le seguenti:

- void (0 byte)
- char (1 byte)
- short (2 bytes)
- int (4 bytes)
- float (4 bytes)
- long (8 bytes)
- double (8 bytes)
- long double (8 bytes)

Come vediamo non esiste il tipo boolean, quindi dobbiamo per forza allocare 1 byte, infatti solitamente il boolean viene emulato con un char.

L'operatore *sizeof* può essere applicato ad un tipo (in questo caso dobbiamo racchiuderlo tra parentesi), oppure ad un'espressione, e ci restituisce la grandezza in byte occupata in memoria.

```c
#include <stdio.h>
void main() {
  int x = 10;
  printf("variable x      : %lu\n", sizeof x);
  printf("expression 1/2 : %lu\n", sizeof 1/2);
  printf("int type        : %lu\n", sizeof(int));
  printf("char type       : %lu\n", sizeof(char));
  printf("float type      : %lu\n", sizeof(float));
  printf("double type     : %lu\n", sizeof(double));
}
```

C si evolve attorno all'uso di **puntatori**, ovvero degli alias per zone di memorie condivise tra diverse variabili/funzioni. L'uso di puntatori è abilitato da due operatori: * ed &:

- * ha significati diversi a seconda se usato in una dichiarazione o in un'assegnazione:

- $\int$ $*punt;$ crea una variabile puntatore ad intero
- $\int$ $valore = *(punt);$ ottiene il valore puntato dalla variabile puntatore $punt$

- & ottiene l'indirizzo di memoria in cui è collocata una certa variabile
  $long\ whereIsValore = \&valore;$

Ecco alcuni esempi:

```
Esempi:
float   pie   =   3.4;
float  *pPie  =  &pie;
pie        *=       2;
float pie4 = *pPie * 2;
```

In realtà il tipo sui puntatori non serve a niente per il puntatore stesso, visto che tutti i puntatori sono della stessa dimensione (contengono tutti degli indirizzi di memoria). Però il tipo sui puntatori servono al momento della dereferenziazione per capire il tipo della variabile che si sta leggendo.

```
int     i     =     42;
int   *   punt   =   &i;
int b = *(punt);
```

Tipo	Nome	Valore	Indirizzo
int	i	42	0xaaaabbbb
int *	punt	0xaaaabbbb	0xccccdddd
int	b	42	0x11112222

```
i = 20;
```



Nome	Valore
i	**20**
punt	0xaaaabbbb
b	?

C consente anche di creare dei puntatori a delle funzioni (ovvero variabili che possono contenere l'indirizzo di funzioni differenti), che invece sono leggermente diversi nella sintassi rispetto ai puntatori normali, e ci permettono di avere un puntatore che può assumere funzionalità diverse.
Per dichiarare un puntatore a una funzione dobbiamo fare così:

$$ret_type\ (*\ pntName)(argType,\ argType,\ \ldots)$$

```c
#include <stdio.h>
float xdiv(float a, float b) {
  return a/b;
}
float xmul(float a, float b) {
  return a*b;
}
int main() {
  float (*punt)(float,float);
  punt = xdiv;
  float res = punt(10,10);
  punt = &xmul; //& opzionale
  res = (*punt)(10,10);
  printf("%f\n", res);
  return;
}
```

Ovviamente per poter assegnare allo stesso puntatore due funzioni, le funzioni devono avere lo stesso numero di argomenti e gli argomenti devono essere dello stesso tipo.
La sintassi per assegnare una funzione ad un puntatore è usando &, ma GCC è abbastanza intelligente da prendere in automatico l'indirizzo della funzione quando utilizziamo un puntatore, quindi possiamo omettere & prima del nome della funzione.

A parte casi particolari (es. sviluppo moduli per kernel) l'applicazione deve sempre avere una funzione "**main**" che è utilizzata come punto di ingresso. Se la funzione main non è presente, il sistema operativo non può sapere da che punto far iniziare l'esecuzione di quel file. Di conseguenza ogni file (tranne certi casi particolari) deve avere una funzione main.

Il valore di ritorno della funzione main è di tipo $\int ¿$, ed è un intero che rappresenta il codice di uscita dell'applicazione (messo nella variabile $\$?$ in bash) ed è 0 di default se omesso. Possiamo anche definire il main come void, ma non è standard (e quindi potrebbe dare problemi nel momento in cui il nostro programma viene utilizzato da altri). Se vogliamo essere compliant con lo standard dobbiamo ritornare i valori giusti per ogni casistica di esecuzione.

I parametri del main sono $\int \ arcg$, ovvero il numero di argomenti della chiamata all'eseguibile (incluso il nome dell'eseguibile) e $char \ * \ argv[\,]$, ovvero la lista degli argomenti sottoforma di vettore di stringhe (un vettore di stringhe equivale a un vettore di vettori di caratteri, quindi è spesso indicata come $char \ ** \ argv$).

Per compilare un file C usando GCC scriviamo:

$$gcc \ \ main.c \ \ -o \ \ main.out$$

Il flag -o (output) specifica il nome del file in output. Per poi eseguire la nostra applicazione scriviamo:

$$./main.out \ \ arg1 \ \ arg2$$

Per stampare qualcosa in output sul terminale esistono due funzioni:

- $\int printf(const\ \ char\ \ *format,\ ...)$
  Invia sempre i dati sul canale *stdout*
- $\int fprintf(FILE\ \ *stream,\ const\ \ char\ \ *format,\ ...)$
  Si può specificare il canale di uscita

Entrambi stampano sul canale qualcosa secondo il formato specificato. Il formato è una stringa contenente contenuti stampabili (testo, a capo, …) ed eventuali segnaposto identificabili dal formato generale:

$$\%[flags][width][.precision][length]specifier$$

Alcuni esempi di segnaposto sono:

- $\%d$ o $\%i$ (numero intero con segno)
- $\%f$ (numero in virgola mobile)
- $\%c$ (carattere)
- $\%s$ (stringa)
- $\%p$ (puntatore)

Ad ogni segnaposto deve corrispondere un ulteriore argomento del tipo corretto.

```c
#include <stdio.h>

int main(int argc, char **argv) {
  printf("%d\n", argc);
  printf("%s\n", argv[0]);
  return 0;
}
```

Il compilatore, nella fase di preprocessing, elabora tutte le **direttive** presenti nel file sorgente. Ogni direttiva viene introdotta con # e può essere di vari tipi:

- $\#include\ \ <lib>$
  Copia il contenuto del file *lib* (cercando nelle cartelle delle librerie) nel file corrente
- $\#include\ \ "lib"$
  Come sopra però prima cerca il file nella cartella corrente
- $\#define\ \ VAR\ \ VAL$
  Crea una costante VAR con il contenuto VAL, e sostituisce ogni occorrenza di VAR con VAL
- $\#define\ \ MUL(A,B)\ \ \ A*B$
  Per dichiarare una funzione (queste funzioni hanno una sintassi limitata!)
- $\#ifdef,\ \#ifndef,\ \#if,\ \#else,\ \#endif$
  Rende l'inclusione di parte di codice dipendente da una condizione.

Le macro (quelle con define) possono essere passate a GCC con $-D\ \ NAME=VALUE$.

Un header file (o file di intestazione) è un file che aiuta il programmatore nell'utilizzo di librerie durante la programmazione. Un header file del linguaggio ANSI C è un semplice file di testo che contiene i prototipi delle funzioni definite nel relativo file .c. I prototipi permettono al compilatore di produrre un codice oggetto che può essere facilmente unito (detto volgarmente "linkato") con quello della relativa libreria in futuro, anche senza avere la libreria sottomano al momento.
I file header in c hanno l'estensione $.h$. Quindi quando vogliamo usare una funzione di libreria (o comunque esterna) nel nostro file citiamo solo l'header file.

Le **librerie** possono essere usate attraverso la direttiva $\#include$, tra le librerie più importanti ci sono:

- $stdio.h$
  Contiene funzioni che permettono di interagire con i file, ad esempio
  $FILE,\ EOF,\ stderr,\ stdin,\ stdout,\ fclose(),\ etc...$
- $stdlib.h$
  Contiene funzioni per operazioni standard, ad esempio
  $atof(),\ atoi(),\ malloc(),\ free(),\ exit(),\ system(),\ rand(),\ etc...$
- $string.h$
  Contiene funzioni per operazioni sulle stringhe, ad esempio
  $memset(),\ memcpy(),\ strncat(),\ strcmp(),\ strlen(),\ etc...$
- $math.h$
  Contiene funzioni per operazioni matematiche, ad esempio
  $\sin(),\ \cos(),\ \sqrt{}(),\ floor(),\ etc...$
- $unistd.h$
  Contiene funzioni per operazioni a livello di unix, ad esempio
  $STDOUT_FILENO,\ read(),\ write(),\ fork(),\ pipe(),\ etc...$
- $fcntl.h$
  Contiene funzioni per interagire con i file in una determinata maniera, ad esempio
  $creat(),\ open(),\ etc...$

Altri due costrutti importanti di C sono le **structs** e le **unions**:

- Una struttura (struct) è una collezione di variabili di diversi tipi, raggruppate insieme in una singola entità. Ogni variabile all'interno della struttura occupa una posizione di memoria separata, dunque le variabili possono coesistere. Puoi accedere alle variabili all'interno di una struttura utilizzando l'operatore punto . per specificare il nome della variabile e il nome della struttura. Le strutture sono generalmente utilizzate per rappresentare oggetti complessi con proprietà diverse.
  Per le strutture, la dimensione totale allocata sarà la somma delle dimensioni dei singoli membri, tenendo conto dell'allineamento richiesto dai tipi di dati. Questo significa che la dimensione totale di una struttura può essere maggiore della somma delle dimensioni dei suoi membri, a causa dell'allineamento.
- Un'unione (union) è una struttura speciale in cui tutte le variabili condividono la stessa area di memoria. Ciò significa che le variabili all'interno di un'unione occupano la stessa posizione di memoria e condividono lo stesso spazio. Ciò comporta che solo una delle

variabili all'interno dell'unione può essere utilizzata alla volta. Puoi accedere alle variabili all'interno di un'unione utilizzando l'operatore punto . come per le strutture. Le unioni sono spesso utilizzate per rappresentare diversi tipi di dati in una stessa area di memoria.
Per le unioni, la dimensione allocata sarà uguale alla dimensione del membro più grande all'interno dell'unione. L'unione condivide la stessa area di memoria per tutti i membri, quindi viene allocata solo la quantità di memoria necessaria per contenere il membro più grande.

```
struct Books{
    char author[50];
    char title[50];
    int bookID;
} book1, book2;

struct Books book3 = {"Rowling","Harry Potter",2};
strcpy(book1.title,"Moby Dick");
book2.bookID = 3;
```

```
union Result{
    int intero;
    float decimale;
} result1, result2;

union Result result3;
result3.intero = 22;
result3.decimale = 11.5;
```

Possiamo definire dei nuovi tipi di variabile o di funzioni con l'operatore *typedef*. Questo operatore è molto utile per rinominare le struct e le union, che altrimenti (come si vede dall'esempio) hanno un nome lungo e composto.

In C si possono usare anche le enum.
Le enum, abbreviazione di "enumeration" (enumerazione), sono un tipo di dato in C utilizzato per definire un insieme di costanti simboliche, generalmente utilizzate per rappresentare insiemi di valori correlati.
Le enum in C vengono utilizzate per rendere il codice più leggibile e mantenibile, in quanto consentono di assegnare significato simbolico a valori specifici. Puoi utilizzare le enum per definire insiemi di costanti che sono strettamente correlati tra loro, come le costanti che rappresentano i giorni della settimana o le opzioni di un menu.
Si può associare un valore numerico a ciascuna costante all'interno di un'enum in C. L'associazione di un numero a ciascuna costante all'interno dell'enum può essere utile quando si desidera utilizzare valori specifici per rappresentare i membri dell'insieme. Tieni presente che non è obbligatorio assegnare valori numerici in sequenza. Puoi assegnare qualsiasi valore numerico desiderato a ciascuna costante.

```c
#include <stdio.h>

enum State {Undef = 9, Working = 1, Failed = 0};
void main() {
    enum State state=Undef;
    printf("%d\n", state); // output è "9"
}
```

Esiste una funzione, *void exit* $(\int$ *status*$)$ che può essere chiamata all'interno di qualsiasi funzione per terminare il processo e restituire un valore di ritorno specifico *status*.
L'effetto è lo stesso di usare *return status* all'interno della funzione main.
La funzione non ha un valore di ritorno (è *void*) proprio perché non sono eseguite ulteriori istruzioni dopo di essa.
Il processo chiamante è informato della terminazione tramite un "segnale" apposito (vedremo più avanti il concetto di segnale e come gestirli).

Vettori e stringhe

I **vettori** sono sequenze di elementi omogenei (tipicamente liste di dati dello stesso tipo, ad esempio liste di interi o di caratteri).
I vettori in C si realizzano con un puntatore al primo elemento della lista.
Per determinare la grandezza di un vettore, devi considerare la dimensione di ogni elemento moltiplicata per il numero totale di elementi nel vettore.

`char str[7] = {'c', 'i', 'a', 'o', 56,57,0}` : 7*1 = 7 bytes

`str` è dunque un puntatore a `char` (al primo elemento) e si ha che:

`str[n]` corrisponde a `*(str+n)`

e in particolare `str[0]` corrisponde a `*(str+0)=*(str)=*str`

Vediamo dunque che i puntatori possono essere usati anche per scorrere un array.
In genere GCC è intelligente e quando andiamo ad incrementare il valore di un puntatore di un array viene in automatico moltiplicata per la grandezza del tipo dell'array.

Le **stringhe** in C sono vettori di caratteri, ossia puntatori a sequenze di bytes, la cui terminazione è definita dal valore convenzionale 0 (zero) che equivale a ¿.
In particolare un vettore di stringhe è un vettore di vettori di caratteri e dunque:

```
char c;         #carattere
char * str;     #vettore di caratteri / stringa
char **strarr; #vettore di vettore di caratteri / vettore di stringhe
```

Si comprende quindi la segnatura della funzione main con $**argv$.

C supporta l'uso delle stringhe, che sono a tutti gli effetti degli array di caratteri, ma possono utilizzare una sintassi differente, più semplificata:

```
int nome[DIM];
long nome[] = {1,2,3,4};
char string[] = "ciao";
char string2[] = {'c','i','a','o'};
nome[0] = 22;
```

Gli array sono generalmente di dimensione statica e non possono essere ingranditi durante l'esecuzione del programma. Per array dinamici dovranno essere usati costrutti particolari (come malloc).
Le stringhe, quando acquisite in input o dichiarate con la sintassi "stringa", terminano automaticamente con il carattere ¿ e sono dunque di grandezza str_len+1.
È fondamentale terminare le stringhe con il carattere ¿, altrimenti non possiamo utilizzare la sintassi semplificata e tutte le funzioni di libreria delle stringhe.

Dunque esistono diversi modi per dichiarare ed inizializzare una stringa, ma questi hanno comportamenti diversi:

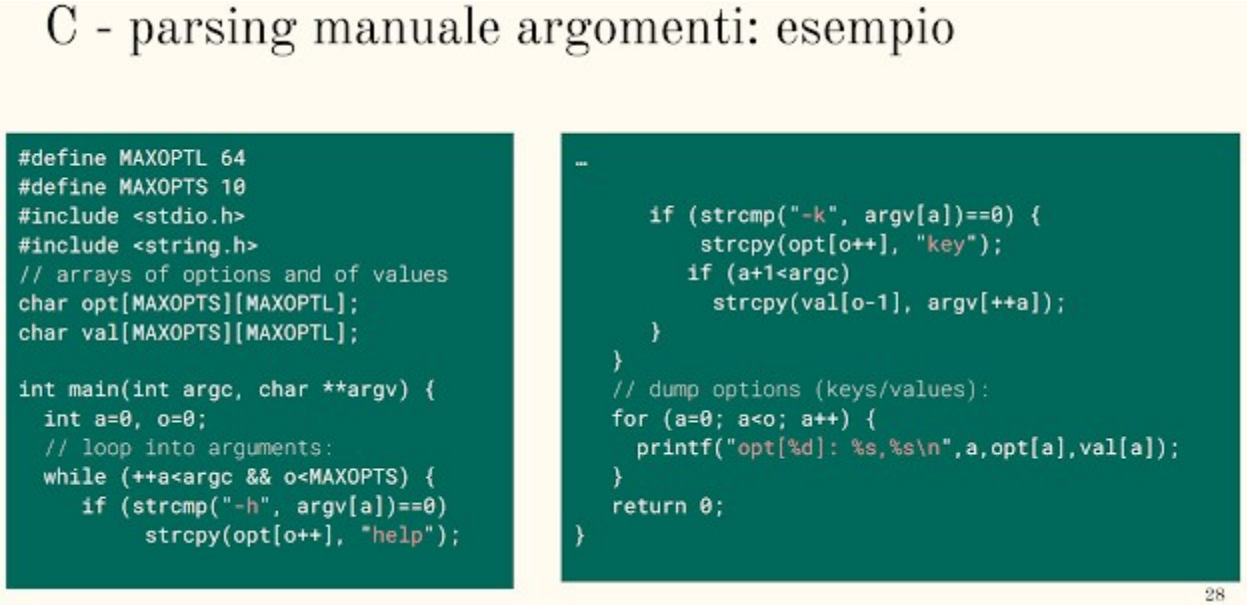
```
char string[] = "ciao"; // writable string in the stack
char *string2 = "ciao";  // READ-ONLY string
string[2] = 'a';
string2[2] = 'a'; // Segmentation fault!
```

Questo perché nel secondo caso, ovvero rappresentazione tramite puntatore a carattere, il compilatore salva la stringa in un'area di memoria read only.

Dato che le stringhe sono riferite con un puntatore al primo carattere non ha senso fare assegnamenti e confronti diretti, ma si devono usare delle funzioni apposite. La libreria standard $string.h$ ne definisce alcune come ad esempio:

```
char * strcat(char *dest, const char *src) aggiunge src in coda a dest
char * strchr(const char *str, int c) cerca la prima occorrenza di c in str
int strcmp(const char *str1, const char *str2) confronta str1 con str2
size_t strlen(const char *str) calcola la lunghezza di str
char * strcpy(char *dest, const char *src)  copia la stringa src in dst
char * strncpy(char *dest, const char *src, size_t n)  copia n caratteri
dalla stringa src in dst
```

La funzione *strlen* è di tipo ¿ t per sottolineare che è una grandezza e distinguerlo da $\int$ ¿, visto che $\int$ ¿ può assumere anche valori negativi.
È meglio usare la funzione *strncpy* rispetto a *strcpy* perchè specificando il numero di caratteri da copiare è più difficile commettere errori (come copiare una stringa grande dentro una stringa piccola).



Interazione con i file: streams and file descriptors

In questo capitolo vedremo in che modo un processo può interagire con i file del sistema operativo.
In UNIX esistono due modi per interagire con i file:

- Gli **streams**
- I **file descriptors**

Un file descriptor è un intero associato a un file aperto o a un dispositivo di I/O. È una rappresentazione "grezza" del file o del dispositivo e viene utilizzato per leggere o scrivere dati. I file descriptor sono comunemente utilizzati nelle chiamate di sistema di basso livello come `open()`, `read()`, `write()`, `close()`. I file descriptor sono numeri interi non negativi, dove 0 rappresenta lo standard input (stdin), 1 rappresenta lo standard output (stdout) e 2 rappresenta lo standard error (stderr).

Gli streams, d'altra parte, sono una rappresentazione più astratta dei file o dei dispositivi di I/O. Sono implementati attraverso la libreria di input/output standard di C (stdio.h) e forniscono funzioni come `fopen()`, `fclose()`, `fread()`, `fwrite()`, `fprintf()`, `fscanf()`, ecc. Gli streams possono essere associati a file, input da tastiera o output su schermo. Uno stream ha un buffer associato che consente di leggere o scrivere i dati in modo più efficiente. Inoltre, gli streams

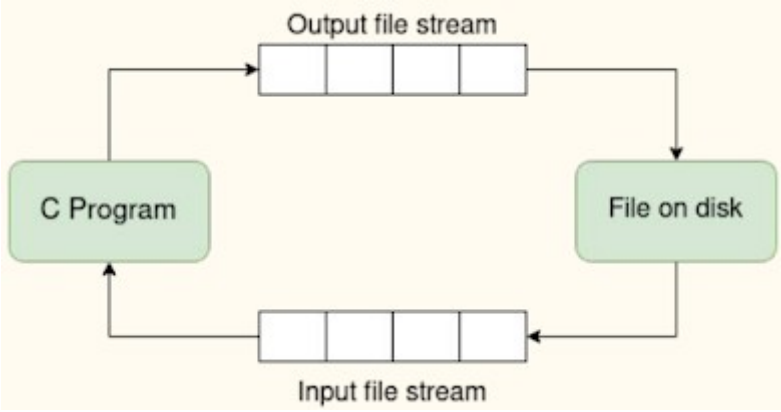
forniscono funzionalità aggiuntive come la gestione del posizionamento del file, il controllo del buffering e la conversione dei dati.

La principale differenza tra file descriptor e stream risiede nella loro implementazione e nell'interfaccia di programmazione associata. I file descriptor offrono un'interfaccia di programmazione di basso livello, che richiede un maggiore coinvolgimento del programmatore nella gestione delle operazioni di I/O, mentre gli streams offrono un'interfaccia di programmazione di alto livello più user-friendly, che semplifica l'interazione con i file e i dispositivi di I/O.

In sintesi, i file descriptor sono più vicini al livello di sistema operativo e richiedono una programmazione più dettagliata, mentre gli streams forniscono un'astrazione più conveniente e semplificata per la gestione dei dati di input e output in C.

Streams

Gli streams sono una sorta di intermediario tra il processo e i file, infatti i dati vengono prima salvati sopra lo stream (sopra dei buffer) e poi vengono letti/scritti. Quindi, quando proviamo a scrivere in un file tramite l'uso di uno stream, la scrittura non è istantanea, ma c'è un piccolo delay dato da questo intermediario.
Quando si scrive su uno stream, i dati vengono memorizzati nel buffer associato a tale stream invece di essere scritti direttamente sul dispositivo sottostante (come un file o la console). Questo buffering consente di migliorare le prestazioni dell'I/O, poiché la scrittura dei dati su un dispositivo può essere un'operazione relativamente costosa in termini di tempo.



Utilizzando gli streams, un file è descritto da un puntatore ad una struttura di tipo $FILE$ (definita in $stdio.h$).
I dati possono essere letti e scritti in vari modi (ad esempio un carattere alla volta, una riga alla volta, etc) ed essere interpretati di conseguenza.

In genere le funzioni le quali il nome inizia con f sono associate all'utilizzo degli streams e le altre all'utilizzo dei file descriptors.

Per **aprire un file** mediante l'utilizzo di stream si usa la funzione $fopen()$:

$$FILE\ *fopen(const\ char*\ filename,\ const\ char*\ mode);$$

la quale restituisce un $FILE$ pointer (restituisce $NULL$ in caso di errore) che permette di gestire il file *filename* (il percorso del file) nella modalità specificata da *mode*. Quando apriamo un file mediante streams, vengono allocati i due buffer visti prima (di input e di output).
La modalità *mode* è una stringa che può essere:

- r: Apre il file in modalità di sola lettura. Il file deve esistere, altrimenti l'apertura fallirà.
- w: Apre il file in modalità di sola scrittura. Se il file esiste già, il suo contenuto viene eliminato. Se il file non esiste, viene creato.
- a: Apre il file in modalità di append. I dati vengono scritti alla fine del file, senza cancellare il contenuto esistente. Se il file non esiste, viene creato.
- $r+$: Apre il file in modalità di lettura e scrittura. Il file deve esistere, altrimenti l'apertura fallirà.
- $w+$: Apre il file in modalità di lettura e scrittura. Se il file esiste già, il suo contenuto viene eliminato. Se il file non esiste, viene creato.
- $a+$: Apre il file in modalità di lettura e append. I dati possono essere letti da qualsiasi posizione nel file, ma vengono sempre scritti alla fine. Se il file non esiste, viene creato.

I permessi devono essere compatibili con quelli del processo/utente su quel file.

È molto importante **chiudere il file** al termine del suo utilizzo per disallocare le risorse precedentemente allocate per gestire quel file e quindi per essere più efficienti, questo si fa con la funzione $fclose()$:

$$\int\ fclose(FILE\ *stream);$$

Ora vedremo le funzioni che ci permettono di interagire con i file, ricordandoci che gli stream mettono a disposizione una serie limitata di funzioni per interagire con i file rispetto ai file descriptor.

Quando andiamo a leggere un file tramite stream, il puntatore dello stream scorre il file, e non è possibile tornare indietro. Le funzioni per **leggere da file** mediante streams sono le seguenti:

- $\int\ fgetc(FILE\ *stream)$
  Restituisce il prossimo carattere puntato dallo stream, più precisamente restituisce un intero che rappresenta un carattere in ASCII
- $char\ *fgets(char\ *str,\ \int\ n,\ FILE\ *stream)$
  Restituisce un puntatore al buffer se la lettura è avvenuta con successo e salva la stringa in str. Si ferma quando $n-1$ caratteri sono stati letti (riservando l'ultimo carattere per il terminatore di stringa), oppure quando viene letto un carattere di nuova linea ¿, oppure quando la fine del file viene raggiunta.
  Questa funzione inserisce anche il carattere di terminazione di stringa in str (per questo legge $n-1$ caratteri), e eventualmente, se viene letto, il carattere ¿.
- $\int\ fscanf(FILE\ *stream,\ const\ char\ *format,\ ...)$
  Questa funzione permette di salvare i dati letti su altri tipi di variabili, infatti riesce ad interpretare il contenuto, convertirlo e inserirlo nella variabile di tipo scelto. Questa

funzione restituisce il numero di variabili di cui è stato fatto correttamente il matching prima di ottenere un errore di matching. Dopo la stringa *format* dobbiamo mettere i puntatori delle variabili in cui inserire i dati, e *format* è una stringa che specifica i tipi delle varie variabili tramite i segnaposto.

- $\int feof(FILE * stream)$
  Restituisce 1 se lo stream ha raggiunto la fine del file (altrimenti 0)

Le funzioni per **scrivere su un file** mediante streams invece sono le seguenti:

- $\int fputc(\int char, FILE * stream)$
  Per scrivere un singolo carattere *char* sul file.
  La funzione restituisce il carattere scritto se l'operazione ha successo, altrimenti restituisce EOF in caso di errore.
- $\int fputs(const\ char * str, FILE * stream)$
  Per scrivere una stringa *str* sul file, senza includere nella stampa il carattere terminatore null!
  È molto importante passare una stringa ben formattata, perché altrimenti la funzione non sa quando fermarsi.
  La funzione restituisce un valore non negativo se l'operazione ha successo, altrimenti restituisce EOF in caso di errore.
- $\int fprintf(FILE * stream, const\ char * format, ...)$
  Per scrivere il contenuto di alcune variabili sul file, seguendo la stringa *format*.
  Come *stream* si possono anche usare i canali di *stdout* e *stderr*, infatti ha lo stesso funzionamento di *printf*, con la differenza che si può scegliere il canale di uscita, infatti $printf("ciao");$ e $fprintf(stdout, "ciao");$ sono equivalenti

Se il file non è stato aperto con i permessi di scrittura, tutte queste funzioni ritornano un errore.

Come abbiamo già detto ci sono dei buffer intermediari tra il processo e il file, e quindi le operazioni non sono immediate (i dati vengono scritti sul buffer e solo successivamente vengono scritti sul file).
Il **flush** è l'operazione che esegue la trascrizione sul file dallo stream. Questa operazione avviene/viene invocata quando:

- Il programma termina con un *return* dal main o con $exit()$ da qualsiasi funzione
- Quando la funzione $fprintf()$ inserisce una nuova riga (¿)
- Quando viene invocata la funzione $\int fflush(FILE * stream)$
- Quando viene invocata la funzione $void\ rewind(FILE * stream)$
- Quando viene invocata la funzione $fclose(FILE * stream)$

Dunque possiamo "forzare" la scrittura sul file tramite la funzione apposita di flush.

Il **rewind** è un meccanismo che ci permette di ripristinare la testina dello stream all'inizio del file.

Ecco due esempi di utilizzo di streams per interagire con i file:

```c
#include <stdio.h>

FILE *ptr;  //Declare stream file
ptr = fopen("filename.txt","r+"); //Open

int id;
char str1[10], str2[10];
while (!feof(ptr)){ //Check end of file
    //Read int, word and word
    fscanf(ptr,"%d %s %s", &id, str1, str2);
    printf("%d %s %s\n",id,str1,str2);
}

printf("End of file\n");
fclose(ptr); //Close file
```

```c
#include <stdio.h>
#define N 10

FILE *ptr;
ptr = fopen("fileToWrite.txt","w+");
fprintf(ptr,"Content to write"); //Write content to file
rewind(ptr); // Reset pointer to begin of file
char chAr[N], inC;
fgets(chAr,N,ptr); // store the next N-1 chars from ptr in chAr
printf("'%d' '%s'",chAr[N-1], chAr);
do{
    inC = fgetc(ptr); // return next available char or EOF
    printf("%c",inC);
}while(inC != EOF); printf("\n");
fclose(ptr);
```

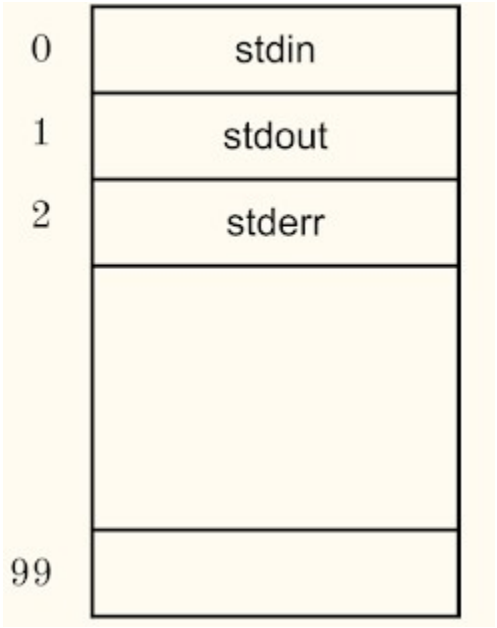
EOF è una macro che posso utilizzare per controllare se un carattere è il carattere di end of file.

File descriptors

Nell'interazione con i file mediante file descriptors, i file sono descritti da dei semplici **interi** (chiamati file descriptors) che puntano alle rispettive entry nella file table del processo mantenuta dal sistema operativo.
Il file descriptor dunque è un intero che funge da identificatore e serve al kernel per identificare il file con cui siamo interessati ad interagire. All'interno di un programma i file descriptor sono univoci. I dati possono essere letti e scritti soltanto un buffer alla volta di cui spetta al programmatore stabilire la dimensione. Con i file descriptor non c'è l'astrazione del buffer e questo ci permette di avere maggiore controllo.

Un insieme di system call permette di effettuare le operazioni di input e output mantenendo un controllo maggiore su quanto sta accadendo a prezzo di un'interfaccia meno amichevole rispetto agli streams.

Il kernel all'avvio di ogni programma, crea ed associa al programma una **file table**, che contiene le associazioni tra i file descriptors e i file.
Per accedere al contenuto di un file bisogna creare un canale di comunicazione con il kernel, aprendo il file con la system call *open*, la quale localizza l'i-node del file e aggiorna la file table del processo.
Dunque ogni processo ha associata una file table, dove ogni elemento rappresenta un file aperto dal processo ed è individuato dal file descriptor.
I file descriptor 0, 1 e 2 individuano normalmente standard input, output ed error (aperti automaticamente)



Ogni voce nella file table contiene:

- **File descriptor**: un numero intero univoco assegnato dal sistema operativo a un file aperto dal processo. Il file descriptor viene utilizzato dal processo per identificare e accedere al file nella tabella dei file aperti.
- **Puntatore all'i-node**: un puntatore a una struttura di dati chiamata i-node (o inode) associata al file aperto. L'i-node contiene metadati importanti del file, come le informazioni sulle dimensioni del file, le autorizzazioni di accesso, il numero di blocchi utilizzati, le informazioni sul proprietario del file e altre informazioni relative alla gestione del file.
- **Posizione corrente nel file**: un puntatore che indica la posizione di lettura o scrittura corrente nel file aperto dal processo. Questa informazione è utile quando il processo esegue operazioni di lettura o scrittura su un file.
- **Flag di controllo**: un insieme di flag che indicano le modalità di apertura del file e le operazioni consentite (lettura, scrittura, append, ecc.).
- Altre informazioni di gestione del file

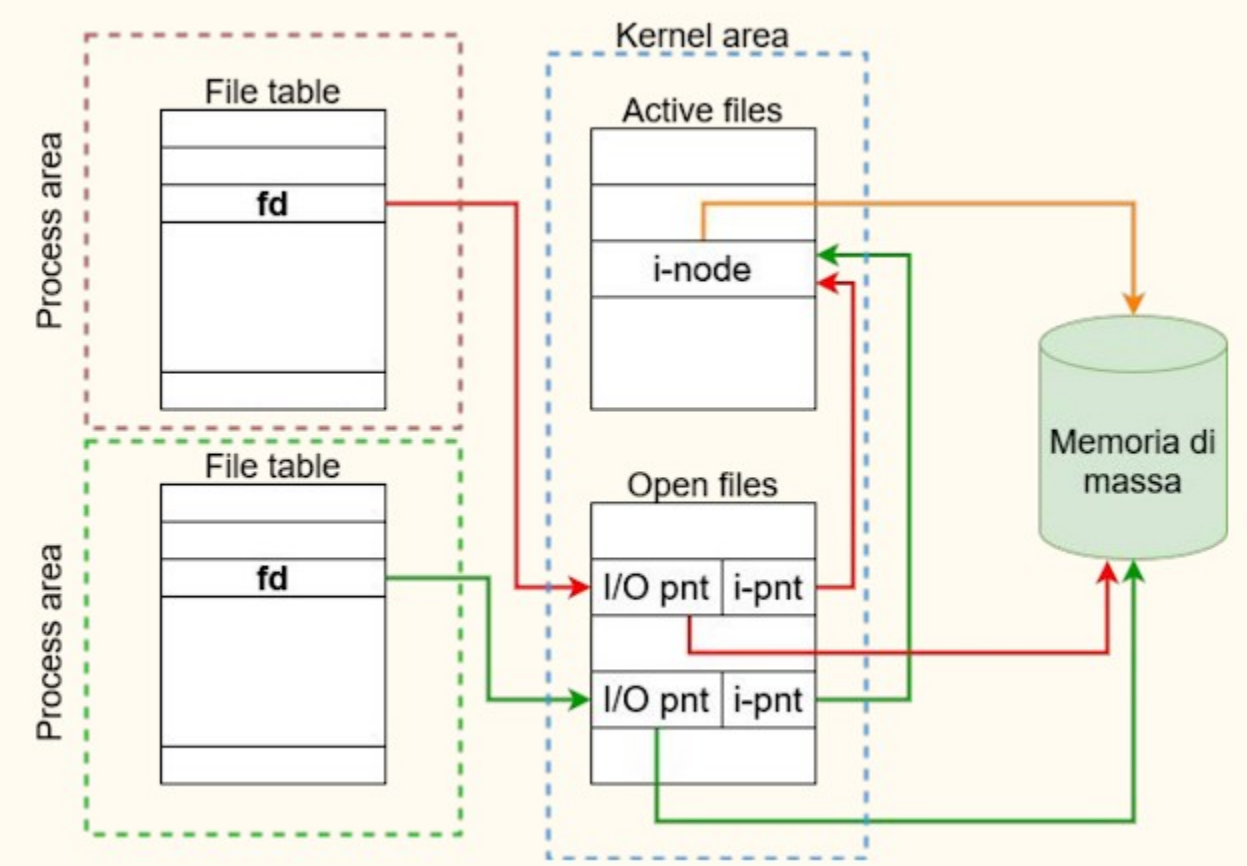
Ogni file nel filesystem ha un i-node associato. Quando un processo apre un file, il sistema operativo cerca l'i-node corrispondente e crea una voce nella tabella dei file aperti del processo, che contiene un puntatore all'i-node del file. Questo collegamento all'i-node consente al sistema operativo di accedere rapidamente alle informazioni del file quando il processo esegue operazioni su di esso, senza dover cercare l'i-node ad ogni operazione.

Oltre alle file table associate ad ogni processo, il kernel gestisce l'accesso ai files attraverso altre due strutture dati:

- **La tabella dei files aperti**
  Questa tabella ha una entry per ogni file presente nelle file table, infatti sopra abbiamo descritto il contenuto della file table e della tabella dei files aperti messi assieme.
  La tabella dei files aperti contiene un elemento per ogni file aperto e non ancora chiuso. Questo elemento consiste in:
    - I/O pointer: un puntatore che indica la posizione di lettura o scrittura corrente nel file aperto dal processo
    - I-node pointer: Puntatore a I-node corrispondente nella tabella dei file attivi
- **La tabella dei files attivi**
  Che contiene una copia dell'i-node di ogni file aperto

La tabella dei file aperti può avere più elementi corrispondenti allo stesso file!

L'i-node (index node) è una struttura di dati utilizzata dal sistema operativo per mantenere le informazioni sul file nel filesystem. Contiene i metadati del file, come le informazioni sulle dimensioni del file, le autorizzazioni di accesso, il timestamp dell'ultima modifica, il numero di blocchi utilizzati dal file e altre informazioni importanti. L'i-node mantiene un collegamento tra il nome del file nel filesystem e i dati effettivi del file memorizzati sul disco.



Per **aprire un file** mediante file descriptors bisogna usare la funzione $open()$:

$$\int open(const\ \ char\ \ *pathname, \int flags\ [,\ mode_t\ \ mode]);$$

dove $pathname$ è il percorso del file e $flags$ definiscono la modalità di apertura del file.
I più comuni interi per $flags$ (che vengono messi in OR, ovvero sommati) sono:

- O_RDONLY, O_WRONLY, O_RDWR
  Almeno uno di questi è obbligatorio
- O_CREAT
  Crea il file se non esiste
- O_EXCL
  Utilizzato insieme a O_CREAT, restituisce un errore se il file esiste già
- O_APPEND
  Apre il file in append-mode (auto $lseek$ con ogni scrittura)
- O_TRUNC
  Tronca il file alla dimensione di zero se esiste già (se usato con la modalità scrittura)

Il parametro $mode$ è opzionale, e consiste sempre in interi (sempre ORed) per specificare i privilegi da assegnare al nuovo file:

- S_IRUSR (o S_IREAD): Il proprietario del file ha il diritto di lettura.
- S_IWUSR (o S_IWRITE): Il proprietario del file ha il diritto di scrittura.

- S_IXUSR (o S_IEXEC): Il proprietario del file ha il diritto di esecuzione (nel caso di file eseguibili).
- S_IRGRP: Il gruppo ha il diritto di lettura.
- S_IWGRP: Il gruppo ha il diritto di scrittura.
- S_IXGRP: Il gruppo ha il diritto di esecuzione.
- S_IROTH: Gli altri utenti hanno il diritto di lettura.
- S_IWOTH: Gli altri utenti hanno il diritto di scrittura.
- S_IXOTH: Gli altri utenti hanno il diritto di esecuzione.
- S_IRWXU: Rappresenta i diritti di accesso per il proprietario del file (user). Questa costante indica che il proprietario ha il diritto di lettura (S_IRUSR), scrittura (S_IWUSR) e esecuzione (S_IXUSR). Quindi, S_IRWXU è una combinazione degli accessi in lettura, scrittura ed esecuzione per il proprietario del file.
- S_IRGRP: Rappresenta i diritti di accesso per il gruppo al quale appartiene il file. Questa costante indica che il gruppo ha il diritto di lettura (S_IRGRP). Il gruppo non ha diritti di scrittura o esecuzione impliciti con questa costante.
- S_IROTH: Rappresenta i diritti di accesso per gli altri utenti (non proprietario né appartenenti al gruppo). Questa costante indica che gli altri utenti hanno il diritto di lettura (S_IROTH). Gli altri utenti non hanno diritti di scrittura o esecuzione impliciti con questa costante.

Per **chiudere il file** usiamo il comando $close()$:

$$\int\ close(\int\ fd);$$

così da liberare l'intero e togliere l'entry dalla file table.

Nei file descriptor abbiamo una sola funzione per leggere e una sola funzione per scrivere.

Per **leggere da file** si usa la funzione $read()$:

$$ssize_t\ read(\int\ fd,\ void\ *buf,\ ¿t\ count);$$

e quello che viene letto viene salvato nel buffer buf fino a $count$ bytes di dati. Viene restituito il numero di bytes che vengono effettivamente letti.

Per **scrivere su file** si usa la funzione $write()$:

$$ssize_t\ write(\int\ fd,\ const\ void\ *buf,\ ¿t\ count);$$

e si scrive sul file fino a $count$ bytes di dati dal buffer buf. Viene restituito il numero di bytes che vengono effettivamente scritti.

Queste due funzioni non inseriscono in automatico il carattere terminatore, il carattere new line o simili, quindi dobbiamo inserirli manualmente ogni volta che le usiamo.

Un'ulteriore funzione molto utile è $off_t\ lseek()$:

$$off_t\ lseek(\int\ fd,\ off_t\ offset,\ \int\ whence);$$

che serve a riposizionare l'offset del file a seconda dell'argomento *offset* partendo da una certa posizione *whence*.
whence può assumere i seguenti valori:

- *SEEK _ SET*: dall'inizio del file
- *SEEK _ CUR*: dalla posizione attuale
- *SEEK _ END*: dalla fine del file

Come si può notare è più complicato utilizzare i file descriptor, però c'è più espressività e meno astrazione.

Ecco due esempi di interazione coi file tramite file descriptors:

```c
#include <unistd.h>
#include <stdio.h>
#include <fcntl.h>

//Open new file in Read only
int openedFile = open("filename.txt", O_RDONLY);
char content[10]; int canRead;
do{
    bytesRead = read(openedFile,content,9); //Read 9B to content
    content[bytesRead]=0;
    printf("%s",content);
} while(bytesRead > 0);
close(openedFile);
```

```c
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
//Open file (create it with user R and W permissions)
int openFile = open("name.txt", O_RDWR|O_CREAT, S_IRUSR|S_IWUSR);
char toWrite[] = "Professor";

write(openFile, "hello  world\n", strlen("hello world\n")); //Write to file
lseek(openFile, 6, SEEK_SET); // move I/O pointer
write(openFile, toWrite, strlen(toWrite)); //Write to file

close(openFile);
```

Canali standard

I canali standard, ovvero in, *out* e *err* che hanno indici/entries 0, 1 e 2 rispettivamente, sono rappresentati come strutture stream sotto le variabili *stdin*, *stdout* e *stderr* e come macro $STDIN_FILENO$, $STDOUT_FILENO$ e $STDERR_FILENO$, che sono i valori dei file descriptors (equivalgono a 0, 1 e 2 rispettivamente).

La funzione *fileno* restituisce l'indice di uno stream, dunque abbiamo che:

- $fileno(stdin) = STDIN_FILENO = 0$
- $fileno(stdout) = STDOUT_FILENO = 1$
- $fileno(stderr) = STDERR_FILENO = 2$

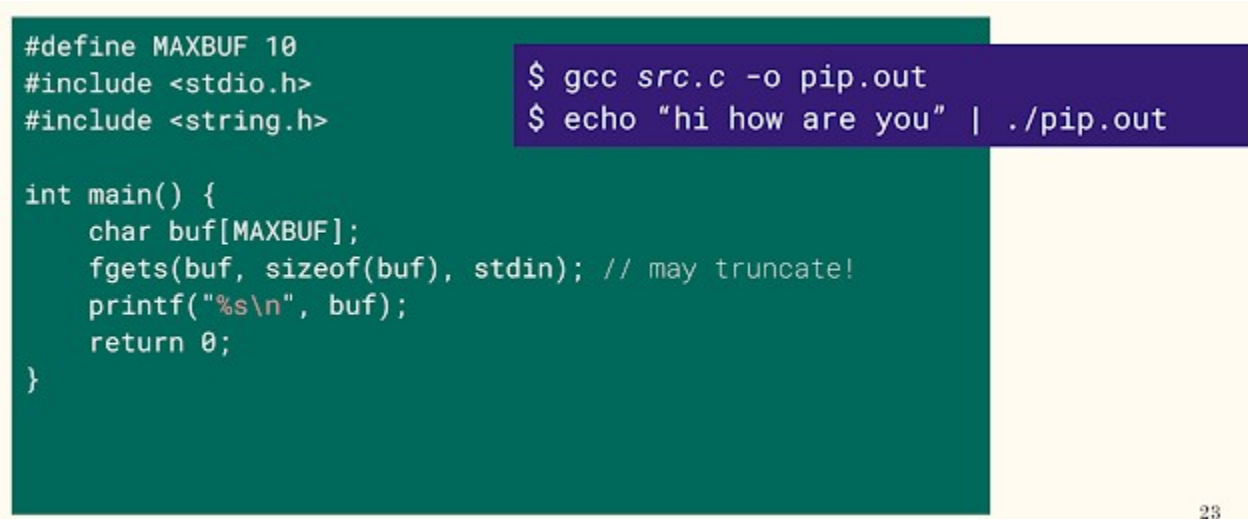
La funzione $isatty(stdin)$ ritorna 1 se l'esecuzione è interattiva (ovvero se l'input arriva dal terminale), altrimenti ritorna 0.

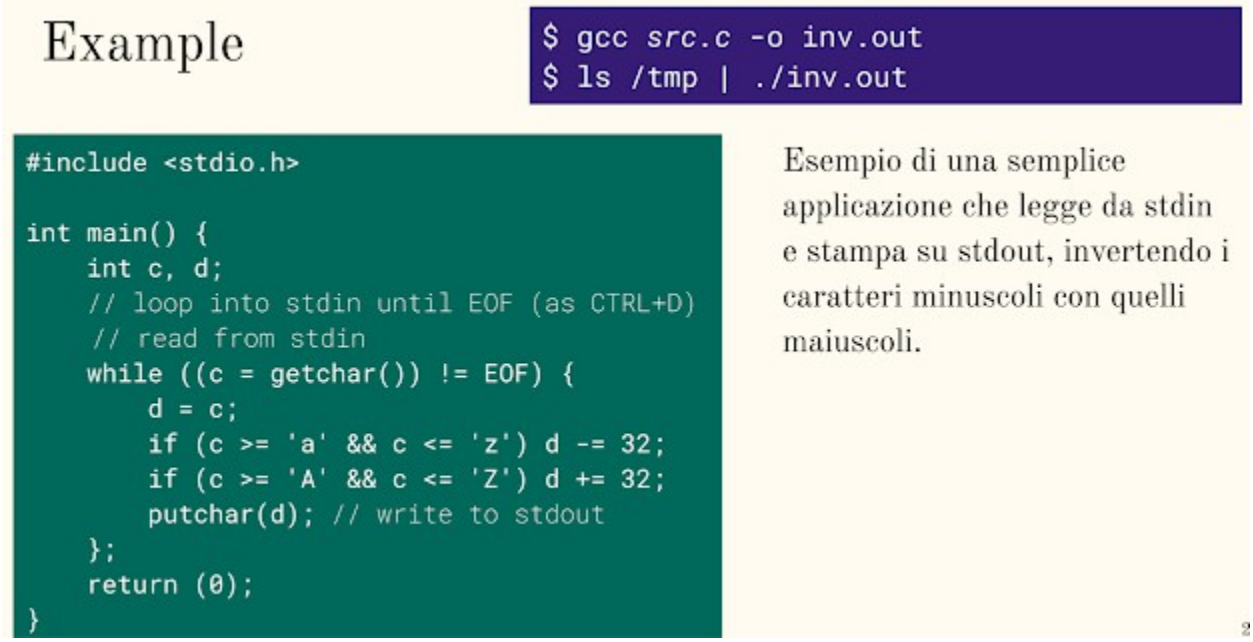
Piping con bash

Normalmente un'applicazione eseguita da bash ha accesso ai canali standard *stdin*, *stdout* e *stderr* (tastiera/video).
Se le applicazioni vengono usate in un'operazione di piping (ad esempio $ls \mid wc\ -l$) allora l'output dell'applicazione sulla sinistra diventa l'input dell'applicazione sulla destra.

Bisogna fare attenzione però perché l'**esecuzione è parallela**, ovvero entrambi i comandi sono eseguiti nello stesso momento.

In questo esempio l'input viene troncato proprio perché l'esecuzione è parallela:





In conclusione, esistono diversi modi per interagire con i file: flussi e descrittori di file.
Ogni approccio ha i suoi pro e i suoi contro e ognuno ruota attorno ad una diversa rappresentazione di un file:

- Gli stream operano su puntatori di file (cioè buffer)
- I descrittori di file operano su interi (cioè voci di una struttura dati)

Comunicazione con il kernel e processi: system calls e forking

Architettura

Il **kernel** è l'elemento di base di un sistema Unix-like, ovvero il nucleo del sistema operativo. Il kernel infatti è incaricato della gestione delle risorse essenziali:

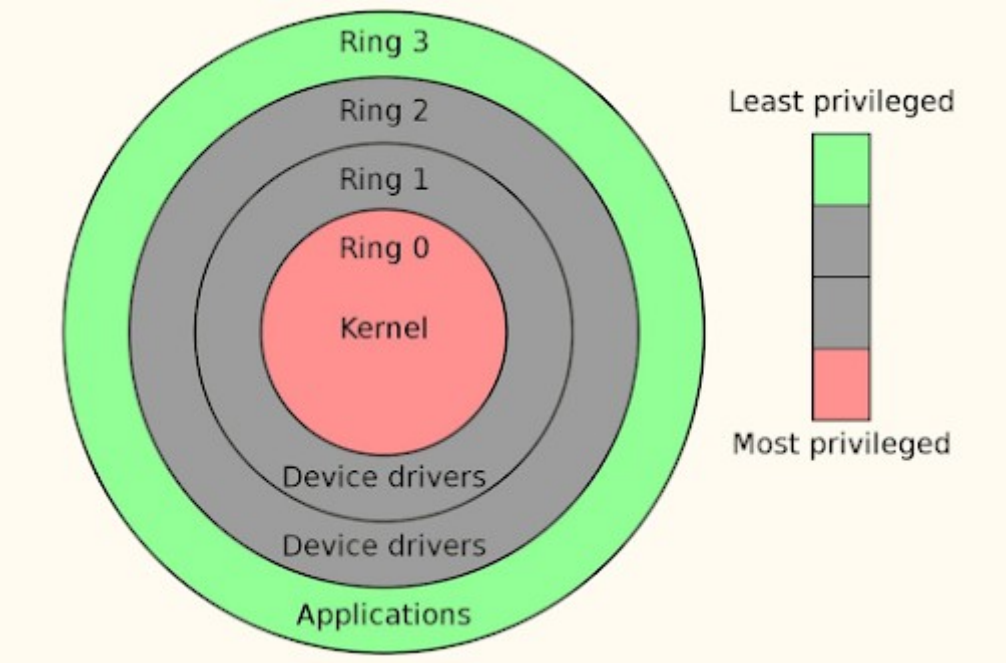
- CPU
- Memoria
- Periferiche
- Ecc…

Ad ogni boot il sistema verifica lo stato delle periferiche, monta la prima partizione (root file system) in read-only e carica il kernel in memoria. Il kernel lancia il primo programma (*systemd*, sostituto di *init*) che, a seconda della configurazione voluta (target), inizializza il sistema di conseguenza.
Il resto delle operazioni, tra cui l'interazione con l'utente, vengono gestite con i programmi eseguiti dal kernel.

I programmi utilizzati dall'utente che vogliono accedere alle periferiche chiedono al kernel di farlo per loro.
L'interazione tra programmi ed il resto del sistema viene mascherata da alcune caratteristiche intrinseche ai processori, come la gestione hardware della **memoria virtuale** (attraverso la MMU). La MMU è responsabile della traduzione degli indirizzi di memoria virtuali generati dai processi in indirizzi fisici corrispondenti nella memoria fisica del sistema. Questa traduzione è fondamentale per consentire l'utilizzo efficiente della memoria virtuale e fornire un'astrazione del sistema di memoria al livello del processo.
Ogni programma vede se stesso come **unico possessore della CPU** e non gli è dunque possibile disturbare l'azione degli altri programmi (stabilità dei sistemi Unix-like).

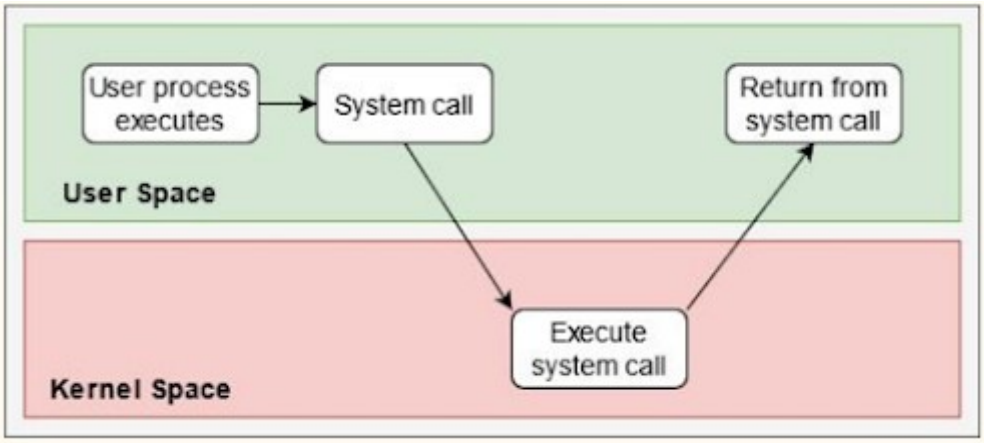
Nei sistemi Unix-like ci sono due livelli di privilegi:

- **User space**
  Ambiente in cui vengono eseguiti i programmi
- **Kernel space**
  Ambiente in cui viene eseguito il kernel



System calls

Le interfacce con cui i programmi accedono all'hardware si chiamano **system calls**, e sono letteralmente "chiamate al sistema" che il kernel esegue nel kernel space, restituendo i risultati al programma chiamante nello user space.



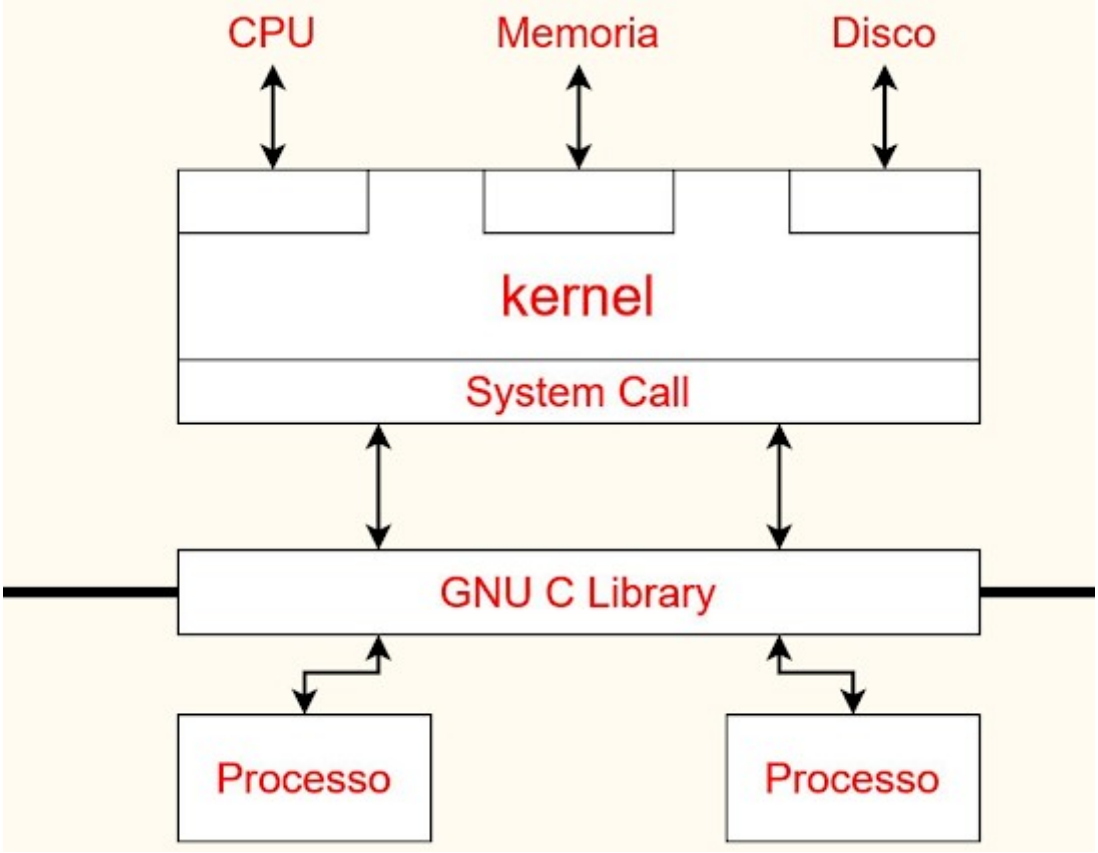
In caso di errore le system call restituiscono il valore -1 e settano la variabile globale $errno$.
Errori validi sono numeri positivi e seguono lo standard POSIX, il quale definisce degli alias.

Utilizzando il comando di shell

ldd ($otool$ $-L$ su mac)

su di un file eseguibile si possono visualizzare le librerie condivise caricate.
Fra le librerie condivise caricate vi sono tipicamente anche:

- $ld-linux.so$
  Quando un programma è caricato in memoria, il sistema operativo passa il controllo a $ld-linux.so$ anziché al normale punto di ingresso dell'applicazione. Questa libreria trova e carica le librerie richieste, prepara il programma e poi gli passa il controllo
- $libc.so$
  La libreria GNU C solitamente nota come glibc che contiene le funzioni basilari più comuni.



Vediamo ora una serie di esempi di system call (si consiglia di leggere attentamente la documentazione di GNU C e la pagina man di linux).

Vediamo le system call della libreria *time.h*, che permettono di **ottenere la data e l'ora attuale**:

Get time: time() e ctime()

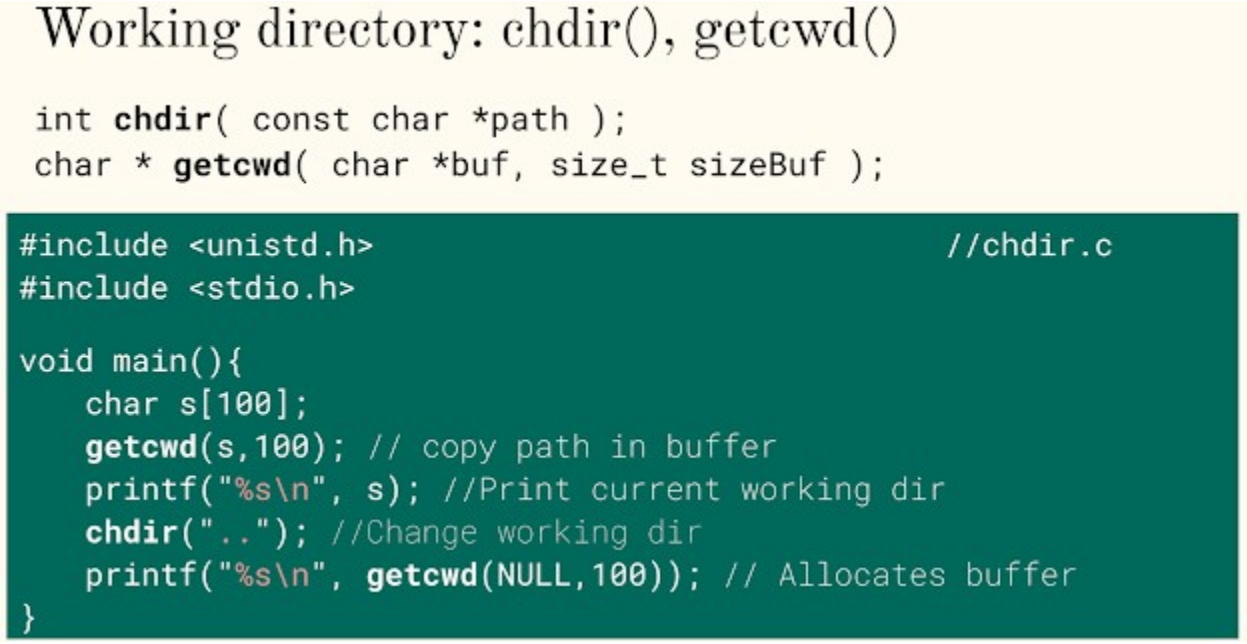
```
time_t time( time_t *second )
char * ctime( const time_t *timeSeconds )
```

```
#include <time.h>                                    //time.c
#include <stdio.h>

void main(){
    time_t theTime;
    time_t whatTime = time(&theTime); //seconds since 1/1/1970
    //Print date in Www Mmm dd hh:mm:ss yyyy
    printf("Current time = %s= %d\n", ctime(&whatTime),theTime);
}
```

La funzione *time()* ritorna ed assegna al parametro passato con riferimento il numero di secondi passati dalla mezzanotte del 1/1/1970.
La funzione *ctime()* prende in input un parametro di tipo *time_t* e ritorna una stringa formattata come una data.

Vediamo le system call della libreria *unistd.h*, che consentono di **lavorare con la working directory**:



La funzione *getcwd()* acquisisce il pathname assoluto della directory corrente di lavoro e lo ritorna.
Il nome viene anche copiato nel buffer puntato da *buf* e di dimensione *sizeBuf*.
Se il pathname assoluto richiede un buffer più grande di size, la funzione fallisce.

La funzione *chdir()* cambia la directory corrente in quella specificata nella *path*.

Le **operazioni con i file** le abbiamo già viste la scorsa volta:

Operazioni con i file

```
int open(const char *pathname, int flags, mode_t mode);
int close(int fd);
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
off_t lseek(int fd, off_t offset, int whence);


FILE *fopen(const char *filename, const char *mode)
int fclose(FILE *stream)

....
```

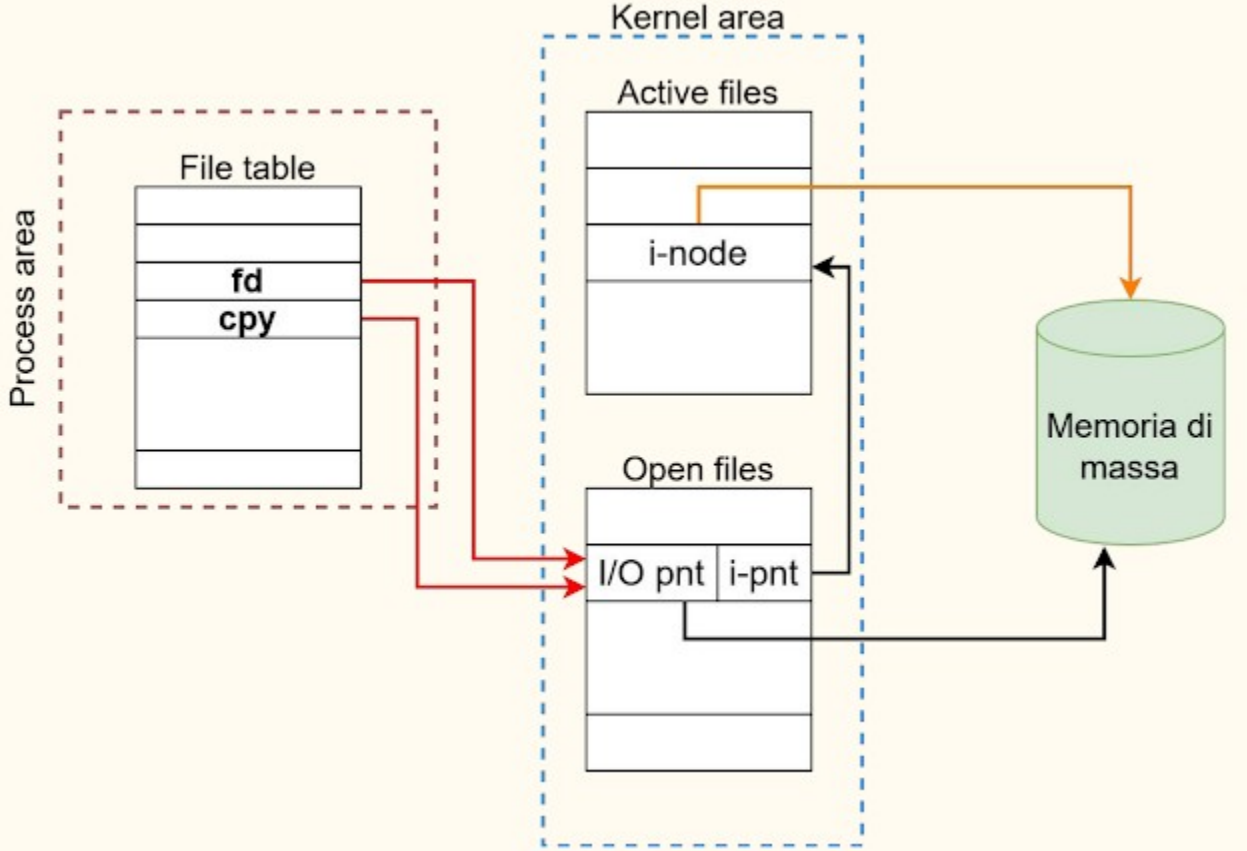
Per **lavorare con i file descriptor, più precisamente per duplicarli**, ci sono le seguenti system calls:

Duplicazione file descriptors: dup(), dup2()

```
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

```c
#include <unistd.h> <stdio.h> <fcntl.h>
int main(void){
    char buf[51];
    int fd = open("file.txt",O_RDWR); //file exists
    int r = read(fd,buf,50); //Read 50 bytes from 'fd' in 'buf'
    buf[r] = 0; printf("Content: %s\n",buf);
    int cpy = dup(fd); // Create copy of file descriptor
    dup2(cpy,22); // Copy cpy to descriptor 22 (close 22 if opened)
    lseek(cpy,0,SEEK_SET); // Move I/O on all 3 file descriptors!
    write(22,"This is a fine\n",16); // Write starting from 0-pos
    close(cpy); //Close ONE file descriptor
}
```

La funzione $dup()$ viene utilizzata per duplicare un descrittore di file esistente. Prende come argomento un descrittore di file aperto e restituisce un nuovo descrittore di file che punta allo stesso file o alla stessa risorsa del descrittore originale. Questo crea una copia del descrittore di file, consentendo di fare riferimento allo stesso file da più punti nel codice prendendo il numero più piccolo disponibile per il nuovo file descriptor.
La funzione $dup2()$ viene utilizzata per duplicare un descrittore di file esistente su un altro descrittore specificato dall'utente. A differenza di $dup()$ $dup2()$ consente di specificare il descrittore di file di destinazione. Se il descrittore di file di destinazione è già aperto, $dup2()$ lo chiude automaticamente prima di duplicare il descrittore di file originale.



Per **cambiare i permessi di un file** ci sono le seguenti system calls:

Permessi: chmod(), chown()

```
int chown(const char *pathname, uid_t owner, gid_t group)
int fchown(int fd, uid_t owner, gid_t group)
int chmod(const char *pathname, mode_t mode)
int fchmod(int fd, mode_t mode)
```

```
#include <fcntl.h>                                          //chown.c
#include <unistd.h>
#include <sys/stat.h>
void main(){
    int fd = open("file",O_RDONLY);
    fchown(fd, 1000, 1000); // Change owner to user:group 1000:1000
    chmod("file",S_IRUSR|S_IRGRP|S_IROTH); // Permission to r/r/r
}
```

```
$ cd .. ; touch file ; ls -la
$ ./executable.o ; ls -la
```

16

Queste funzioni si possono utilizzare sia usando il percorso del file, sia utilizzando i file descriptor per riferirsi al file.
In *mode* utilizziamo le stesse macro che abbiamo visto per l'apertura del file tramite file descriptor.

Vediamo ora la **exec family**.
La famiglia di funzioni "exec" in C si riferisce a un gruppo di funzioni che consentono di eseguire un nuovo programma all'interno di un processo esistente. Queste funzioni includono execl, execle, execlp, execv, execvp e altre varianti. L'uso delle funzioni "exec" è comune nei sistemi operativi di tipo Unix, come Linux.
Le funzioni "exec" sostituiscono il codice del processo corrente con un nuovo programma specificato dall'utente. Esse consentono di eseguire programmi diversi all'interno di un processo senza dover creare un nuovo processo separato.
Una volta chiamata una delle funzioni "exec", il processo corrente viene sovrascritto dal nuovo programma, e l'esecuzione continua da lì. Tuttavia, le proprietà del processo originale, come l'ID del processo (PID) e i file descriptor aperti, vengono mantenute nel nuovo programma a meno che non vengano specificamente modificati.
Le funzioni "exec" restituiscono un valore solo se si verifica un errore durante l'esecuzione. In caso di successo, il controllo non viene mai restituito alla funzione chiamante.

La famiglia di funzioni $exec$ ha come scopo finale l'esecuzione di un programma, sostituendo l'immagine del processo corrente con una nuova immagine. Può essere usato per runnare un programma C all'interno di un altro programma C. **Il PID del processo e la sua file table non cambiano**!
La chiamata di sistema di base è $execve()$, ma vedremo tutti i suoi alias che differiscono solo per gli argomenti accettati, mantenendo lo stesso identico comportamento. Ogni alias è composto dalla parole chiave $exec$ seguita dalle seguenti lettere:

- l: Indica che gli argomenti della funzione vengono passati come elenco di parametri separati nella chiamata alla funzione (una lista di argomenti)
- v: Indica che gli argomenti della funzione vengono passati come array di stringhe
- p: Indica che la funzione utilizzerà il percorso di ricerca nell'environment per cercare l'eseguibile specificato. Questo significa che non è necessario fornire un percorso assoluto per il programma da eseguire, ma può essere specificato solo il nome dell'eseguibile. La funzione quindi cercherà l'eseguibile nel percorso di ricerca specificato nell'environment (variabile $PATH$).
- e: Indica che la funzione permette di specificare un environment da passare al nuovo programma. Questo consente di modificare o aggiungere variabili d'ambiente per il programma in esecuzione. Usa un vettore di variabili d'ambiente (ad esempio $name = value$)

Da notare che ogni vettore di elementi deve terminare con l'elemento $NULL$!

```
int execv     (const char *path, char *const argv[])
int execvp    (const char *file, char *const argv[])
int execve    (const char *path, char *const argv[],
               char *const envp[]);
int execvpe   (const char *file, char *const argv[],
               char *const envp[])

int execl  (const char *path, const char * arg0,…,argn,NULL)
int execlp (const char *file, const char * arg0,…,argn,NULL)
int execle (const char *path, const char * arg0,…,argn,NULL,
            char *const envp[])
int execlpe(const char *file, const char * arg0,…,argn,NULL,
            char *const envp[])
```

Esempio: execv()

```c
#include <unistd.h>                                          //execv1.out
#include <stdio.h>
void main(){
    char * argv[] = {"par1","par2",NULL};
    execv("./execv2.out",argv); //Replace current process
    printf("This is execv1\n");
}
```

```c
#include <stdio.h>                                           //execv2.out
void main(int argc, char ** argv){
    printf("This is execv2 with %s and %s\n",argv[0],argv[1]);
}
```

Esempio: execle()

```c
#include <unistd.h>                                          //execle1.out
#include <stdio.h>
void main(){
    char * env[] = {"CIAO=hello world",NULL};
    execle("./execle2.out","par1","par2",NULL,env); //Replace proc.
    printf("This is execle1\n");
}
```

```c
#include <stdio.h>                                           //execle2.out
#include <stdlib.h>
void main(int argc, char ** argv){
    printf("This is execle2 with par: %s and %s. CIAO =
            %s\n",argv[0],argv[1],getenv("CIAO"));
}
```

20

Esempio: dup2/exec

```c
#include <stdio.h>                                           //execvpDup.c
#include <fcntl.h>
#include <unistd.h>

void main() {
  int outfile = open("/tmp/out.txt",
    O_RDWR | O_CREAT, S_IRUSR | S_IWUSR
  );
  dup2(outfile, 1); // copy outfile to FD 1
  char *argv[]={"./time.out",NULL}; // time.out della slide#10
  execvp(argv[0],argv); // Replace current process
}
```

21

Per **chiamare la shell** esiste la system call:

Chiamare la shell: system()

```
int system(const char * string);
```

```
#include <stdlib.h> <stdio.h>                                    //system.c
#include <sys/wait.h> /* For WEXITSTATUS */

void main(){
    int outcome = system("echo ciao"); // execute command in shell
    printf("Outcome = %d\n",outcome);
    outcome = system("if [[ $PWD < \"ciao\" ]]; then echo min; fi");
    printf("Outcome = %d\n",outcome);
    outcome = system("notExistingCommand");
    printf("Outcome = %d\n",WEXITSTATUS(outcome));
}
```

22

Ci sono molte altre system calls per la gestione dei processi e della comunicazione tra i processi che saranno discusse più avanti.

Forking

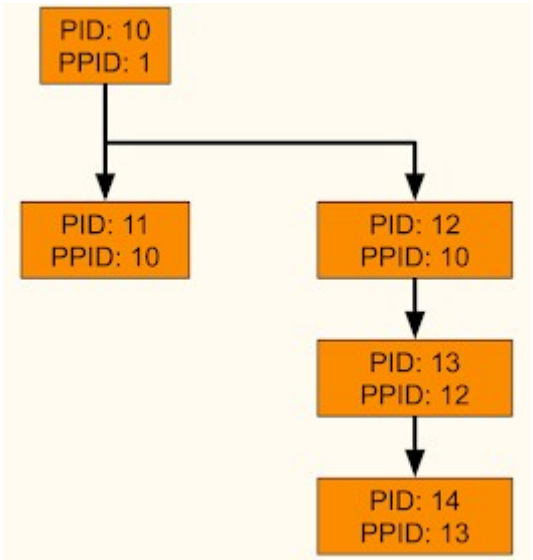
Il **forking** è la “generazione” di nuovi processi (uno alla volta) partendo da uno esistente. La syscall principale per il forking è $fork()$.
Quando un processo attivo invoca questa syscall, il kernel lo “clona” modificando però alcune informazioni, in particolare quelle che riguardano la sua collocazione nella gerarchia complessiva dei processi.
Il processo che effettua la chiamata è definito “padre/genitore”, quello generato è definito “figlio”.

Ad ogni processo è associato un **identificativo univoco** per istante temporale. I processi sono organizzati gerarchicamente (genitore-figlio) e suddivisi in insiemi principali (sessioni) e secondari (gruppi).
Anche gli utenti hanno un loro identificativo e ad ogni processo ne sono abbinati due: quello reale e quello effettivo (di esecuzione):

- **PID** (Process ID): un identificatore univoco assegnato a ciascun processo in esecuzione su un sistema operativo. Ogni processo ha un PID unico che viene utilizzato per identificarlo e gestirlo dal sistema operativo.
- **PPID** (Parent Process ID): l'identificatore del processo padre di un dato processo. Ogni processo, ad eccezione del processo di inizializzazione del sistema operativo, ha un processo padre dal quale è stato creato. Il PPID viene utilizzato per stabilire la gerarchia dei processi, consentendo al sistema operativo di gestire i processi in modo organizzato.
- **SIDB** (Session ID): un identificatore che rappresenta una sessione di login sul sistema operativo. Una sessione viene creata quando un utente accede al sistema e viene associata a tutti i processi avviati durante quella sessione. Il SID viene utilizzato per gestire e controllare l'accesso e le risorse all'interno di una sessione.

- **PGID** (Process Group ID): un identificatore assegnato a un gruppo di processi correlati. I processi possono essere organizzati in gruppi per scopi di gestione e controllo. Ad esempio, un terminale shell può creare un gruppo di processi per tutti i comandi che vengono eseguiti da quella shell.
- **UID/RUID** (Real User ID): l'identificatore dell'utente reale che ha avviato un processo. Indica l'utente che ha effettuato l'accesso al sistema operativo e ha eseguito il processo. Viene utilizzato per stabilire le autorizzazioni e i privilegi di accesso associati all'utente che ha avviato il processo.
- **EUID** (Effective User ID): l'identificatore dell'utente effettivo che possiede i privilegi di accesso per un processo. In alcuni casi, un processo può essere eseguito con privilegi diversi rispetto all'utente che lo ha avviato. L'EUID viene utilizzato per determinare i privilegi effettivi associati al processo, indipendentemente dal RUID.

Ecco un esempio di gerarchia padre-figlio:



Vediamo nello specifico come vengono gestiti gli elementi clonati e gli elementi nuovi nell'operazione di $fork$.
Sono clonati gli elementi principali come il PC (Program Counter), i registri, la tabella dei file (file descriptors) e i dati di processo (variabili). Le meta-informazioni come il "pid" e il "ppid" sono aggiornate (al contrario di quando si utilizza $execve()$!).
L'esecuzione procede per entrambi (quando saranno schedulati!) da $PC+1$ (tipicamente l'istruzione seguente il fork o la valutazione dell'espressione in cui essa è utilizzata)

Prossimo step: `printf`	Prossimo step: assegnamento ad `f`
`fork();` `printf("\n");`	`f=fork();` `printf("\n");`

Le system call per **ottenere i pid** sono le seguenti:

`pid_t getpid()` : restituisce il PID del processo attivo
`pid_t getppid()` : restituisce il PID del processo genitore

```c
#include <stdio.h> <unistd.h> <stdlib.h>                    //ppid.c

void main(){
    printf("Subshell $$ = ");
    fflush(stdout); // Forza l'output di printf
    system("echo $$"); // subshell
    printf("PID: %dPPID: %d\n",getpid(),getppid());
}
```

(includendo `<sys/types.h>` e `<sys/wait.h>`: `pid_t` è un intero che rappresenta un id di processo)

Il valore di ritorno della syscall *fork()* è importante (tipicamente viene catturato da una variabile o usato in un'espressione). Come tutte le syscall il valore è -1 in caso di errore.
Se ha successo entrambi i processi ricevono un valore di ritorno, ma questo è diverso nei due casi:

- Il processo genitore riceve come valore il nuovo PID del processo figlio
- Il processo figlio riceve come valore 0

Esempio

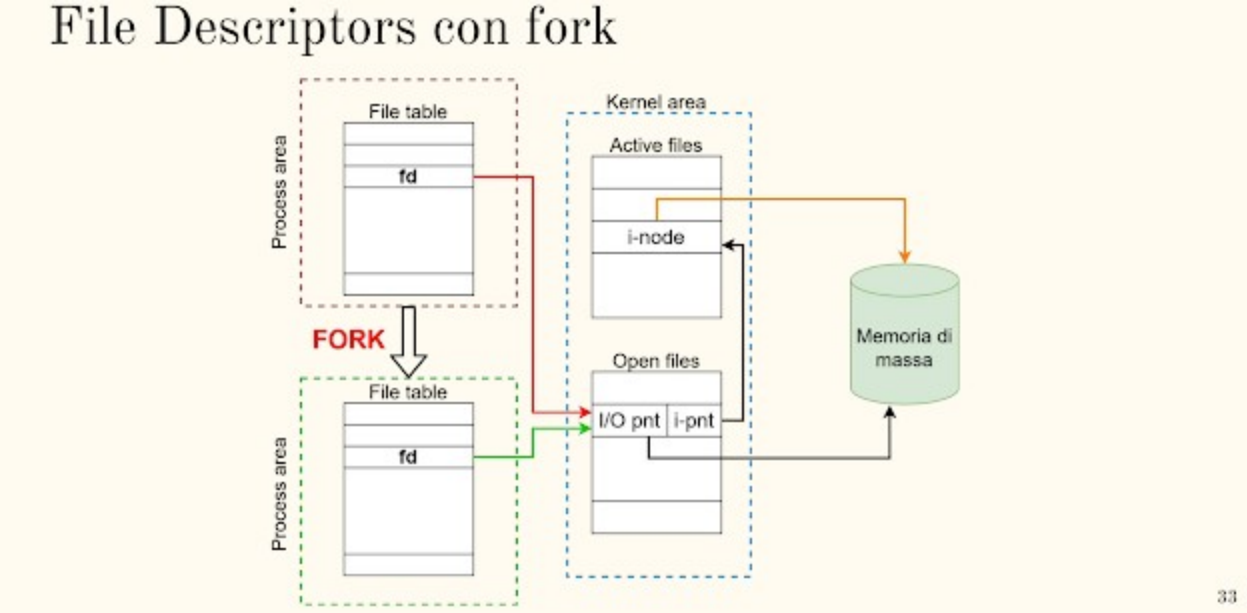
```c
#include <stdio.h>   <unistd.h>
int main(void) {
    int result = fork();
    if(result == 0){
        printf("I'm the child\n");
        return 0;
    }
    else{
        printf("I'm the parent\n");
    }
    return 0;
}
```

```c
int isChild = !fork();
int isParent = fork()!=0
```

Dunque i processi che hanno relazione genitore-figlio:

- Conoscono reciprocamente il loro PID (ciascuno conosce il proprio tramite *getpid()*, il figlio conosce quello del genitore con *getppid()*, il genitore conosce quello del figlio come valore di ritorno di *fork()*)
- Si possono usare altre syscall per semplici interazioni come *wait* e *waitpid*

- Eventuali variabili definite prima del fork sono valorizzate allo stesso modo in entrambi: se riferiscono risorse (ad esempio un “file descriptor” per un file su disco) fanno riferimento esattamente alla stessa risorsa (e dunque il cambiamento da un lato si percepisce anche dall’altro)



La funzione $wait()$:

$$pid_t \ \ wait \ (\int \ *status)$$

attende il cambio di stato di un processo figlio (uno qualsiasi) restituendone il PID e salvando in status lo stato del figlio (se il puntatore non è $NULL$).
Il **cambio di stato** avviene se il figlio viene terminato o la sua esecuzione interrotta/ripresa a seguito di un segnale (vedremo prossimamente i segnali).
Se non esiste alcun figlio restituisce immediatamente -1.
Nel nostro caso ci interessa principalmente la terminazione del figlio. Questa system call ci permette di bloccare il processo padre (anche sincronizzarlo) fino a quando il figlio non ha finito le sue operazioni.
Il comando

$$while(wait(NULL)>0);$$

fa aspettare al processo padre la terminazione di tutti i figli.

Lo **stato di ritorno** è un numero che comprende più valori “composti” interpretabili con apposite macro, molte utilizzabili a mo’ di funzione (altre come valore) passando lo “stato” ricevuto come risposta come ad esempio:

`WEXITSTATUS(sts)`: restituisce lo stato vero e proprio (ad esempio il valore usato nella “exit”).
`WIFCONTINUED(sts)`: true se il figlio ha ricevuto un segnale SIGCONT.
`WIFEXITED(sts)`: true se il figlio è terminato normalmente.
`WIFSIGNALED(sts)`: true se il figlio è terminato a causa di un segnale non gestito.
`WIFSTOPPED(sts)`: true se il figlio è attualmente in stato di “stop”.
`WSTOPSIG(sts)`: numero del segnale che ha causato lo “stop” del figlio.
`WTERMSIG(sts)`: numero del segnale che ha causato la terminazione del figlio.

```c
#include <stdio.h> <unistd.h> <sys/wait.h>
int main(void) {
    int isChild = !fork();
    if(isChild){
        sleep(3); return 5;
    }
    int childStatus;
    wait(&childStatus);
    printf("Children terminated? %d\nReturn code: %d\n",
        WIFEXITED(childStatus),WEXITSTATUS(childStatus));
    return 0;
}
```

36

Per **specificare quale processo aspettare** in una wait si usa la funzione $waitpid()$

$$pid_t \ \ waitpid(pid_t \ \ pid, \int *status, \int options)$$

che consente un’attesa selettiva basata sui parametri.
pid può essere:

- $-n$: aspetta un figlio qualsiasi nel gruppo $|-n|$ (il concetto di gruppo lo vedremo nelle prossime lezioni)
- -1: aspetta un figlio qualsiasi
- 0: aspetta un figlio qualsiasi appartenente allo stesso gruppo del padre
- n: aspetta il figlio con $PID = n$

options è un parametro che consiste nei seguenti parametri ORed:

- *WNOHANG*: ritorna immediatamente se nessun figlio è terminato (non si resta in attesa!)
- *WUNTRACED*: ritorna anche se un figlio si è interrotto senza terminare
- *WCONTINUED*: ritorna anche se un figlio ha ripreso l’esecuzione

$wait(st)$ è l’equivalente di $waitpid(-1, \ st, \ 0)$

Ovviamente è possibile utilizzare più fork all’interno di un codice (creando figli dei figli etc):

```c
#include <stdio.h>            //fork1.c
#include <unistd.h>
int main() {
  fork(); fork(); fork();
  printf("hello\n");
  return 0;
}
```

Ecco un esempio di fork and wait:

```c
#include <stdio.h> <stdlib.h> <unistd.h> <time.h> <sys/wait.h>        //fork2.c
int main() {
  int fid=fork(), wid, st, r; // Generate child
  srand(time(NULL)); // Initialise random
  r=rand()%256; // Get random between 0 and 255
  if (fid==0) {  //If it is child
     printf("Child... (%d)", r); fflush(stdout);
     sleep(3);  // Pause execution for 3 seconds
     printf(" done!\n");
     exit(r); // Terminate with random signal
  } else { // If it is parent
     printf("Parent...\n");
     wid=wait(&st);  // wait for ONE child to terminate
     printf("...child's id: %d==%d (st=%d)\n", fid, wid, WEXITSTATUS(st));
  }
}
```

Normalmente quando un processo termina il suo stato di uscita viene “catturato” dal genitore: alla terminazione il sistema tiene traccia di questo insieme di informazioni (lo stato) fino a che il genitore le utilizza consumandole (con *wait* o *waitpid*). Se il genitore non cattura lo stato d’uscita, i suoi processi figli vengono definiti **zombie** (in realtà non ci sono più, ma esiste un riferimento in sospeso nel sistema).

Se un genitore termina prima del figlio, quest’ultimo viene definito **orfano** e viene “adottato” dal processo principale (tipicamente “systemd” con *PID* pari a 1).

Un processo zombie che diventa anche orfano è poi gestito dal processo che lo adotta (che effettua periodicamente dei *wait*/*waitpid* appositamente per questo scopo).

Per ispezionare la lista dei processi attivi si può usare il comando *ps* (process status):

- *ps*: Esegue il comando *ps* senza argomenti, visualizzando i processi associati all'utente corrente nella forma di una lista. Mostra solo i processi in esecuzione nella shell corrente.
- *ps -a*: Mostra tutti i processi associati all'utente corrente, inclusi i processi di altri utenti e i processi del sistema, ad eccezione dei processi controllati da processi demoni.

- $ps\ -H$: è utilizzato per visualizzare una rappresentazione ad albero dei processi, mostrando la gerarchia dei processi figlio-padre.
- $ps\ -e$ o $ps\ -A$: Visualizza tutti i processi di tutti gli utenti in esecuzione sul sistema.
- $ps\ -f$: Visualizza informazioni dettagliate sui processi in un formato a colonna. Include informazioni come l'UID, il PID, il PPID, il C (utilizzo della CPU), lo stato (STAT), il tempo di CPU, il comando eseguito, ecc.

In conclusione, tramite l'uso dei file-descriptors, di fork e della famiglia di istruzioni exec è possibile generare più sottoprocessi e "redirezionare" i loro canali di in/out/err.
Sfruttando anche $wait$ e $waitpid$ è possibile costruire un albero di processi che interagiscono tra loro (non avendo ancora a disposizione strumenti dedicati è possibile sfruttare il file-system, ad esempio con file temporanei, per condividere informazioni/dati).

Inter-Process-Communication: signals

Ci sono vari eventi che possono avvenire in maniera asincrona rispetto al normale flusso di un programma, alcuni dei quali in maniera inaspettata e non predicibile (ad esempio una richiesta di terminazione o di sospensione del programma da parte di un utente, la terminazione di un processo figlio o un errore generico).
Unix prevede la gestione di questi eventi attraverso i **segnali**: quando il sistema operativo si accorge di un certo evento, genera un segnale da mandare al processo interessato il quale potrà decidere (nella maggior parte dei casi) come comportarsi. Quando un processo rileva che gli è stato inviato un segnale, è tenuto a rispondere.

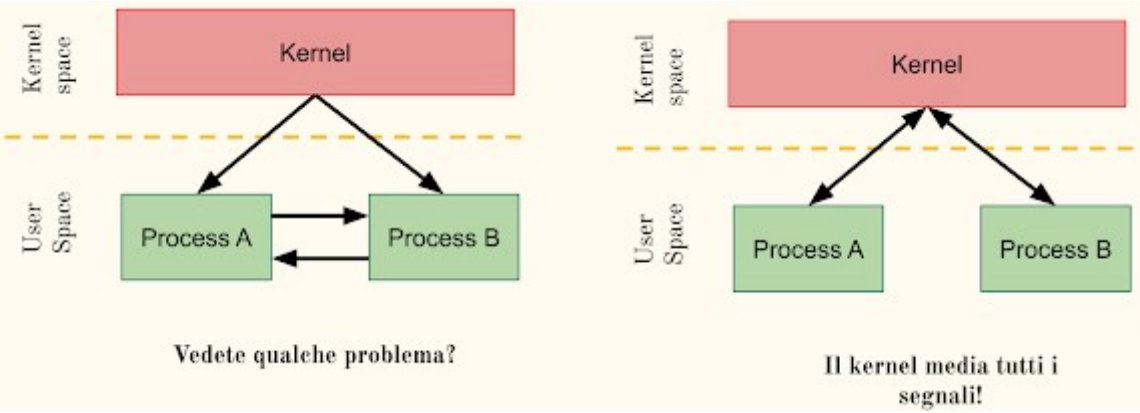
Ci sono vari tipi di segnale, e il numero di segnali cambia a seconda del sistema operativo. Linux definisce 32 tipi di segnale.
Ogni segnale viene associato ad un **valore numerico** e ad una **etichetta mnemonica** nel formato $SIGXXX$ (definita nella libreria $signal.h$). Ecco alcuni esempi dei segnali più importanti e usati:

- $SIGALRM$: Generato quando scade un timer impostato da $alarm()$ o $setitimer()$.
- $SIGCHLD$: Generato quando un processo figlio termina o viene arrestato.
- $SIGCONT$: Usato per riprendere l'esecuzione di un processo precedentemente sospeso (in background) (ad esempio $CTRL+Q$).
- $SIGINT$: Generato dall'utente per interrompere un programma in esecuzione in modo interattivo.
- $SIGKILL$: Usato per terminare un processo in modo forzato.
- $SIGQUIT$: Simile a SIGINT, ma generato quando l'utente richiede un dump del core del processo (ad esempio $CTRL+¿$).
- $SIGSTOP$: Usato per sospendere l'esecuzione di un processo ($CTRL+S$).
- $SIGTSTP$: Generato quando viene premuto il tasto di sospensione (di solito Ctrl+Z) per sospendere un processo. ($CTRL+Z$)
- $SIGTERM$: Usato per terminare un processo in modo ordinato. (è come un "soft kill", infatti il programma può decidere di ignorare questo segnale, e non killa i processi figli)

- *SIGSYS*: Questo segnale viene generato quando viene eseguita un'invocazione di sistema non valida o non supportata. Solitamente si verifica quando un programma tenta di utilizzare un'istruzione di sistema che non è consentita o non è supportata dal sistema operativo (in genere a noi non succederà mai visto che utilizziamo sempre funzioni wrapper e non chiamiamo mai system call in maniera “cruda”).
- *SIGUSR*1: Segnale personalizzato riservato all'uso dell'utente.
- *SIGUSR*2: Segnale personalizzato riservato all'uso dell'utente.

Per il nostro scopo sono molto importanti gli ultimi due, ovvero *SIGUSR*1 e *SIGUSR*2, i quali non hanno azioni predefinite (di default fanno il quit) ma sono fatti apposta per venire programmati dall’utente. Non è obbligatorio usare questi segnali per programmare delle azioni personalizzate, infatti possiamo modificare anche le azioni degli altri segnali, però i segnali standard vengono invocati anche dal kernel e quindi potremmo avere dei comportamenti indesiderati.

Nella realtà un segnale non viene inviato direttamente da un processo ad un altro, perché questa comunicazione (come vediamo nell’immagine) viene intermediata sempre dal kernel.

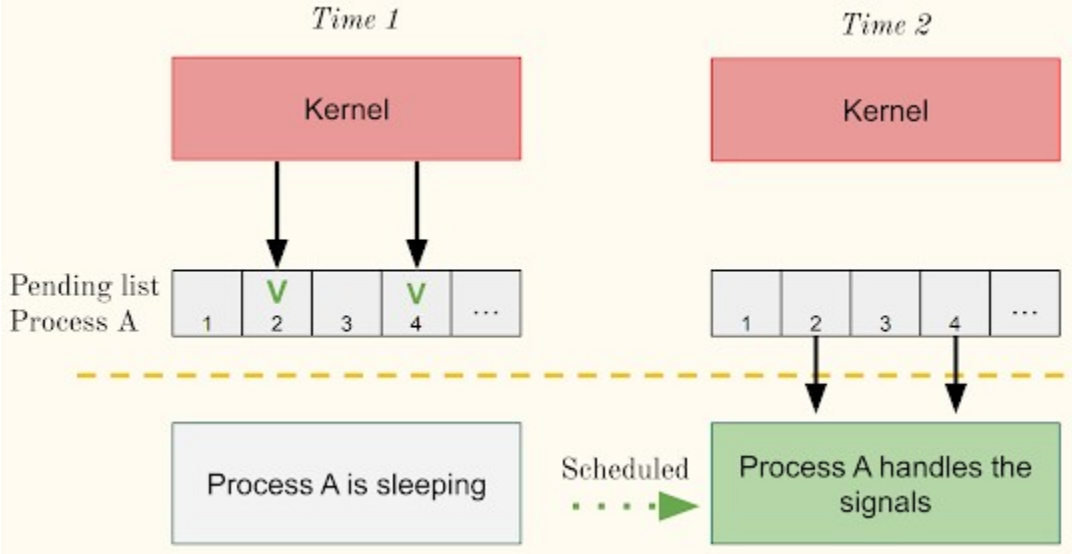


Per la gestione dei segnali, nella process table per ogni processo vengono mantenute due liste:

- **Pending signals**
  Questa lista contiene i segnali emessi dal kernel e che il processo deve ancora gestire
- **Blocked signals**
  Questa lista contiene i segnali che non devono essere comunicati al processo

La lista blocked signals può anche essere vista come una maschera dei segnali (signal mask) visto che viene applicata come un filtro alla lista pending signals.

Ad ogni schedulazione del processo queste due liste vengono controllate per consentire al processo di reagire nella maniera più adeguata. Visto che il controllo sulle liste avviene solamente quando il processo viene schedulato, il segnale tipicamente non viene gestito immediatamente: la lista pending signals viene aggiornata immediatamente, ma il processo rileverà il segnale solamente quando è il suo turno:



La gestione dei segnali dunque ha priorità massima all'interno del processo, infatti si dice che la gestione dei segnali è preemptive (in quanto interrompiamo la normale esecuzione del processo per eseguire le funzionalità che gestiscono il segnale).

I segnali sono anche detti **software interrupts** perchè sono, a tutti gli effetti, delle interruzioni del normale flusso del processo generate dal sistema operativo (invece che dall'hardware, come per gli hardware interrupts).
Come per gli interrupts, il programma può decidere come gestire l'arrivo di un segnale (presente nella lista pending):

- Eseguire l'azione di **default** (non è detto che esista)
- Ignorare il segnale (non è sempre possibile, come ad esempio per *SIGKILL*) e far proseguire normalmente il programma
- Eseguire un handler personalizzato e far interrompere il programma per eseguire l'handler

Anche se concettualmente possiamo pensare che sia il programma che consulta le due liste e esegue la funzionalità adeguata per ogni segnale, in realtà è il kernel che viene informato dal programma su quale funzione chiamare per ogni segnale, e le chiama direttamente lui stesso non appena il processo viene schedulato.

In genere ogni segnale ha un suo **default handler**, e tipicamente esegue una tra le seguenti azioni:

- Ignorare il segnale
- Terminare il processo
- Continuare l'esecuzione (se il processo era in stop)
- Stoppare il processo

L'handler di default può essere sostituito da una funzione "custom" e comportarsi di conseguenza al rilevamento del segnale. Gli unici segnali che non permettono la modifica dell'handler sono *SIGKILL* e *SIGSTOP*.

La sostituzione dell'handler viene fatta tramite la funzione $signal()$ (definita in $signal.h$)

La funzione $signal()$ ha la seguente firma

$$sighandler_t \ \ signal(\int \ \ signum, \ \ sighandler_t \ \ handler);$$

e imposta il nuovo signal handler *handler* al segnale corrispondente al numero *signum*. Questa funzione restituisce il signal handler precedente.
handler può assumere i valori:

- SIG_DFL: handler di default per quel segnale
- SIG_IGN: per ignorare il segnale
- $typedef \ \ void \ \ (*sighandler_t)(\int)$: handler personalizzato

```
#include <signal.h> <stdio.h> <stdlib.h>
void main(){
    signal(SIGINT,SIG_IGN);    //Ignore signal
    signal(SIGCHLD,SIG_DFL);   //Use default handler
}
```

Come vediamo l'handler ha come parametro un intero, l'intero corrisponde al numero del segnale che viene catturato e questo ci permette di assegnare lo stesso handler a più segnali diversi.
Dunque un handler personalizzato deve essere una funzione di tipo *void* che accetta come argomento un $\int$ ¿ rappresentante il segnale catturato.

```
#include <signal.h> <stdio.h>                          //param.c

void myHandler(int sigNum){
    if(sigNum == SIGINT) printf("CTRL+C\n");
    else if(sigNum == SIGTSTP) printf("CTRL+Z\n");
}

signal(SIGINT,myHandler);
signal(SIGTSTP,myHandler);
```

Come abbiamo detto $signal()$ restituisce un riferimento all'handler che era precedentemente assegnato al segnale e può assumere valore:

- $NULL$: se l'handler precedente era l'handler di default
- 1: se l'handler precedente era SIG_IGN
- $<address>$: l'handler precedente era $*(address)$

```c
#include <signal.h> <stdio.h>                                    //return.c
void myHandler(int sigNum){}
int main(){
    printf("DFL: %p\n",signal(SIGINT,SIG_IGN));
    printf("IGN: %p\n",signal(SIGINT,myHandler));
    printf("Custom: %p == %p\n",signal(SIGINT,SIG_DFL),myHandler);
}
```

Ecco le azioni di default dei segnali che abbiamo visto prima:

SIGXXX	description	default
SIGALRM	(alarm clock)	quit
SIGCHLD	(child terminated)	ignore
SIGCONT	(continue, if stopped)	ignore
SIGINT	(terminal interrupt, CTRL + C)	quit
SIGKILL	(kill process)	quit
SIGSYS	(bad argument to syscall)	quit with dump
SIGTERM	(software termination)	quit
SIGUSR1/2	(user signal 1/2)	quit
SIGSTOP	(stopped)	quit
SIGTSTP	(terminal stop, CTRL + Z)	quit

Finora abbiamo visto come gestire e definire i comportamenti al rilevamento di un determinato segnale.
Per **inviare un segnale** bisogna usare la funzione $kill()$:

$$\int kill(pid_t\ \ pid, \int sig);$$

Da terminale possiamo inviare un segnale usando

$$kill\ \ -<signo>\ \ <pid_t>$$

A seconda dell'argomento pid si decide a quali processi inviare il segnale:

- $pid > 0$: il segnale viene inviato al processo con $PID = pid$
- $pid = 0$: il segnale viene inviato ad ogni processo dello stesso gruppo del processo che lo invia
- $pid = -1$: il segnale viene inviato ad ogni processo possibile (ovvero con stesso UID/RUID). Se si ha privilegi root, il segnale può essere inviato a tutti i processi.
- $pid < -1$: il segnale viene inviato ad ogni processo del gruppo $|pid|$

Il valore di ritorno di $kill()$ è molto importante, perché viene restituito 0 se il segnale viene inviato, -1 in caso di errore.
A differenza della gestione dei segnali, in cui non possiamo modificare alcuni segnali, per quanto ne riguarda l'invio non ci sono limitazioni, infatti ogni tipo di segnale può essere inviato e

non deve essere necessariamente un segnale corrispondente ad un evento effettivamente avvenuto.

Abbiamo già visto che *kill* è anche un programma in bash che accetta come primo argomento il tipo di segnale (con il comando *kill -l* si ottiene la lista dei segnali) e come secondo argomento il PID del processo su cui inviarlo.

```c
#include <signal.h> <stdio.h> <stdlib.h> <unistd.h>  //bash.c
void myHandler(int sigNum){
    printf("[%d]ALARM!\n",getpid());
    exit(0);
}
int main(){
    signal(SIGALRM,myHandler);
    printf("I am %d\n",getpid());
    while(1);
}
```

```
$ gcc bash.c -o bash.out
$ ./bash.out
# On new window/terminal
$ kill -14 <PID>
```

Per generare un segnale *SIGALRM* sul processo corrente dopo un lasso di tempo specificato in secondi si può usare la funzione *alarm()*:

$$unsigned \int alarm(unsigned \int seconds);$$

e restituisce il numero di secondi rimanenti dal precedente allarme programmato (e in questo caso viene "sovrascritto"), o 0 se non era presente alcun allarme programmato in precedenza. Questa funzione va in conflitto con la funzione *sleep()*, infatti l'uso di *sleep()* potrebbe annullare il timer impostato da *alarm()* quindi non bisogna mai usarle assieme.
Ecco un esempio di utilizzo di *alarm()*:

```c
#include <signal.h> <stdio.h> <stdlib.h> <unistd.h>  //alarm.c
short cnt = 0;
void myHandler(int sigNum){printf("ALARM!\n"); cnt++;}
int main(){
    signal(SIGALRM,myHandler);
    alarm(0); //Clear any pending alarm
    alarm(5); //Set alarm in 5 seconds
    //Set new alarm (cancelling previous one)
    printf("Seconds remaining to previous alarm %d\n",alarm(2));
    while(cnt<1);
}
```

Per mettere in pausa un processo, ovvero in attesa di qualche evento, si usa la funzione

$$\int pause();$$

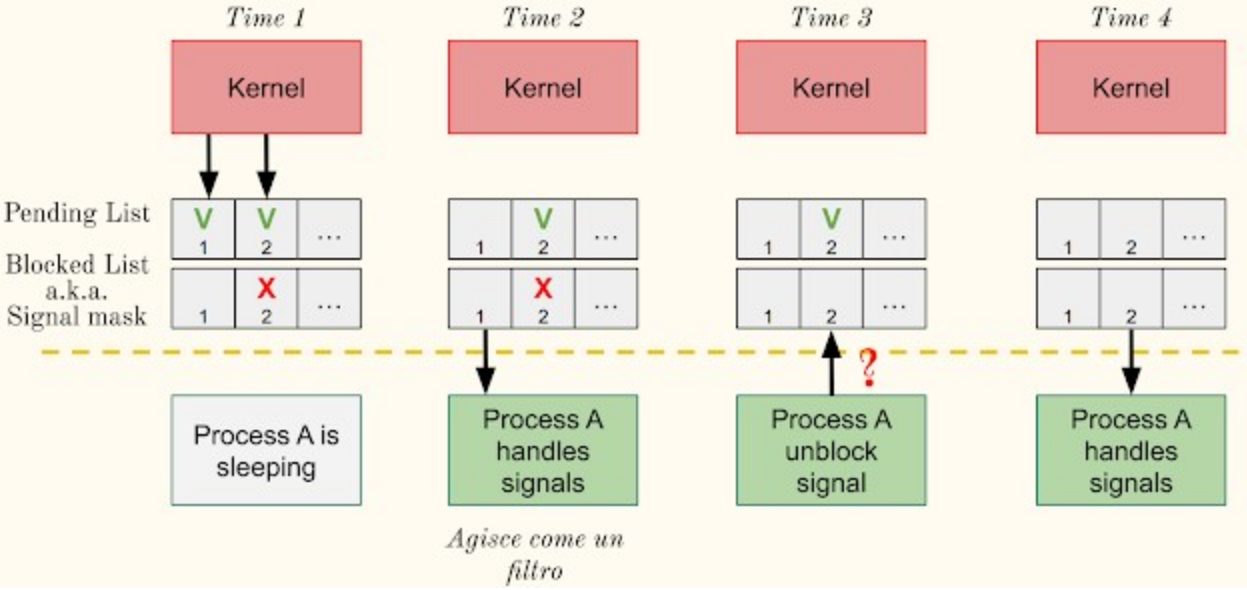
```c
#include <signal.h> <unistd.h> <stdio.h>            //pause.c

void myHandler(int sigNum){
    printf("Continue!\n");
}
int main(){
    signal(SIGCONT,myHandler);
    signal(SIGUSR1,myHandler);
    pause();
}
```

```
$ gcc pause.c -o pause.out
$ ./pause.out
# On new window/terminal
$ kill -18/-10 <PID>
```

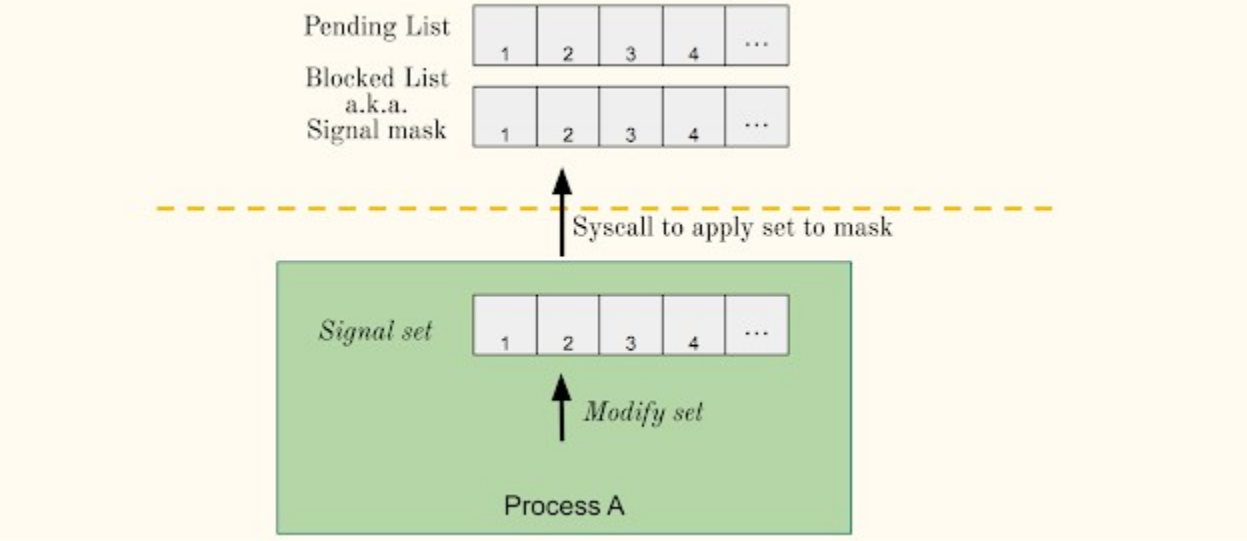
La funzione *pause*() mette in pausa l'esecuzione del programma e attende la ricezione di un qualsiasi segnale. Quando un segnale viene ricevuto, l'esecuzione del programma viene ripresa e la funzione *pause*() restituisce -1.

Vediamo ora come **bloccare i segnali**. La lista “blocked signals” contiene la lista dei segnali che vengono volutamente non gestiti. A differenza dei segnali ignorati che non verranno mai gestiti (vengono tolti dalla lista “pending signals”), i segnali bloccati sono solo **temporaneamente non gestiti**, infatti restano nella lista “pending signals” fino a che non vengono gestiti oppure fino a quando il loro handler non è tramutato in *ignore*. Infatti nel momento in cui sblocchiamo un segnale, potrebbe essere che vengano gestiti dei segnali che sono arrivati prima dello sblocco.
Ecco la visione concettuale dei concetti di cui abbiamo appena parlato:



Per modificare la lista “blocked signals”, un processo deve modificare la propria **signal mask**, ovvero una struttura dati mantenuta nel kernel che può essere alterata con la funzione *sigprocmask*().

Per modificare la signal mask, dobbiamo prima alterare localmente una **signal set**, la quale non applica veramente le modifiche. Per applicare le modifiche dobbiamo passare la signal set alla funzione *sigprocmask*().



Questo passaggio intermedio viene fatto perché la signal mask viene memorizzata come struttura dati ottimizzata che non può essere modificata direttamente.
Il signal set è una struttura dati di tipo *sigset_t* che contiene un elenco di segnali. Questo insieme può essere modificato con funzioni dedicate e poi può essere utilizzato, come abbiamo già detto, per modificare la maschera di segnale stessa.

```
int sigemptyset(sigset_t *set); Svuota
int sigfillset(sigset_t *set);  Riempie
int sigaddset(sigset_t *set, int signo); Aggiunge singolo
int sigdelset(sigset_t *set, int signo); Rimuove singolo
int sigismember(const sigset_t *set, int signo); Interpella
```

Per applicare le modifiche quindi si utilizza la funzione *sigprocmask*():

$$\int sigprocmask(\int how, const\ sigset_t\ *restrict\ set, sigset_t\ *restrict\ oldset);$$

dove *how* è la modalità di applicazione, *set* è il nuovo *sigset_t* e *oldset* è quello vecchio.
Questa funzione applica la maschera *set* con la modalità *how*.
Le modalità, ovvero i valori che può assumere *how* sono:

- *SIG_BLOCK*: i segnali in *set* sono aggiunti alla vecchia maschera
- *SIG_UNBLOCK*: i segnali in *set* sono rimossi dalla vecchia maschera
- *SIG_SETMASK*: *set* diventa la nuova maschera

Se *oldset* non è nullo, in esso verrà salvata la vecchia maschera (anche se *set* è nullo).

```c
#include <signal.h>

int main(){
    sigset_t mod,old;
    sigfillset(&mod); // Add all signals to the blocked list
    sigemptyset(&mod); // Remove all signals from blocked list
    sigaddset(&mod,SIGALRM); // Add SIGALRM to blocked list
    sigismember(&mod,SIGALRM); // is SIGALRM in blocked list?
    sigdelset(&mod,SIGALRM); // Remove SIGALRM from blocked list

    // Update the current mask with the signals in 'mod'
    sigprocmask(SIG_BLOCK,&mod,&old);
}
```

Per verificare se un segnale è presente nella lista “pending signals” esiste la funzione $sigpending()$:

$$\int sigpending(sigset_t *set);$$

```c
//sigpending.c
#include <signal.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

sigset_t mod,pen;
void handler(int signo){
    printf("SIGUSR1 received\n");
    sigpending(&pen);
    if(!sigismember(&pen,SIGUSR1))
      printf("SIGUSR1 not pending\n");
    exit(0);
}
```

```c
int main(){
    signal(SIGUSR1,handler);
    sigemptyset(&mod);
    sigaddset(&mod,SIGUSR1);
    sigprocmask(SIG_BLOCK,&mod,NULL);
    kill(getpid(),SIGUSR1);
    // sent but it's blocked...
    sigpending(&pen);
    if(sigismember(&pen,SIGUSR1))
        printf("SIGUSR1 pending\n");
    sigprocmask(SIG_UNBLOCK,&mod,NULL);
    while(1);
}
```

Vediamo ora la funzione $sigaction()$:

$\int sigaction(\int signum, const\ struct\ sigaction\ *restrict\ act, struct\ sigaction\ *restrict\ oldact);$

questa system call è molto simile a $signal()$, però notiamo subito che è molto più complessa infatti ci permette di specificare molte più cose.
Infatti non passiamo solamente la funzione handler ma passiamo una struttura:

```
struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask; //Signals blocked during handler
    int sa_flags; //modify behaviour of signal
    void (*sa_restorer)(void); //Deprecated, not POSIX
};
```

Inoltre otteniamo anche tramite il terzo parametro la vecchia struttura che era associata a quel segnale.

Analizziamo la struttura *sigaction*:

- *sa_handler* è un handler come gli altri che abbiamo visto
- *sa_sigaction* è un handler in cui possiamo specificare ulteriori cose, come le informazioni sul segnale (ad esempio chi l'ha inviato: *si_pid*) e sul contesto
- *sa_mask* è una sigset che contiene tutti i segnali che vogliamo che vengano bloccati durante l'esecuzione del nostro handler (per far sì che non lo interrompano)
- *sa_flags* è un insieme di flags (ORed) che vanno a specificare la maniera in cui andiamo a gestire questo segnale, il suo comportamento (i flags della variabile *sa_flags* dovrebbero essere sempre inizializzati almeno a 0!)

Se utilizziamo *sa_sigaction* dobbiamo settare il flag *SA_SIGINFO*. Con il flag *SA_RESETHAND* facciamo in modo che il nostro handler personalizzato venga utilizzato una volta sola, e le successive chiamate ritornino sul default handler.

```
#include <signal.h> <unistd.h> <stdlib.h> <stdio.h>   //sigaction.c
void handler(int signo){
    printf("signal received\n");
}
int main(){
    struct sigaction sa; //Define sigaction struct
    sa.sa_handler = handler; //Assign handler to struct field
    sigemptyset(&sa.sa_mask); //Define an empty mask
    sigaction(SIGUSR1,&sa,NULL);
    kill(getpid(),SIGUSR1);
}
```

L'uso di *sigaction()* però è meno portabile, nonostante permetta più espressione.

La funzione *sigqueue()* ci permette di inviare anche un payload assieme al segnale:

$$\int sigqueue(pid_t\ \ pid, \int sig, const \cup sigval\ \ value);$$

Viene inviato un segnale di tipo *sig* al processo identificato da *pid*, accompagnato da un payload *value*. Il payload è rappresentato dalla seguente union:

```
union sigval {
    int   sival_int;
    void *sival_ptr;
};
```

Questa struttura può contenere un puntatore qualsiasi oppure un intero, però se passiamo un puntatore ad un altro processo, esso non potrà farci nulla di concreto in quanto le locazioni di memoria puntate da un processo sono senza significato per gli altri processi.
Il *value* verrà ricevuto solo ed esclusivamente se il processo destinatario utilizza sigaction con *SA _ SIGINFO*.

Esempio invio

```
#include <signal.h> <unistd.h> <stdlib.h> <stdio.h>  //sigqueueS.c

int main(int argc, char ** argv){
    union sigval value;
    value.sival_int = atoi(argv[2]);
    sigqueue(atoi(argv[1],SIGUSR1,value);
    while(1);
}
```

Esempio ricezione

```
#include <signal.h> <unistd.h> <stdlib.h> <stdio.h>  //sigqueueR.c
void handler(int signo, siginfo_t * info, void * empty){
    printf("Received integer\n",info->si_value.sival_int);
}
int main(){
    struct sigaction sa;
    sa.sa_sigaction = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags |= SA_SIGINFO; // Use sa_sigaction
    sigaction(SIGUSR1,&sa,NULL);
    while(1);
}
```

In conclusione i segnali sono uno strumento di comunicazione tra processi molto semplice ma efficace: essendo un metodo molto “antico” non è particolarmente flessibile (essendo nato quando le risorse erano più limitate dei tempi attuali), ma nella sua forma base è comodo e multipiattaforma. L’uso di “signal” è standard, ma alcuni comportamenti possono essere indefiniti, mentre “sigaction” è più affidabile ma meno portabile tra sistemi operativi.

Un'importante differenza è che $signal()$ associa l'handler ad una sola occorrenza del segnale, dunque dobbiamo riassociarlo all'handler ad ogni utilizzo.
Invece con $sigaction()$ possiamo con i flag fare in modo che non si resetti l'handler una volta gestito il segnale.

Errori in C

Durante l'esecuzione di un programma si possono riscontrare diversi tipi di errore:

- System calls che falliscono
- Divisioni per zero
- Problemi di memoria
- Etc

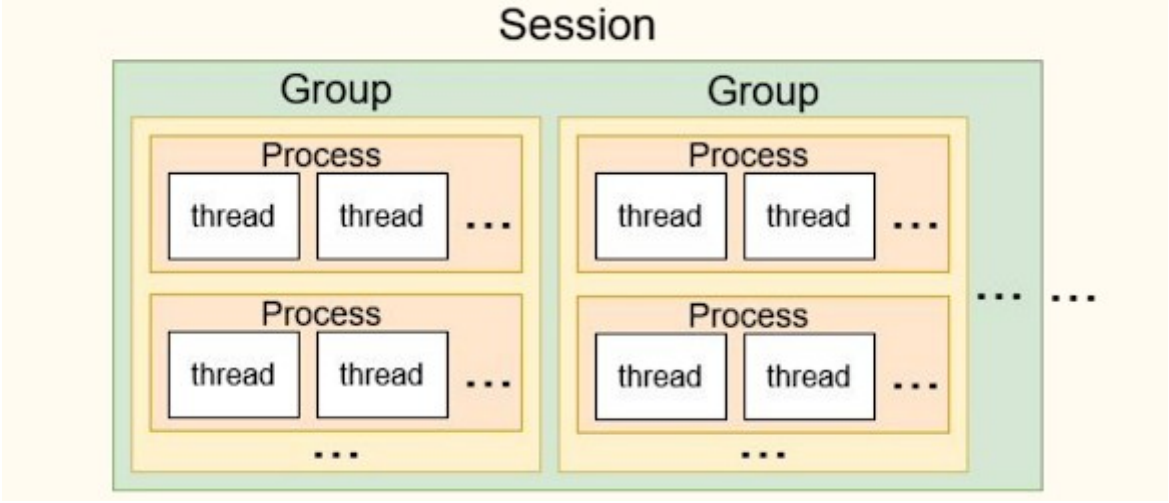
Alcuni di questi errori non sono fatali (ovvero non terminano immediatamente l'esecuzione del programma).
Ad esempio una system call che fallisce è un errore non fatale, e questo tipo di errori possono essere "indagati" attraverso la variabile intera *errno*, la quale conterrà il codice dell'ultimo errore generato dal sistema.
Per poterla rendere effettiva e utilizzabile, la variabile deve essere dichiarata come *extern*, in questo modo il processo la riempirà automaticamente ogni volta che viene riscontrato un errore.

Visto che *errno* è un semplice numero intero che identifica il tipo di errore, possiamo utilizzare la funzione $char\ *strerror(\int\ errnum)$ per convertire l'intero in una stringa comprensibile che descrive in lingua inglese il motivo dell'errore.

Un'altra funzione molto utile per quanto riguarda gli errori è $void\ \ perror(const\ \ char\ *str)$ la quale stampa direttamente su *stderr* la stringa che descrive l'errore, preceduta da *str* e :
In sintesi è equivalente ad eseguire $fprintf(stderr,\ "str:\ \%s¿",\ strerror(errno));$

Process groups

All'interno di Unix i processi vengono raggruppati secondi vari criteri, dando vita a sessioni, gruppi e threads.



I **process groups** vengono utilizzati perché consentono una migliore gestione dei segnali e della comunicazione tra i processi.
Infatti come abbiamo visto, un processo può:

- Aspettare che tutti i processi figli appartenenti ad un determinato gruppo terminino
- Mandare un segnale a tutti i processi appartenenti ad un determinato gruppo

```
waitpid(-33,NULL,0); // Wait for a children in group 33 (|-33|)
kill(-45,SIGTERM); // Send SIGTERM to all children in group 45
```

Mentre, generalmente, una **sessione** è collegata ad un terminale, i processi vengono raggruppati nel seguente modo:

- In bash, processi concatenati tramite pipes appartengono allo stesso gruppo
  $cat\ \ /tmp/ciao.txt\ \mid\ wc\ \ -l\ \mid\ grep\ \ `2'$
- Alla loro creazione, i figli di un processo ereditano il gruppo del padre
- Inizialmente, tutti i processi appartengono al gruppo di 'init', ed ogni processo può cambiare il suo gruppo in qualunque momento.

Anche i gruppi dei processi hanno un identificativo, il $PGID$, il quale corrisponde al PID del process group leader, ovvero il processo che ha creato quel gruppo (il primo elemento del gruppo). Il process group leader può successivamente cambiare gruppo senza problemi.
Infatti, quando creiamo un nuovo processo da bash, esso apparterrà al gruppo della bash.
Un gruppo che non contiene nessun processo non può esistere e dunque viene eliminato.

Le system calls per la gestione dei gruppi sono le seguenti:

- $\int\ \ setpgid(pid_t\ \ pid,\ pid_t\ \ pgid);$
  Per settare il gruppo del processo pid, mettendolo nel gruppo $pgid$
- $pid_t\ \ getpgid(pid_t\ \ pid);$
  Per ottenere il $pgid$ del processo pid

Possiamo usare il valore 0 per semplificarci la vita:

- $setpgid(0,\ x)$: Impostiamo il $pgid$ del processo corrente a x

- $setpgid(x, 0)$: Impostiamo il $pgid$ di x allo stesso valore di x
- $getpgid(0)$: Otteniamo il $pgid$ del processo corrente

Non è possibile assegnare un processo ad un gruppo non esistente, l'unico modo per creare un nuovo gruppo è utilizzando il $pgid$ equivalente al pid del processo (ad esempio $setpgid(0, 0)$), e in questo modo il processo diventerà il group leader di quel gruppo.

```c
#include <stdio.h> <unistd.h> <sys/wait.h>              //setpgid.c
int main(void){
    int isChild = !fork(); //new child
    printf("PID %d PPID: %d GID %d\n",getpid(),getppid(),getpgid(0));
    if(isChild){
        isChild = !fork(); //new child
        if(!isChild) setpgid(0,0); // Become group leader
        sleep(1);
        fork(); //new child
        printf("PID %d PPID: %d GID %d\n",getpid(),getppid(),getpgid(0));
    }; while(wait(NULL)>0);
}
```

Nel caso in cui al posto di $pgid$ venga specificato un gruppo che non esiste (e di cui il valore non corrisponde al pid del processo), non viene eseguita nessuna azione ma viene solamente aggiornato il valore di $errno$.

In conclusione l'organizzazione dei processi in gruppi consente di organizzare meglio le comunicazione e di coordinare le operazioni avendo in particolare la possibilità di inviare dei segnali ai gruppi nel loro complesso.

IPC: pipes and fifos

Il piping, come abbiamo già visto, connette l'output (*stdout* e *stderr*) di un comando all'input (*stdin*) di un altro comando, consentendo dunque la comunicazione tra i due.
Ecco degli esempi:

```
ls . | sort -R      #stout -> stdin
ls nonExistingDir |& wc       #stdout e stderr -> stdin
cat /etc/passwd | wc | less   #out -> in, out-> in
```

Nel piping, i processi sono eseguiti tutti in **concorrenza** (nello stesso momento) utilizzando un buffer:

- Se il buffer è pieno, lo scrittore (processo a sinistra) si sospende fino ad avere spazio libero
- Se il buffer è vuoto il lettore (processo a destra) si sospende fino ad avere i dati

Le pipe dunque sono un meccanismo bloccante per i processi.
Il piping è la comunicazione più basilare tra processi che esiste su linux.

Ecco un esempio in cui viene usato il piping:

Esempio

```
// output.out
#include <stdio.h>
#include <unistd.h>
int main(){
  for (int i = 0; i<3; i++) {
    sleep(2);
    fprintf(stdout,
    "Written in buffer");
    fflush(stdout);
  };
};
```

```
// input.out
#include <stdio.h>
#include <unistd.h>
int main() {
  char msg[50]; int n=3;
  while((n--)>0){
    int c = read(0,msg,49);
    if (c>0) {
        msg[c]=0;
        fprintf(stdout,
        "Read: '%s' (%d)\n",msg,c);
    };
  };
};
```

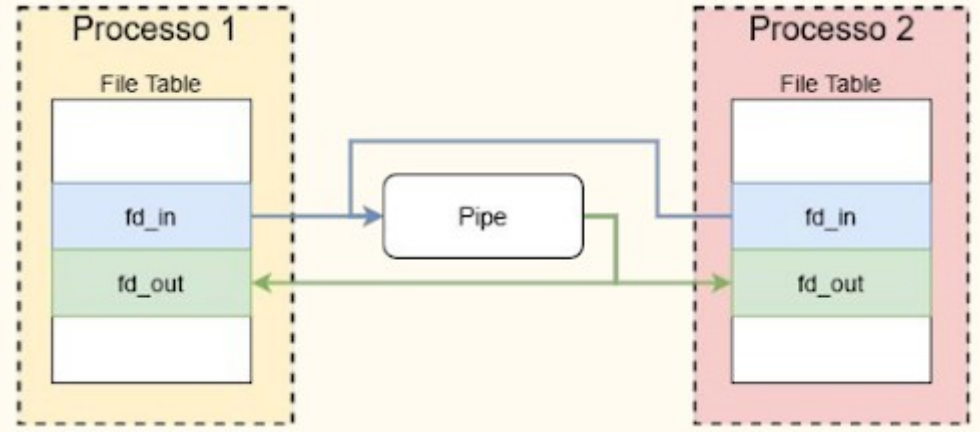
```
$ ./output.out | ./input.out
```

Pipe anonime

Le **pipe anonime**, come quelle usate su shell, 'uniscono' due processi aventi un **antenato comune** (oppure tra padre-figlio). Il collegamento è **unidirezionale** (in realtà può essere bidirezionale solo che non saremmo in grado di distinguere chi ha scritto cosa sulla pipe) ed avviene utilizzando un buffer di dimensione finita.
Per interagire con il buffer (la pipe) si usano due file descriptors:

- uno per il lato in scrittura
- uno per il lato in lettura

Visto che i processi figli ereditano i file descriptors, questo consente la comunicazione tra i processi.



La system call per **creare una pipe** anonima è $pipe()$:

$$\int\ pipe(\int\ pipefd[2]);$$

a cui passiamo un array di due interi:

- Nella prima posizione, $pipefd[0]$, viene inserito il file descriptor per accedere alla pipe in lettura
- Nella seconda posizione, $pipefd[1]$, viene inserito il file descriptor per accedere alla pipe in scrittura

Se viene riscontrato qualche errore nella creazione della pipe, la system call ritorna -1, altrimenti viene ritornato 0.

```c
#include <stdio.h>                                          //pipe.c
#include <unistd.h>
int main(){
    int fd[2], cnt = 0;
    while(pipe(fd) == 0){ //Create unnamed pipe using 2 file descriptors
        cnt++;
        printf("%d,%d,",fd[0],fd[1]);
    }
    printf("\n Opened %d pipes, then error\n",cnt);
    int op = open("/tmp/tmp.txt",O_CREAT|O_RDWR,S_IRUSR|S_IWUSR);
    printf("File opened with fd %d\n",op);
}
```

Per eseguire la **lettura dalla pipe** si usa la funzione, che abbiamo già visto per i file descriptor, $read()$:

$$\int read(\int fd[0], char * data, \int num)$$

e vengono restituiti valori differenti a seconda della situazione:

- Successo: viene restituito il numero di bytes effettivamente letti (sono disponibili meno bytes di quelli richiesti num, vengono letti solo quelli presenti)
- Se il buffer è vuoto e tutti i processi hanno chiuso il lato di scrittura viene tornato 0
- Se il buffer è vuoto ma ci sono ancora processi col lato di scrittura aperto il processo si sospende e rimane in attesa fino alla disponibilità dei dati o fino a che tutti i processi chiudono il lato di scrittura.

```c
#include <stdio.h>                                          //readPipe.c
#include <unistd.h>
int main(void){
    int fd[2]; char buf[50];
    int esito = pipe(fd); //Create unnamed pipe
    if(esito == 0){
        write(fd[1],"writing",8); // Writes to pipe
        int r = read(fd[0],&buf,50); //Read from pipe
        printf("Last read %d. Received: '%s'\n",r,buf);
        // close(fd[1]); // hangs when commented
        r = read(fd[0],&buf,50); //Read from pipe
        printf("Last read %d. Received: '%s'\n",r,buf);
    }
}
```

Per eseguire la **scrittura sulla pipe** si usa la funzione, che abbiamo già visto per i file descriptor, $write()$:

$$\int write(\int fd[1], char * data, \int num)$$

e viene restituito il numero di byte scritti.
Tuttavia, se il lato in lettura è stato chiuso viene inviato un segnale $SIGPIPE$ allo scrittore (default handler: quit) e la chiamata restituisce -1 (dobbiamo gestire questo segnale).
L'errore tuttavia viene trovato solamente nel momento in cui viene restituito -1 dalla system call, dunque nell'handler del segnale non visualizzeremo l'errore.

Se vengono scritti meno bytes di quelli che ci possono stare ($PIPE_BUF$) la scrittura è "atomica" (tutto assieme), in caso contrario non c'è garanzia di atomicità e la scrittura sarà bloccata (in attesa che il buffer venga svuotato) o fallirà se il flag $O_NONBLOCK$ viene usato.
Per **modificare le proprietà** di una pipe, possiamo usare la system call $fcntl()$:

$$\int\ fcntl(\int\ fd,\ F_SETFL,\ O_NONBLOCK);$$

la quale manipola i file descriptors (possiamo per l'appunto rendere non bloccanti i file descriptor sia in lettura che in scrittura).

```c
#include <unistd.h> <stdio.h> <signal.h> <errno.h> <stdlib.h> //writePipe.c
void handler(int signo){
    printf("SIGPIPE received\n");   perror("Error in handler");
}
int main(void){
    signal(SIGPIPE,handler);
    int fd[2]; char buf[50];
    int esito = pipe(fd); //Create unnamed pipe
    close(fd[0]); //Close read side
    printf("Attempting write\n");
    int numOfWritten = write(fd[1],"writing",8);
    printf("I've written %d bytes\n",numOfWritten);
    perror("Error after write");
}
```

Un tipico esempio per eseguire una **comunicazione unidirezionale** con una pipe anonima tra un processo scrittore $P1$ ed un processo lettore $P2$ è il seguente:

- $P1$ crea una $pipe()$
- $P1$ esegue un $fork()$ e crea $P2$
- $P1$ chiude il lato lettura: $close(fd[0])$
- $P2$ chiude il lato scrittura: $close(fd[1])$
- $P1$ e $P2$ chiudono anche l'altro fd appena finiscono di comunicare

```c
#include <stdio.h> <unistd.h> <sys/wait.h>          //uni.c
int main(){
    int fd[2]; char buf[50];
    pipe(fd); //Create unnamed pipe
    int p2 = !fork();
    if(p2){
        close(fd[1]);
        int r = read(fd[0],&buf,50); //Read from pipe
        close(fd[0]); printf("Buf: '%s'\n",buf);
    }else{
        close(fd[0]);
        write(fd[1],"writing",8); // Write to pipe
        close(fd[1]);
    }
    while(wait(NULL)>0);
}
```

Un tipico esempio di **comunicazione bidirezionale** (dunque ci servono due pipe) tra un processo scrittore $P1$ ed un processo lettore $P2$ è il seguente:

- $P1$ crea due $pipe()$: $pipe1$ e $pipe2$
- $P1$ esegue un $fork()$ e crea $P2$
- $P1$ chiude il lato lettura di $pipe1$ ed il lato scrittura di $pipe2$
- $P2$ chiude il lato scrittura di $pipe1$ ed il lato lettura di $pipe2$
- $P1$ e $P2$ chiudono anche gli altri fd appena finiscono di comunicare

```c
#include <stdio.h> <unistd.h> <sys/wait.h>   #define READ 0 #define WRITE 1      //bi.c
int main(){
     int pipe1[2], pipe2[2]; char buf[50];
     pipe(pipe1); pipe(pipe2); //Create two unnamed pipe
     int p2 = !fork();
     if(p2){
          close(pipe1[WRITE]); close(pipe2[READ]);
          int r = read(pipe1[READ],&buf,50); //Read from pipe
          close(pipe1[READ]); printf("P2 received: '%s'\n",buf);
          write(pipe2[WRITE],"Msg from p2",12); // Writes to pipe
          close(pipe2[WRITE]);
     }else{
          close(pipe1[READ]); close(pipe2[1]);
          write(pipe1[WRITE],"Msg from p1",12); // Writes to pipe
          close(pipe1[WRITE]);
          int r = read(pipe2[READ],&buf,50); //Read from pipe
          close(pipe2[READ]); printf("P1 received: '%s'\n",buf);
     }
     while(wait(NULL)>0);
}
```

Ovviamente per gestire una comunicazione più complessa e articolata è necessario definire un "protocollo", ad esempio:

- Messaggi di lunghezza fissa (definita prima di inviare i messaggi)
- Marcatore di fine messaggio (per esempio con carattere $NULL$ o newline)

Più in generale occorre definire la sequenza di messaggi attesi.

Ecco un esempio in cui vengono usate le pipe anonime per redirigere lo *stdout* di un comando sullo *stdin* di un altro comando:

```c
#include <stdio.h> <unistd.h> #define READ 0 #define WRITE 1    //redirect.c
int main (int argc, char *argv[]) {
    int fd[2];
    pipe(fd); // Create an unnamed pipe
    if (fork() != 0) { // Parent, writer
        close(fd[READ]); // Close unused end
        dup2(fd[WRITE], 1); // Duplicate used end to stdout
        close(fd[WRITE]); // Close original used end
        execlp(argv[1], argv[1], NULL); // Execute writer program
        perror("error"); // Should never execute
    } else { // Child, reader
        close(fd[WRITE]); // Close unused end
        dup2(fd[READ], 0); // Duplicate used end to stdin
        close(fd[READ]); // Close original used end
        execlp(argv[2], argv[2], NULL); // Execute reader program
        perror("error"); // Should never execute
    }
}
```

Usare *./redirect ls wc* è equivalente ad usare *ls | wc*

FIFO

Le **pipe con nome**, o **FIFO**, sono delle pipe che corrispondono a dei file speciali nel filesystem grazie ai quali i processi, senza vincoli di gerarchia, possono comunicare. Un processo può accedere ad una di queste pipe se ha i permessi sul file corrispondente ed è vincolato, ovviamente, dall'esistenza del file stesso.
Con le FIFO possono comunicare anche due processi che non possiedono antenati in comune, è necessario infatti solamente sapere il nome della FIFO (ed avere i permessi).

Le FIFO sono interpretate come dei file, perciò si possono usare le funzioni di scrittura/lettura dei file viste nelle scorse lezioni. Restano però delle pipe, con i loro vincoli e le loro capacità.
NB: le FIFO non sono dei file effettivi, quindi *lseek()* non funziona, il loro contenuto è sempre vuoto e non vi ci si può scrivere con un editor!
Infatti i file che rappresentano le FIFO sono dei semplici riferimenti ai buffer (presenti in memoria) della FIFO.
In sostanza, le FIFO forniscono un'interfaccia file-like per la comunicazione interprocesso, consentendo ai processi di scambiarsi dati in modo sequenziale attraverso un meccanismo di code, non essendo però dei veri file.

Normalmente **aprire una FIFO blocca il processo finchè anche l'altro lato non è stato aperto**. Le differenze tra pipe anonime e FIFO sono solo nella loro creazione e gestione.

La system call per **creare una FIFO** è *mkfifo()*:

$$\int mkfifo(const\ \ char\ \ *pathname,\ mode_t\ \ mode);$$

che crea una FIFO (ovvero un file speciale), solo se non esiste già, con il nome *pathname*. Il parametro *mode* può definire i privilegi del file nella solita maniera.

```c
#include <sys/stat.h><sys/types.h><unistd.h><fcntl.h><stdio.h>  //fifo.c
int main(void){
    char * fifoName = "/tmp/fifo1";
    mkfifo(fifoName,S_IRUSR|S_IWUSR); //Create pipe if it doesn't exist
    perror("Created?");
}
```

Per **aprire la pipe** usiamo *open*(), come con i file descriptor:

$$\int open(const\ \ char\ \ *pathname,\ \int flags\ [,\ mode_t\ \ mode]);$$

Ecco un esempio in cui vediamo bene che l'apertura della pipe è bloccante:

```c
#include <sys/stat.h><sys/types.h><unistd.h><fcntl.h><stdio.h>  //fifo.c
int main(void){
    char * fifoName = "/tmp/fifo1";
    mkfifo(fifoName,S_IRUSR|S_IWUSR); //Create pipe if it doesn't exist
    perror("Created?");
    if (fork() == 0){ //Child
        open(fifoName,O_RDONLY); //Open READ side of pipe...stuck!
        printf("Open read\n");
    }else{
        sleep(3);
        open(fifoName,O_WRONLY); //Open WRITE side of pipe
        printf("Open write\n");
    }
}
```

Ecco un esempio di due processi che scrivono e leggono in maniera alternata dalla pipe. Questo è possibile appunto per l'apertura bloccante.

Esempio comunicazione: writer (1/2)

```c
#include<sys/stat.h><sys/types.h><unistd.h><fcntl.h><stdio.h><string.h>

//fifoWriter.c
int main (int argc, char *argv[]) {
    int fd;   char * fifoName = "/tmp/fifo1";
    char str1[80], * str2 = "I'm a writer";
    mkfifo(fifoName,S_IRUSR|S_IWUSR); //Create pipe if it doesn't exist
    fd = open(fifoName, O_WRONLY); // Open FIFO for write only
    write(fd, str2, strlen(str2)+1); // write and close
    close(fd);
    fd = open(fifoName, O_RDONLY); // Open FIFO for Read only
    read(fd, str1, sizeof(str1)); // Read from FIFO
    printf("Reader is writing: %s\n", str1);
    close(fd);
}
```

Esempio comunicazione: reader (2/2)

```c
#include<sys/stat.h><sys/types.h><unistd.h><fcntl.h><stdio.h><string.h>

//fifoReader.c
int main (int argc, char *argv[]) {
    int fd; char * fifoName = "/tmp/fifo1";
    mkfifo(fifoName,S_IRUSR|S_IWUSR); //Create pipe if doesn't exist
    char str1[80], * str2 = "I'm a reader";
    fd = open(fifoName , O_RDONLY); // Open FIFO for read only
    read(fd, str1, 80); // read from FIFO and close it
    close(fd);
    printf("Writer is writing: %s\n", str1);
    fd = open(fifoName , O_WRONLY); // Open FIFO for write only
    write(fd, str2, strlen(str2)+1); // Write and close
    close(fd);
}
```

Al contrario delle pipe anonime, le **FIFO sono persistenti**, dunque restano nel sistema anche quando i processi terminano.

È possibile utilizzare le FIFO da terminale, leggendo e scrivendo dati tramite gli operandi di redirezione:

```
echo "message for pipe" > /path/nameOfPipe

cat /path/nameOfPipe
```

Per creare la FIFO direttamente da terminale si usa il comando *mkfifo*.

NB: non si possono scrivere dati usando editor di testo! Una volta consumati i dati, questi non sono più presenti sulla fifo.

Un riassunto sull'utilizzo delle pipe anonime e delle FIFO è nella seguente tabella:

	pipe	FIFO
Rappresentazione	Buffer	Buffer
Riferimento	anonimo	File
Accesso	2 File descriptors	1 File descriptor
Persistenza	Eliminata alla terminazione di tutti i processi	Esiste finchè esiste il file
Vincoli accesso	Antenato comune	Permessi sul file
Creazione	pipe()	mkfifo()
Max bytes per atomicità	PIPE_BUF = 4096 on Linux, minimo 512 Bytes POSIX	

In conclusione, PIPE e FIFO ("named pipes") sono sistemi di comunicazione tra processi ("parenti", tipicamente padre-figlio, nel primo caso e in generale nel secondo caso) che consentono scambi di informazioni (messaggi) e sincronizzazione (grazie al fatto di poter essere "bloccanti").

IPC: message queues

Una coda di messaggi, **message queue**, è una lista concatenata memorizzata all'interno del kernel ed identificata con un ID (un intero positivo univoco), chiamato **queue identifier**.
Questo ID viene condiviso tra i processi interessati, e viene generato attraverso una chiave univoca.
Una coda deve essere innanzitutto generata in maniera analoga ad una FIFO, impostando dei permessi. Ad una coda esistente si possono aggiungere o recuperare messaggi tipicamente in modalità "autosincrona" (bloccante), ovvero che la lettura attende la presenza di un messaggio, la scrittura attende che via sia spazio disponibile. Questi comportamenti possono però essere configurati tramite opportune flags.

Quindi per comunicare tramite le code non usiamo un riferimento ad un file, ma utilizziamo un id (che deve essere condiviso tra i processi che intendono comunicare).

Quando trattiamo message queues, vengono utilizzati due identificativi:

- **Queue Identifier**
  Che è l'id univoco della coda, generato dal kernel ed associato ad una specifica key.
  Questo ID è un intero positivo e viene usato per interagire con la coda.
  Questo identificatore viene utilizzato per riferirsi alla coda dei messaggi in tutte le successive operazioni di invio o ricezione dei messaggi.
  Il queue identifier non è univocamente associato ad una determinata coda, ma lo stesso valore può essere assegnato a più code in tempi diversi.
- **Key**
  Che consiste in un intero che identifica un insieme di risorse condivisibili nel kernel, come semafori, memoria condivisa e **code**. Questa chiave univoca deve essere nota a più processi, e viene usata per ottenere il queue identifier.
  La key funge da identificatore per una specifica coda dei messaggi. È possibile considerare la chiave come un'etichetta o un identificatore arbitrario associato alla coda dei messaggi.
  Se due processi diversi vogliono interagire con una coda, non hanno modo di sapere quale è il queue identifier da utilizzare per comunicare, dunque la key serve per far sì che due processi diversi che non si conoscono ottengano lo stesso queue identifier (visto che non lo conosciamo a priori ma lo sappiamo solo quando viene creata la coda).
  La key è dunque un intero positivo univoco che deve essere condiviso dai processi che vogliono condividere la coda.

In pratica, la chiave viene utilizzata per identificare la coda dei messaggi desiderata tra quelle presenti nel sistema, mentre l'identificatore della coda viene assegnato dal sistema operativo per identificare in modo univoco la coda dei messaggi nel contesto di un processo specifico.

Per la **creazione della coda** (e per ottenere il queue identifier) viene utilizzata la system call $msgget()$:

$$\int msgget(key_t\ key, \int msgflg)$$

a cui passiamo la key della coda e i flag che definiscono il comportamento della system call nei vari casi. Questa system call restituisce il queue identifier di una coda basandosi sulla key e sui flags:

- IPC_CREAT: crea una coda se non esiste già, altrimenti restituisce l'identificativo di quella già esistente
- IPC_EXCL: (da usare con il precedente) fallisce se coda già esistente (e restituisce -1)
- $0xxx$: permessi per accedere alla coda, analogo a quello che si può usare nel file system. In alternativa si possono usare S_IRUSR, etc.

```c
#include <sys/types.h> <sys/ipc.h> <sys/msg.h>              //msgget.c
key_t queueKey = 56; //Unique key
int queueId = msgget(queueKey, 0777 | O_CREAT | IPC_EXCL);
```

Visto che utilizzare un intero condiviso tra processi diversi come key non è molto comodo, perchè bisognerebbe trovare un intero per ogni message queue, solitamente si utilizza un meccanismo per ricevere un id univoco in maniera automatica.
Questo viene fatto tramite la funzione $ftok()$:

$$key_t \;\; ftok(const \;\; char \;\; *path, \int \;\; id)$$

che restituisce una chiave basandosi sul $path$(una cartella o un file), esistente ed accessibile nel file-system, e sull'id numerico. La chiave dovrebbe essere univoca e sempre la stessa per ogni coppia $< path, id >$ in ogni istante sullo stesso sistema.

Un metodo di utilizzo, per evitare possibili conflitti, potrebbe essere generare un path (es. un file) temporaneo univoco, usarlo, eventualmente rimuoverlo, ed usare l'id per rappresentare diverse "categorie" di code, a mo' di indice.

```
#include <sys/ipc.h>                                    //ftok.c
key_t queue1Key = ftok("/tmp/unique", 1);
key_t queue2Key = ftok("/tmp/unique", 2); ...
```

Ecco un esempio completo di creazione di una message queue:

```
<sys/types.h><sys/ipc.h> <sys/msg.h><stdio.h><fcntl.h>  //ipcCreation.c
void main(){
    remove("/tmp/unique"); //Remove file
    key_t queue1Key = ftok("/tmp/unique", 1); //Get unique key → fail
    creat("/tmp/unique", 0777); //Create file
    queue1Key = ftok("/tmp/unique", 1); //Get unique key → ok
    int queueId = msgget(queue1Key ,0777 | IPC_CREAT); //Create queue → ok
    queueId = msgget(queue1Key , 0777); //Get queue → ok
    msgctl(queue1Key,IPC_RMID,NULL);  //Remove non existing queue → fail
    msgctl(queueId,IPC_RMID,NULL); //Remove queue → ok
    queueId = msgget(queue1Key , 0777); //Get non existing queue → fail
    queueId = msgget(queue1Key , 0777 | IPC_CREAT); //Create queue → ok
    queueId = msgget(queue1Key , 0777 | IPC_CREAT); //Get queue → ok
    queueId = msgget(queue1Key , 0777 | IPC_CREAT | IPC_EXCL); /* Create
                                    already existing queue -> fail */
}
```

Con la funzione $msgctl()$ si può eliminare la coda (lo vedremo meglio dopo).

Un fattore molto importante da sapere è che **le queue sono persistenti**. Così come i files, le code sono delle strutture persistenti che continuano ad esistere, assieme ai messaggi in esse salvati, anche alla terminazione del processo che le ha create. L'eliminazione della coda deve essere infatti esplicita.
Infatti se impostiamo il flag IPC_EXCL, e usiamo più volte lo stesso programma senza rimuovere la coda, ci verrà dato un errore.

Analizziamo ora come avviene la comunicazione.
Ogni messaggio inserito nella coda ha:

- Un tipo, categoria, rappresentato da un intero >0
- Un payload, un insieme di dati (bytes) di lunghezza corretta

- Una grandezza (non negativa)

Ad esempio si può usare la seguente struttura msg_buffer:

```
struct msg_buffer{
    long mtype;
    char mtext[100];
} message;
```

Il payload del messaggio non deve essere necessariamente un campo testuale: può essere una qualsiasi struttura dati. Infatti, un messaggio può anche essere senza payload.

```
typedef struct book{
    char title[10];
    char description[100];
    unsigned short chapters;
} Book;

struct msg_buffer{
    long mtype;
    Book mtext;
} message_rcv;

struct msg_empty{
    long mtype;
} message_empty;
```

Al contrario delle FIFO, i messaggi in una coda possono essere recuperati anche sulla base del tipo e non solo del loro ordine "assoluto" di arrivo.

Per **inviare un messaggio** si usa la system call $msgsnd()$:

$$\int msgsnd(\int msqid, const\ void\ *msgp, ¿t\ msgsz, \int msgflg)$$

Viene aggiunta alla message queue identificata $msqid$ **una copia** del messaggio puntato da $msgp$, con un payload di dimensione $msgsz$. Notiamo che $msgsz$ è la dimensione del payload, e non del messaggio intero (che contiene anche il tipo), si può usare quindi $sizeof(msgp.mtext)$.

Il messaggio viene inserito immediatamente se c'è abbastanza spazio disponibile, altrimenti la chiamata si blocca fino a che abbastanza spazio diventa disponibile. Se $msgflg$ è IPC_NOWAIT allora la chiamata fallisce in assenza di spazio.
La chiamata ritorna 0 in caso di successo e -1 in caso di errore (ad esempio quando fallisce in assenza di spazio con il flag IPC_NOWAIT).

Per **ricevere un messaggio** si usa la system call $msgrcv()$

$$ssize_t\ msgrcv(\int msqid, void\ *msgp, ¿t\ msgsz, long\ msgtyp, \int msgflg)$$

Viene rimosso un messaggio dalla message queue identificata da *msqid* e viene salvato nel buffer *msgp*, la lunghezza massima del payload del messaggio da leggere viene specificata da *msgsz*. Nel caso in cui il payload abbia lunghezza maggiore di *msgsz* e *msgflg* è $MSG_NOERROR$, allora il payload viene troncato (viene persa la parte in eccesso), se $MSG_NOERROR$ non è specificato allora il payload non viene eliminato e la chiamata fallisce.

Se non sono presenti messaggi, la chiamata si blocca in loro attesa. Il flag IPC_NOWAIT fa fallire la syscall se non sono presenti messaggi.

A seconda del valore di *msgtyp* cambia il comportamento della system call:

- $msgtyp = 0$: viene recuperato il primo messaggio della coda (politica FIFO)
- $msgtyp > 0$: viene recuperato il primo messaggio di tipo *msgtyp* (oppure il primo messaggio diverso da *msgtyp* se MSG_EXCEPT è impostato come flag
- $msgtyp < 0$: viene recuperato il primo messaggio il cui tipo T è minore uguale di $|msgtyp|$, ovvero $T \leq |msgtyp|$

Un esempio di comunicazione all'interno dello stesso processo è il seguente:

```c
#include <sys/types.h><sys/ipc.h><sys/msg.h><string.h><stdio.h>  //ipc.c
struct msg_buffer{
    long mtype;
    char mtext[100];
} msgp, msgp2; //Two different message buffers
int main(void){
    msgp.mtype = 20;
    strcpy(msgp.mtext,"This is a message");
    key_t queue1Key = ftok("/tmp/unique", 1);
    int queueId = msgget(queue1Key , 0777 | IPC_CREAT | IPC_EXCL);
    int esito = msgsnd(queueId , &msgp, sizeof(msgp.mtext),0);
    esito = msgrcv(queueId , &msgp2, sizeof(msgp2.mtext),20,0);
    printf("Received %s\n",msgp2.mtext);
}
```

Un esempio con un payload più articolato è il seguente:

```c
#include <sys/types.h> <sys/ipc.h> <sys/msg.h>  <string.h>  //ipcBook.c

typedef struct book{
    char title[10];
    char description[200];
    short chapters;
} Book;

struct msg_buffer{
    long mtype;
    Book mtext;
} msgp_snd, msgp_rcv; //Two different message buffers
```

```c
int main(void){
    msgp_snd.mtype = 20;
    strcpy(msgp_snd.mtext.title,"Title");
    strcpy(msgp_snd.mtext.description,"This is a description");
    msgp_snd.mtext.chapters = 17;
    key_t queue1Key = ftok("/tmp/unique", 1);
    int queueId = msgget(queue1Key , 0777 | IPC_CREAT);
    int esito = msgsnd(queueId , &msgp_snd, sizeof(msgp_snd.mtext),0);
    esito = msgrcv(queueId , &msgp_rcv, sizeof(msgp_rcv.mtext),20,0);
    printf("Received: %s %s %d\n",msgp_rcv.mtext.title,
        msgp_rcv.mtext.description, msgp_rcv.mtext.chapters);
}
```

Ed infine un esempio in cui vengono usati i tipi:

```c
#include <sys/types.h> <sys/ipc.h> <sys/msg.h>  <string.h>  //ipcType.c

struct msg_buffer{
    long mtype;
    char mtext[100];
} msgp_snd,msgp_rcv; //Two different message buffers

int main(int argc, char ** argv){
    int to_fetch = atoi(argv[1]); //Input to decide which msg to get
    key_t queue1Key = ftok("/tmp/unique", 1);
    int queueId = msgget(queue1Key , 0777 | IPC_CREAT);
```

```
    msgp_snd.mtype = 20;
    strcpy(msgp_snd.mtext,"A message of type 20");
    int esito = msgsnd(queueId , &msgp_snd, sizeof(msgp.mtext),0);
    msgp_snd.mtype = 10; //Re-use the same message
    strcpy(msgp_snd.mtext,"Another message of type 10");
    esito = msgsnd(queueId , &msgp_snd, sizeof(msgp.mtext),0);
    esito = msgrcv(queueId , &msgp_rcv, sizeof(msgp_rcv.mtext),
        to_fetch,0);
    printf("Received: %s %s %d\n",msgp_rcv.mtext.title,
        msgp_rcv.mtext.description, msgp_rcv.mtext.chapters);
}
```

Per **modificare la coda** (modificare i permessi, ottenere informazioni, cancellare la coda), quindi, si utilizza la system call $msgctl()$:

$$\int msgctl(\int msqid, \int cmd, struct\ \ msqid_ds\ *buf)$$

e viene modificata la coda identificata da $msqid$, secondo i comandi cmd, e buf può essere usata sia come input per specificare i parametri, che come output riempendola con le informazioni sulla coda (ad esempio timestamp dell'ultima scrittura, dell'ultima lettura, numero messaggi nella coda, etc…).
I valori di cmd possono essere:

- IPC_STAT: recupera informazioni da kernel
- IPC_SET: imposta alcuni parametri a seconda di buf
- IPC_RMID: rimuove immediatamente la coda
- IPC_INFO: recupera informazioni generali sui limiti delle code nel sistema
- MSG_INFO: come IPC_INFO ma con informazioni differenti
- MSG_STAT: come IPC_STAT ma con informazioni differenti

Il parametro buf è di tipo $struct\ \ msqid_ds$, e questa struttura è così composta:

```
struct msqid_ds {
    struct ipc_perm msg_perm;   /* Ownership and permissions */
    time_t msg_stime;  /* Time of last msgsnd(2) */
    time_t msg_rtime;  /* Time of last msgrcv(2) */
    time_t msg_ctime; //Time of creation or last modification by msgctl
    unsigned long   msg_cbytes; /* # of bytes in queue */
    msgqnum_t       msg_qnum;   /* # of messages in queue */
    msglen_t        msg_qbytes; /* Maximum # of bytes in queue */
    pid_t           msg_lspid;  /* PID of last msgsnd(2) */
    pid_t           msg_lrpid;  /* PID of last msgrcv(2) */
};
```

A sua volta $struct\ \ msqid_ds$ contiene un'altra struttura $struct\ \ ipc_perm$, la quale contiene i permessi della coda:

```c
struct ipc_perm {
    key_t __key;    /* Key supplied to msgget(2) */
    uid_t uid;      /* Effective UID of owner */
    gid_t gid;      /* Effective GID of owner */
    uid_t cuid;     /* Effective UID of creator */
    gid_t cgid;     /* Effective GID of creator */
    unsigned short mode;    /* Permissions */
    unsigned short __seq;   /* Sequence number */
};
```

Quindi tramite buf possiamo specificare in che maniera modificare la coda (ovviamente dipende anche da cmd), e in base a certi cmd, i dati richiesti verranno inseriti proprio in buf.

Ecco un esempio in cui leggiamo i permessi di una coda e successivamente li modifichiamo:

```c
int main(void){
    struct msqid_ds mod;
    int esito = open("/tmp/unique",O_CREAT,0777);
    key_t queue1Key = ftok("/tmp/unique", 1);
    int queueId = msgget(queue1Key , IPC_CREAT | S_IRWXU  );
    msgctl(queueId,IPC_RMID,NULL);
    queueId = msgget(queue1Key , IPC_CREAT | S_IRWXU  );
    esito = msgctl(queueId,IPC_STAT,&mod); //Get info on queue
    printf("Current permission on queue: %d\n",mod.msg_perm.mode);
    mod.msg_perm.mode = 0000;
    esito = msgctl(queueId,IPC_SET,&mod);  //Modify queue
    printf("Current permission on queue: %d\n\n",mod.msg_perm.mode);
}
```

Ecco un esempio in cui prima di tutto creiamo, eliminiamo e ricreiamo la coda, per essere sicuri che non ci sia nulla al suo interno. Successivamente inviamo tre messaggi, ognuno con payload di 100, e poi modifichiamo il limite massimo della dimensione del buffer a 200 (questo va ad influire solo sulle grandezze dei payload). I messaggi già inseriti rimarranno nella coda, però non possiamo più inserire messaggi fino a che la somma dei payload è superiore a 200.
Poi creiamo un processo figlio per vedere che funziona proprio in questa maniera.

```c
#include <sys/types.h><sys/ipc.h><sys/msg.h><string.h><stdio.h><unistd.h><wait.h>
//ipc4.c
struct msg_buffer{
    long mtype;
    char mtext[100];
} msgpSND,msgpRCV;
void main(){
    struct msqid_ds mod;
    msgpSND.mtype = 1;
    strcpy(msgpSND.mtext,"This is a message from sender");
    key_t queue1Key = ftok("/tmp/unique", 1);
    int queueId = msgget(queue1Key , 0777 | IPC_CREAT);
    msgctl(queueId,IPC_RMID,NULL); //Remove queue if exists
    queueId = msgget(queue1Key , 0777 | IPC_CREAT); //Create queue
    msgsnd(queueId, &msgpSND, sizeof(msgpSND.mtext),0); //Send msg
    msgsnd(queueId, &msgpSND, sizeof(msgpSND.mtext),0); //Send msg
    msgsnd(queueId, &msgpSND, sizeof(msgpSND.mtext),0); //Send msg

    msgctl(queueId,IPC_STAT,&mod); //Modify queue
    printf("Msg in queue: %ld\nCurrent max bytes in queue: %ld\n\n",
            mod.msg_qnum, mod.msg_qbytes);

    mod.msg_qbytes = 200; //Change buf to modify queue bytes
    msgctl(queueId,IPC_SET,&mod);  //Apply modification

    printf("Msg in queue: %ld --> same number\nCurrent max bytes in
            queue: %ld\n\n",mod.msg_qnum, mod.msg_qbytes);

    if( fork() != 0 ){ //Parent keep on writing on the queue
        printf("[SND] Sending 4th message with a full queue...\n");
        msgsnd(queueId, &msgpSND, sizeof(msgpSND.mtext),0); //Send msg
        printf("[SND] msg sent\n");
```

```
        printf("[SND] Sending 5th message with IPC_NOWAIT\n");
        if(msgsnd(queueId, &msgpSND, sizeof(msgpSND.mtext),IPC_NOWAIT )
            == -1){ //Send msg
            perror("Queue is full --> Error");
        }
    } else { // Child keeps reading the queue every 3 seconds
        sleep(3); msgrcv(queueId, &msgpRCV, sizeof(msgpRCV.mtext),1,0);
        printf("[Reader] Received msg 1 with msg '%s'\n",msgpRCV.mtext);
        sleep(3); msgrcv(queueId, &msgpRCV, sizeof(msgpRCV.mtext),1,0);
        printf("[Reader] Received msg 2 with msg '%s'\n",msgpRCV.mtext);
        sleep(3); msgrcv(queueId, &msgpRCV, sizeof(msgpRCV.mtext),1,0);
        printf("[Reader] Received msg 3 with msg '%s'\n",msgpRCV.mtext);
        sleep(3); msgrcv(queueId, &msgpRCV, sizeof(msgpRCV.mtext),1,0);
        printf("[Reader] Received msg 4 with msg '%s'\n",msgpRCV.mtext);
        sleep(3);
        if(msgrcv(queueId, &msgpRCV, sizeof(msgpRCV.mtext),1,IPC_NOWAIT)
            == -1){
            perror("Queue is empty --> Error");
        }else{
            printf("[Reader] Received msg 5 with msg '%s'\n",
                    msgpRCV.mtext);
        }
    }
    while(wait(NULL)>0);
}
```

In conclusione, le code sono un comodo metodo di comunicazione per l'invio e la ricezione di informazioni anche "complesse" tra generici.

Gestire lavori multipli e la concorrenza: threads and mutexes

Threads

I thread sono singole sequenze di esecuzione all’interno di un processo, aventi alcune delle proprietà dei processi. I threads non sono completamente indipendenti tra loro e condividono il codice, i dati e le risorse del sistema assegnate al processo di appartenenza. Come ogni singolo processo, i threads hanno alcuni elementi indipendenti, come lo stack, il PC ed i registri del sistema.

I thread dunque sono strettamente dipendenti tra di loro visto che condividono alcune proprietà con gli altri thread appartenenti allo stesso processo (come ad esempio la file table dei file aperti).

La creazione di threads consente un parallelismo delle operazioni in maniera rapida e semplificata. Il context switch tra threads infatti è rapido, così come la loro creazione e terminazione. Inoltre, la comunicazione tra threads è molto veloce.
Questo permette di organizzare in maniera migliore il programma.

Per compilare un'applicazione facente uso di threads è necessario aggiungere il flag *-pthread*:

$$gcc\ -o\ program\ main.c\ -pthread$$

In C i thread corrispondono a delle funzioni eseguite in parallelo al codice principale. Ogni thread è identificato da un *ID* e può essere gestito come un processo figlio, con funzioni che attendono la sua terminazione.

Sebbene da un punto di vista l'esecuzione di diversi thread sia sempre parallela, a causa delle politiche di scheduling delle CPU, l'esecuzione può essere parallela solo se la CPU la supporta, ossia se dispone di più core.

Per creare un thread si utilizza la funzione *pthread_create()*:

$$\int\ pthread_create($$
$$pthread_t\ *restrict\ thread,$$
$$const\ \ pthread_attr_t\ *restrict\ \ attr,$$
$$void\ *(*start_routine)(void\ *),$$
$$void\ *restrict\ \ arg$$
$$);$$

Dove:

- *thread* è l'*ID* del thread che verrà creato
  Quando si crea un nuovo thread, è necessario fornire un puntatore *thread*, che verrà riempito con il nuovo *ID* generato (che viene generato automaticamente e non si può scegliere)
- *attr* sono gli attributi del thread
  attr consente di modificare il comportamento del thread
- *start_routine* è la funzione che eseguirà il thread
  start_routine serve a definire quale funzione deve essere eseguita dal thread
- *arg* sono i parametri che vengono passati alla funzione *start_routine*
  arg è un puntatore *void* che può essere utilizzato per passare qualsiasi argomento sia richiesto (questo tipo di puntatore può essere utilizzato per puntare a dati di qualsiasi tipo, ma non può essere dereferenziato direttamente)

Visto che la struttura o variabile passata come *arg* è un riferimento, la modifica all'interno del thread secondario viene rilevata anche all'interno del thread principale.

Ecco un esempio di creazione di un thread:

```c
#include <stdio.h> <pthread.h> <unistd.h> //threadCreate.c

void * my_fun(void * param){
    printf("This is a thread that received %d\n", *(int *)param);
    return (void *)3;
}
int main(void){
    pthread_t t_id;
    int arg=10;
    // We need to cast the augment to a void *. We are passing the
    address of the variable!
    pthread_create(&t_id, NULL, my_fun, (void *)&arg);
    printf("Executed thread with id %ld\n",t_id);
    sleep(3);
}
```

Il kernel interpreta i thread come **LWP** (Light-Weight Process):

- LWP: l'identificativo del thread dal punto di vista del kernel (è diverso da ID)
  Due thread dello stesso processo hanno lo stesso PID ma diversi LWP (LWP è incrementale)
- NLWP: il numero di thread appartenenti al processo del thread

Un nuovo **thread termina** in uno dei seguenti modi:

- Chiamando la funzione $void\ \ pthread_exit(void\ *\ retval);$ dal thread, specificando un puntatore da restituire (vedremo dopo come fare per ottenere questo valore di ritorno)
- Effettuando un $return;$ dalla funzione (principale) associata al thread, specificando un valore di ritorno
- Cancellando il thread da un altro thread
- Qualche thread chiama $exit()$, o il thread principale, ovvero quello che esegue $main()$, termina (eseguendo $return;$), terminando così tutti i threads

Per **cancellare un thread da un altro thread** si usa la funzione $pthread_cancel()$:

$$\int\ pthread_cancel(pthread_t\ \ thread);$$

dove bisogna passare l'identificativo del thread che si vuole cancellare.
Questa system call invia una **richiesta** di cancellazione al thread specificato, il quale reagirà (come e quando) a seconda di due suoi attributi: $cancel\ \ state$ e $cancel\ \ type$:

- Il $cancel\ \ state$ di un thread definisce se il thread deve terminare o meno quando una richiesta di cancellazione viene ricevuta
- Il $cancel\ \ type$ di un thread definisce come il thread deve terminare.

Sia lo *state* che il *type* possono essere definiti e modificati alla creazione del thread oppure durante la sua esecuzione solamente all'interno del thread stesso.

Per **cambiare il** $cancel\ \ state$ di un thread si usa al suo interno la funzione $pthread_setcancelstate()$:

$$\int\ \ pthread_setcancelstate(\int\ \ state,\ \int\ \ *oldstate);$$

la quale modifica il $cancel\ \ state$ del thread in esecuzione.
oldstate viene riempito con lo stato precedente, invece *state* può contenere una delle seguenti macro:

- $PTHREAD_CANCEL_ENABLE$: ogni richiesta di cancellazione viene gestita a seconda del *type* del thread (questa è la modalità default)
- $PTHREAD_CANCEL_DISABLE$: le richieste di cancellazione non verranno eliminate, ma aspetteranno fino a che il cancel state del thread non diventa $PTHREAD_CANCEL_ENABLE$

Per **cambiare il** $cancel\ \ type$ di un thread si usa al suo interno la system call $pthread_setcanceltype()$:

$$\int\ \ pthread_setcanceltype(\int\ \ type,\ \int\ \ *oldtype);$$

la quale modifica il $cancel\ \ type$ del thread in esecuzione.
oldtype viene riempito con il tipo precedente, invece *type* può contenere una delle seguenti macro:

- $PTHREAD_CANCEL_DEFERRED$: la terminazione aspetta l'esecuzione di un **cancellation point** (questa è la modalità default)
- $PTHREAD_CANCEL_ASYNCHRONOUS$: la terminazione avviene appena la richiesta viene ricevuta

I **Cancellation points** sono delle specifiche funzioni definite nella libreria $pthread.h$. Di solito sono system calls, infatti praticamente tutte le system calls che abbiamo visto sono dei cancellation points.

Ecco un esempio in cui disattiviamo le richieste di cancellazione:

```c
#include <stdio.h> <pthread.h> <unistd.h>                //thCancel.c
int i = 1;
void * my_fun(void * param){
    if(i--)
        pthread_setcancelstate(PTHREAD_CANCEL_DISABLE,NULL); //Change mode
    printf("Thread %ld started\n",*(pthread_t *)param); sleep(3);
    printf("Thread %ld finished\n",*(pthread_t *)param);
}
int main(void){
    pthread_t t_id1, t_id2;
    pthread_create(&t_id1, NULL, my_fun, (void *)&t_id1); sleep(1); //Create
    pthread_cancel(t_id1); //Cancel
    printf("Sent cancellation request for thread %ld\n",t_id1);
    pthread_create(&t_id2, NULL, my_fun, (void *)&t_id2); sleep(1); //Create
    pthread_cancel(t_id2); //Cancel
    printf("Sent cancellation request for thread %ld\n",t_id2);
    sleep(5); printf("Terminating program\n");
}
```

Ed ecco un esempio in cui usiamo i cancellation points (in questo caso la system call $open()$):

```c
#include <stdio.h> <pthread.h> <unistd.h>  <string.h>            //thCancel2.c
int tmp = 0;
void * my_fun(void * param){
    pthread_setcanceltype(*(int *)param,NULL); // Change type
    for (long unsigned i = 0; i < (0x9FFF0000); i++); //just wait
    tmp++;
    open("/tmp/tmp",O_RDONLY);  //Cancellation point!
}
int main(int argc, char ** argv){ //call program with 'async' or 'defer'
    pthread_t t_id1; int arg;
    if(!strcmp(argv[1],"async")) arg = PTHREAD_CANCEL_ASYNCHRONOUS;
    else if(!strcmp(argv[1],"defer")) arg = PTHREAD_CANCEL_DEFERRED;
    pthread_create(&t_id1, NULL, my_fun, (void *)&arg); sleep(1); //Create
    pthread_cancel(t_id1); sleep(5); //Cancel
    printf("Tmp %d\n",tmp);
}
```

Per **aspettare l'esecuzione di un thread** "creato" si usa la system call $pthread_join()$:

$$\int \; pthread_join(\, pthread_t \;\; thread\,,\;\; void \;\; ** \;\; retval\,);$$

Questa funzione ritorna quando il thread con ID uguale a *thread* termina, o subito se il thread è già terminato.

Se il valore di ritorno del thread non è nullo (parametro di $pthread_exit()$ o di *return*), esso viene salvato nella variabile puntata da *retval*. Se il thread era stato cancellato, *retval* è riempito con la macro $PTHREAD_CANCELED$.

Un thread può essere aspettato solamente se è **joinable**, e in questo caso può essere aspettato da al massimo 1 thread.
I thread sono creati per impostazione predefinita nello stato $joinable$, che consente a un altro thread di attendere la loro terminazione tramite il comando $pthread_join()$. I thread joinable non rilasciano le loro risorse alla terminazione, ma quando un thread li aspetta (salvando lo stato di uscita, come i sottoprocessi), o alla terminazione del processo stesso.
Al contrario, i thread **detached** rilasciano le loro risorse immediatamente al termine, ma non permettono ad altri processi di aspettarli (un thread cambiato in detached non può più diventare joinable durante la sua esecuzione, mentre è sempre possibile il contrario)

Per mettere un thread in $detach$ mode si usa la system call $pthread_detach()$:

$$\int\ pthread_detach(pthread_t\ \ thread);$$

Questo comando può essere eseguito da un thread qualunque e cambia il detach state di thread da joinable a detached (ricordiamo che una volta cambiato lo stato non si può invertire)

Ecco due esempi in cui aspettiamo dei thread joinable:

```c
#include <stdio.h> <pthread.h> <unistd.h>                //thJoin.c
void * my_fun(void * param){
    printf("Thread %ld started\n",*(pthread_t *)param); sleep(3);
    char * str = "Returned string";
    pthread_exit((void *)str); //or 'return (void *) str;'
}
int main(void){
    pthread_t t_id;
    void * retFromThread; //This must be a pointer to void!
    pthread_create(&t_id, NULL, my_fun, (void *)&t_id); //Create
    pthread_join(t_id,&retFromThread); // wait thread
    // We must cast the returned value!
    printf("Thread %ld returned '%s'\n",t_id,(char *)retFromThread);
}
```

```c
#include <stdio.h> <pthread.h> <unistd.h> //threadJoin2.c
void * my_fun(void *param){
    printf("This is a thread that received %d\n", *(int *)param);
    return (void *)3;
}
int main(void){
    pthread_t t_id;
    int arg=10, retval;
    pthread_create(&t_id, NULL, my_fun, (void *)&arg);
    printf("Executed thread with id %ld\n",t_id);
    sleep(3);
    pthread_join(t_id, (void **)&retval); //A pointer to a void pointer
    printf("retval=%d\n", retval);
}
```

Notiamo che in questi due esempi abbiamo usato come valore di ritorno delle costanti, e per questo motivo non viene riscontrato nessun problema.
Nel caso in cui invece usiamo delle variabili come valori di ritorno si possono riscontrare problemi, infatti le variabili locali vengono inserite nello stack del thread, e questo stack viene eliminato una volta che il thread termina. Infatti in questo esempio viene ricevuto 0 come valore di ritorno:

```c
#include <stdio.h> <pthread.h> <unistd.h> //threadJoin3.c
void * my_fun(void *param){
    printf("This is a thread that received %d\n", *(int *)param);
    int i = *(int *)param * 2; //Local variable ceases to exist!
    return (void *)&i;
}
int main(void){
    pthread_t t_id;
    int arg=10, retval;
    pthread_create(&t_id, NULL, my_fun, (void *)&arg);
    printf("Executed thread with id %ld\n",t_id);
    sleep(3);
    pthread_join(t_id, (void **)&retval); //A pointer to a void pointer
    printf("retval=%d\n", retval);
}
```

Dunque bisogna sempre restituire come valore di ritorno delle costanti, oppure delle strutture già esistenti fuori dal thread.

Come abbiamo visto prima, alla creazione di un thread passiamo la struttura $pthread_attr_t$, che specifica gli attributi con cui creare il thread (ovvero quelli che abbiamo appena visto). Questa struttura, analogamente a quella utilizzata per gestire le maschere di segnale, è un oggetto utilizzato solo quando viene creato un thread ed è quindi indipendente da esso (se cambia dopo la creazione del thread, gli attributi del thread non cambiano).

La struttura deve essere inizializzata, il che imposta tutti gli attributi al loro valore predefinito. Una volta utilizzata e non più necessaria, la *struct* deve essere distrutta.
I vari attributi della struttura possono e devono essere modificati individualmente con alcune funzioni dedicate:

- $\int pthread_attr_init(pthread_attr_t\ *attr)$
  Inizializza la struttura con tutti gli attributi default.
- $\int pthread_attr_destroy(pthread_attr_t\ *attr)$
  Distrugge la struttura.
- $\int pthread_attr_setxxxx(pthread_attr_t\ *attr,\ params)$
  Imposta un certo attributo ad un certo valore.
- $\int pthread_attr_getxxxx(const\ \ pthread_attr_t\ *attr,\ params)$
  Ottiene un certo attributo

xxxx indica cosa andare a “toccare”:

- $\ldots detachstate(pthread_attr_t\ *attr,\ \int detachstate)$
  Imposta lo stato di separazione del thread. Può essere impostato su $PTHREAD_CREATE_JOINABLE$ o $PTHREAD_CREATE_DETACHED$.
- $\ldots sigmask_np(pthread_attr_t\ *attr, const\ \ sigset_t\ *sigmask)$
  Imposta una maschera dei segnali per il thread
- $\ldots affinity_np(\ldots)$
  Imposta l'attributo CPU affinity mask. L'attributo CPU affinity mask (maschera di affinità della CPU) è un attributo utilizzato per specificare l'affinità di un thread rispetto ai processori o ai core di processore disponibili in un sistema multiprocessore. L'affinità della CPU controlla su quali processori o core di processore un thread può essere pianificato per l'esecuzione.
- $\ldots guardsize(\ldots)$
  Imposta la dimensione della zona di guardia per il thread. La zona di guardia è una regione di memoria utilizzata per rilevare overflow dello stack.
- $\ldots inheritsched(\ldots)$
  Specifica se il thread eredita la politica di scheduling e i parametri dal thread chiamante o se vengono specificati separatamente.
- $\ldots schedparam(\ldots)$
  Imposta i parametri di scheduling per il thread, come la priorità.
- $\ldots schedpolicy(\ldots)$
  Imposta la politica di scheduling per il thread.
- altri

Ecco un esempio in cui inizializziamo un thread con attributi diversi:

```c
#include <stdio.h> <pthread.h> <unistd.h> //threadAttr.c
void * my_fun(void *param){
    printf("This is a thread that received %d\n", *(int *)param);return (void*)3;
}
int main(void){
    pthread_t t_id; pthread_attr_t attr;
    int arg=10, detachSTate;
    pthread_attr_init(&attr);
    pthread_attr_setdetachstate(&attr,PTHREAD_CREATE_DETACHED); //Set detached
    pthread_attr_getdetachstate(&attr,&detachSTate); //Get detach state
    if(detachSTate == PTHREAD_CREATE_DETACHED) printf("Detached\n");
    pthread_create(&t_id, &attr, my_fun, (void *)&arg);
    printf("Executed thread with id %ld\n",t_id);
    pthread_attr_setdetachstate(&attr,PTHREAD_CREATE_JOINABLE); //Inneffective
    sleep(3); pthread_attr_destroy(&attr);
    int esito = pthread_join(t_id, (void **)&detachSTate);
    printf("Esito '%d' is different 0\n", esito);
}
```

Quando viene inviato un segnale ad un processo non si può sapere quale thread del processo andrà a gestirlo. Per evitare problemi o comportamenti inattesi, è importante gestire correttamente le maschere dei segnali dei singoli thread, per fare in modo di essere sicuri che un particolare segnale venga gestito da uno specifico thread, con

$$pthread_attr_setsigmask_np(pthread_attr_t\ *attr, const\ \ sigset_t\ *sigmask)$$

la quale usa *sigmask* per impostare la maschera dei segnali nella struttura *attr*.
Nonostante questo, in genere l'esito di un segnale è "comune" a tutto il processo.

È poi possibile usare funzioni come $sigwait()$ e $sigwaitinfo()$ per gestire l'attesa di un segnale. Con $sigwait()$ il thread si mette in attesa fino a che un segnale (tra quelli presenti nella maschera) non viene ricevuto.

Infine, è possibile inviare un segnale ad un thread specifico usando

$$\int\ \ pthread_kill(pthread_t\ \ thread, \int\ \ sig);$$

Mutex

Quando eseguiamo un programma con più thread, essi condividono alcune risorse, tra le quali le variabili globali. Se entrambi i thread accedono ad una sezione di codice condivisa ed hanno la necessità di accedervi in maniera esclusiva allora dobbiamo instaurare una **sincronizzazione**. I risultati, altrimenti, potrebbero essere inaspettati.
Ecco un esempio di problema di sincronizzazione:

```c
#include <pthread.h> <stdlib.h> <unistd.h><stdio.h>      //syncProblem.c
pthread_t tid[2];
int counter = 0;
void *thr1(void *arg){
    counter = 1;
    printf("Thread 1 has started with counter %d\n",counter);
    for (long unsigned i = 0; i < (0x00FF0000); i++); //wait some cycles
    counter += 1;
    printf("Thread 1 expects 2 and has: %d\n", counter);
}
void *thr2(void *arg){
    counter = 10;
    printf("Thread 2 has started with counter %d\n",counter);
    for (long unsigned i = 0; i < (0xFFF0000); i++); //wait some cycles
    counter += 1;
    printf("Thread 2 expects 11 and has: %d\n", counter);
}
```

```c
void main(void){
    pthread_create(&(tid[0]), NULL, thr1,NULL);
    pthread_create(&(tid[1]), NULL, thr2,NULL);
    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);
}
```

Una possibile soluzione a questo tipo di problemi sono i **semafori mutex**.
I mutex sono dei semafori imposti ai thread. Essi possono proteggere una determinata sezione di codice, consentendo ad un thread di accedervi in maniera esclusiva fino allo sblocco del semaforo. Ogni thread che vorrà accedere alla stessa sezione di codice dovrà aspettare che il semaforo sia sbloccato, andando in *sleep* fino alla sua prossima schedulazione.
I mutex vanno inizializzati e poi assegnati ad una determinata sezione di codice. Il blocco e lo sblocco di un mutex è manuale.
È importante far notare che i mutex non regolano l'accesso alla memoria o alle variabile, ma solo a porzioni di codice.

Per utilizzare i **mutex** dobbiamo **crearli** con la funzione $pthread_mutex_init()$:

$$\int pthread_mutex_init(pthread_mutex_t\ *restrict\ mutex,\ const\ pthread_mutexattr_t\ *restrict\ attr)$$

e dobbiamo passare come parametri gli attributi di inizializzazione *attr* e ci viene ritornato l'identificativo del mutex *mutex*. Per far condividere un mutex tra più thread possiamo definirlo come variabile globale oppure possiamo passarlo come parametro durante la creazione del thread.

Per **distruggere/eliminare un mutex** si utilizza la system call $pthread_mutex_destroy()$:

$$\int pthread_mutex_destroy(pthread_mutex_t\ *mutex)$$

a cui passiamo l'identificativo del mutex.

```c
#include <pthread.h>                                    //createMutex.c
pthread_mutex_t lock;
int main(void){
    pthread_mutex_init(&lock, NULL); // Create mutex with default attrs
    pthread_mutex_destroy(&lock); // Destroy mutex (it can be re-init)
}
```

Per **utilizzare le funzionalità di un mutex** si usano le system call:

$$\int pthread_mutex_lock(pthread_mutex_t\ *mutex)$$
$$\int pthread_mutex_unlock(pthread_mutex_t\ *mutex)$$

le quali appunto implementano il meccanismo di sincronizzazione.
Dopo essere stato creato, un mutex deve essere bloccato per essere efficace. Non appena un thread lo blocca, un altro thread deve attendere che venga sbloccato prima di procedere al suo blocco.
Quando si richiama il blocco, un thread attende che il mutex sia libero e poi lo blocca.
I mutex devono essere usati correttamente perchè altrimenti non ha senso utilizzarli.

```c
#include <pthread.h>

pthread_mutex_t lock;
void * thread(void *){
    pthread_mutex_lock(&lock);
    /* Protected section of code */
    pthread_mutex_unlock(&lock);
}
```

Ecco un esempio di utilizzo di mutex per proteggere la variabile *counter* che è condivisa tra più thread:

```c
#include <pthread.h> <stdlib.h> <unistd.h><stdio.h>     //mutex.c

pthread_mutex_t lock;
pthread_t tid[2];
int counter = 0;

void* thr1(void* arg){
    pthread_mutex_lock(&lock);
    counter = 1;
    printf("Thread 1 has started with counter %d\n",counter);
    for (long unsigned i = 0; i < (0x00FF0000); i++);
    counter += 1;
    pthread_mutex_unlock(&lock);
    printf("Thread 1 expects 2 and has: %d\n", counter);
}...
```

```c
void * thr2(void* arg){
    pthread_mutex_lock(&lock);
    counter = 10;
    printf("Thread 2 has started with counter %d\n",counter);
    for (long unsigned i = 0; i < (0xFFF0000); i++);
    counter += 1;
    pthread_mutex_unlock(&lock);
    printf("Thread 2 expects 11 and has: %d\n", counter);
}
int main(void){
    pthread_mutex_init(&lock, NULL);
    pthread_create(&(tid[0]), NULL, thr1,NULL);
    pthread_create(&(tid[1]), NULL, thr2,NULL);
    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);
    pthread_mutex_destroy(&lock);
}
```

Esistono 4 tipologie di mutex che differiscono per il comportamento in 3 situazioni:

- Blocca quando già bloccato
  Ovvero la situazione in cui un thread esegue la *lock* dopo averla già fatta (senza aver fatto *unlock* nel mentre)
- Sblocca quando bloccato da altri
  Ovvero la situazione in cui un thread esegue la *unlock* quando il mutex è bloccato da un altro thread
- Sblocca se già sbloccato
  Ovvero la situazione in cui un thread esegue la *unlock* dopo averla già fatta (senza aver fatto *lock* nel mentre)

Le 4 tipologie sono le seguenti:

- *PTHREAD _ MUTEX _ NORMAL*
- *PTHREAD _ MUTEX _ ERRORCHECK*
- *PTHREAD _ MUTEX _ RECURSIVE*
- *PTHREAD _ MUTEX _ DEFAULT*

	Blocca quando già bloccato	Sblocca quando bloccato da altri	Sblocca se già sbloccato
PTHREAD_MUTEX_NORMAL	deadlock	undefined	undefined
PTHREAD_MUTEX_ERRORCHECK	error	error	error
PTHREAD_MUTEX_RECURSIVE	Lock count++ so that it requires the same number of unlocks	error	error
PTHREAD_MUTEX_DEFAULT	undefined	undefined	undefined

Ecco un esempio in cui capitano queste situazioni:

```c
#include <pthread.h> <stdlib.h> <unistd.h><stdio.h>      //recursive.c

pthread_mutex_t lock;
pthread_t tid[2];
int counter = 0;

void* thr1(void* arg){
    pthread_mutex_lock(&lock);
    pthread_mutex_lock(&lock);
    counter = 1;
    printf("Thread 1 has started with counter %d\n",counter);
    for (long unsigned i = 0; i < (0x00FF0000); i++);
    counter += 1;
    pthread_mutex_unlock(&lock);
    printf("Thread 1 expects 2 and has: %d\n", counter);
    pthread_mutex_unlock(&lock);
}...
```

```c
void* thr2(void* arg){
    pthread_mutex_lock(&lock); pthread_mutex_lock(&lock);
    counter = 10;
    printf("Thread 2 has started with counter %d\n",counter);
    for (long unsigned i = 0; i < (0xFFF0000); i++);
    counter += 1;
     pthread_mutex_unlock(&lock);
    printf("Thread 2 expects 11 and has: %d\n", counter);
}
void main(){
    pthread_mutexattr_t attr;
    pthread_mutexattr_settype(&attr,PTHREAD_MUTEX_RECURSIVE);
    pthread_mutex_init(&lock, &attr);
    pthread_create(&(tid[0]), NULL, thr1,NULL);
    pthread_create(&(tid[1]), NULL, thr2,NULL);
    pthread_join(tid[0], NULL); pthread_join(tid[1], NULL);
    pthread_mutex_destroy(&lock);
}
```

In conclusione:

- I “thread” sono una sorta di “processi leggeri” che permettono di eseguire funzioni “in concorrenza” in modo più semplice rispetto alla generazioni di processi veri e propri (forking).
- I “MUTEX” sono un metodo semplice ma efficace per eseguire sezioni critiche in processi multithread.
  È importante limitare al massimo la sezione critica utilizzando lock/unlock per la porzione di codice più piccola possibile, e solo se assolutamente necessario (per esempio quando possono capitare accessi concorrenti).